```
fn main() -> i32 {
    println("Hello, Amiga!")
    return 0
}
```

NOVUS

Programmer's Reference Guide

A modern language for the Amiga

Version 0.1 | December 2025

For Amiga 68020+ | AmigaOS 2.0+

Novus Programmer's Reference Guide

Version 0.1 — December 2025

Novus is an open source project.
Visit `https://github.com/barryw/Novus` for source code and updates.

Typeset in Latin Modern using LaTeX.

Contents

Foreword

The Amiga was ahead of its time. In 1985, while other personal computers offered beeps and static screens, the Amiga delivered pre-emptive multitasking, dedicated graphics and audio hardware, and a sophisticated operating system that wouldn't be matched by mainstream competitors for nearly a decade.

For many of us, the Amiga wasn't just a computer—it was where we learned to code, to create, to push hardware to its limits. The demoscene flourished. Games achieved visual and audio fidelity that seemed impossible. A generation of programmers cut their teeth on 68000 assembly, crafting tight loops that squeezed every cycle from the machine.

Then the platform faded from the mainstream. But it never truly died.

Today, a vibrant community keeps the Amiga alive. Original hardware is lovingly maintained and expanded. FPGA recreations achieve cycle-perfect accuracy. Emulation lets new generations experience what made these machines special. And remarkably, people still write software for them—not out of obligation, but out of genuine passion.

Yet the tools available to modern Amiga developers are largely the same ones we used thirty years ago. C compilers like VBCC and GCC produce excellent code, but the languages themselves haven't evolved. The patterns and safety mechanisms that modern systems programming takes for granted—sum types, pattern matching, resource management without garbage collection—remain out of reach.

Novus aims to change that.

This is not an attempt to modernize the Amiga by hiding what makes it special. Quite the opposite. Novus is designed to *embrace* the Amiga's architecture: its custom chips, its message-passing OS, its elegant hardware. The goal is to give developers modern ergonomics and safety while preserving—even enhancing—the direct hardware access that makes Amiga programming rewarding.

Whether you're a veteran who remembers waiting for Workbench to load from floppy, or a newcomer curious about this legendary platform, we hope Novus helps you write code that's safer, clearer, and more enjoyable—without sacrificing an ounce of the control that Amiga development demands.

New code for classic machines. Welcome to Novus.

December 2025

Chapter 1

Introduction

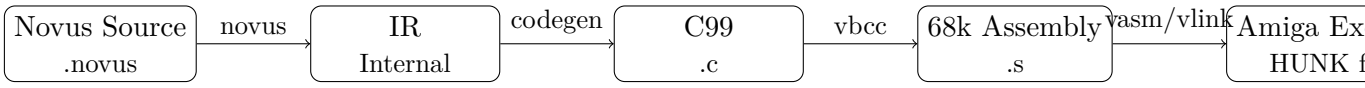
1.1 What is Novus?

Novus is a systems programming language designed specifically for the Amiga 68k platform. It transpiles to C99, which VBCC then compiles to native 68020+ machine code. The result is executables that integrate deeply with AmigaOS and run on real hardware and accurate emulators alike.

The name comes from the Latin word for "new"—reflecting the project's mission to bring modern language design to classic machines. Novus is not a port of an existing language, nor is it a lowest-common-denominator cross-platform tool. Every design decision is made with the Amiga in mind.

1.1.1 The Compilation Pipeline

Understanding how Novus code becomes an Amiga executable helps you debug problems and optimize performance. The pipeline has four stages:



1. **Novus Compiler**: Parses your source, performs type checking, generates intermediate representation (IR)
2. **Code Generation**: Converts IR to clean, readable C99 code
3. **VBCC**: Compiles C to optimized 68k assembly
4. **VASM/VLINK**: Assembles and links into an AmigaOS HUNK executable

You can examine any stage:

```
# Emit IR to see the compiler's internal representation
novus compile myfile.novus -o out --emit-ir

# Emit assembly to see exactly what instructions are generated
novus compile myfile.novus -o out --emit-asm
```

1.1.2 Key Characteristics

Native Performance Novus transpiles to C99, leveraging VBCC's mature 68k code generator to produce Amiga executables. There is no interpreter, no virtual machine, no runtime overhead beyond what you explicitly request.

Modern Safety The type system catches errors at compile time. `Result` and `Option` types make error handling explicit. Resource cleanup is deterministic via the `Drop` trait and `defer` blocks. You get safety without garbage collection.

Direct Hardware Access Novus doesn't hide the Amiga's hardware behind abstractions you can't escape. Custom chip registers, Exec library calls, copper lists, blitter operations—all are first-class citizens with typed, safe interfaces.

Readable Output The generated C code is clean and readable. When you need to understand what the compiler is doing—and on the Amiga, you often do—the output won't fight you.

Amiga-Native Async The `async/await` model maps directly to AmigaOS primitives: Exec signals and message ports. No hidden threads, no runtime scheduler—just the OS facilities the Amiga provides.

1.1.3 A First Comparison

Here's a simple function that opens a file and reads its size. First, the C version:

```c
// C: Opening a file and getting its size
LONG get_file_size(const char *path) {
    BPTR lock = Lock(path, ACCESS_READ);
    if (!lock) {
        return -1;  // Caller must check return value!
    }

    struct FileInfoBlock *fib = AllocMem(sizeof(struct
        FileInfoBlock),
                                         MEMF_PUBLIC);
    if (!fib) {
        UnLock(lock);  // Must remember to clean up!
        return -1;
    }

    LONG size = -1;
    if (Examine(lock, fib)) {
        size = fib->fib_Size;
    }

    FreeMem(fib, sizeof(struct FileInfoBlock));  // Must match
        size!
    UnLock(lock);  // Must remember cleanup order!
    return size;
}
```

Now the same function in Novus:

```
// Novus: Opening a file and getting its size
fn get_file_size(path: &str) -> Result<i32, DosError> {
    let lock = Lock::new(path, ACCESS_READ)?  // ? propagates
        errors
    let fib = FileInfoBlock::alloc()?

    lock.examine(&fib)?
    return Result::Ok(fib.size())
}  // lock and fib automatically cleaned up via Drop
```

The Novus version is shorter, but more importantly:

- The return type `Result<i32, DosError>` makes failure explicit—callers *must* handle it
- The `?` operator propagates errors automatically, no forgotten checks
- Resources are cleaned up automatically when they go out of scope
- No size mismatches possible—`FileInfoBlock` knows its own size

Coming from C

In C, every function that can fail returns a value that might be an error. You must remember to check it. You must remember to clean up. You must match every allocation size.
In Novus, the type system enforces these rules. A `Result` *must* be handled. Resources with `Drop` *will* be cleaned up. The compiler tracks sizes for you.

1.2 Why Novus Exists

The Amiga platform has capable C compilers. VBCC, in particular, generates excellent code and remains actively maintained. So why create a new language?

1.2.1 The Limits of C

C was revolutionary in 1972. It remains useful today precisely because it's minimal—a portable assembler that gets out of your way. But that minimalism has costs:

- **No sum types.** Representing "this value is either an error or a result" requires conventions that the compiler can't enforce.
- **No built-in error handling.** Return codes are easily ignored. Exceptions don't exist. Every function call is an opportunity for silent failure.
- **Manual resource management.** Matching every `AllocMem` with a `FreeMem`, every `OpenLibrary` with a `CloseLibrary`, every `Lock` with an `UnLock`—across all control flow paths, including error cases.
- **Primitive module system.** Header files and `#include` are textual substitution. Name collisions, include order dependencies, and compile time explosion are facts of life.
- **No metaprogramming.** The preprocessor is powerful but crude. Anything beyond simple macros quickly becomes write-only code.

1.2.2 Real Bugs from Real Software

These aren't theoretical concerns. They're the source of real bugs in real Amiga software:

The Forgotten `UnLock`:

```
BOOL check_file(const char *path) {
    BPTR lock = Lock(path, ACCESS_READ);
    if (!lock) return FALSE;

    struct FileInfoBlock fib;
    if (!Examine(lock, &fib)) {
        return FALSE;  // BUG: lock leaked!
```

```
    }

    UnLock(lock);
    return TRUE;
}
```

Every early return is an opportunity to forget cleanup. In a large function with multiple resources, the combinations explode.

The Ignored Return Value:

```
void process_data(void) {
    Open("output.dat", MODE_NEWFILE);   // BUG: return value
        ignored!
    Write(file, data, size);            // file is uninitialized
}
```

C happily compiles this. The result: random corruption.

The Mismatched Size:

```
struct MyData *data = AllocMem(sizeof(struct MyData),
    MEMF_PUBLIC);
// ... 500 lines later, after the struct changed ...
FreeMem(data, sizeof(struct OtherData));  // BUG: wrong size!
```

Memory corruption. Guru meditation. Hours of debugging.

1.2.3 How Novus Prevents These Bugs

Each of these bugs is impossible in Novus:

The Forgotten `UnLock`:

```
fn check_file(path: &str) -> Result<bool, DosError> {
    let lock = Lock::new(path, ACCESS_READ)?

    let fib = FileInfoBlock::alloc()?
    lock.examine(&fib)?

    return Result::Ok(true)
}  // lock.drop() called automatically on ALL paths
```

The Ignored Return Value:

```
fn process_data() -> Result<(), DosError> {
    // This won't compile - Result must be handled:
    // let _ = open("output.dat", MODE_NEWFILE)  // Error: unused
        Result

    let file = open("output.dat", MODE_NEWFILE)?  // Must use ?
        or match
    write(&file, data, size)?
    return Result::Ok(())
}
```

The Mismatched Size:

```
fn allocate_data() -> Result<Allocation<MyData>, ExecError> {
```

```
    let data = Allocation::<MyData>::alloc(MEMF_PUBLIC)?
    return Result::Ok(data)
}  // data.drop() frees with correct size automatically
```

1.2.4 Modern Solutions, Amiga Constraints

Languages like Rust address these problems elegantly. But Rust targets modern systems with megabytes of RAM, gigahertz CPUs, and operating systems that provide virtual memory and threads. Its runtime assumptions don't map to the Amiga.

Novus takes a different approach: provide modern ergonomics within Amiga constraints.

- `Result<T, E>` and `Option<T>` are zero-cost at runtime—they're just tagged unions that the compiler understands.
- The `Drop` trait compiles to cleanup code inserted at scope exit. No hidden allocations, no unwinding tables.
- Pattern matching compiles to efficient jump tables or comparison chains as appropriate.
- The module system is compile-time only. No runtime symbol resolution, no dynamic linking overhead.
- Fixed-point math uses inline 68020+ instructions, not floating-point emulation.

The result is a language that *feels* modern but *runs* like hand-tuned assembly.

1.3 Design Philosophy

Novus follows several core principles that inform every language decision.

1.3.1 Explicit Over Implicit

If something allocates memory, the code should say so. If something can fail, the type should reflect it. If a function has side effects, they should be visible at the call site.

```
// Allocation is explicit - you know this allocates
var buffer = MemoryBlock::alloc(1024, MEMF_CHIP)?

// Failure is explicit - Result type, can't ignore
let file = open("data.dat", MODE_OLDFILE)?

// Mutation is explicit at the call site
process(&var player)  // Clearly modifying player
```

This doesn't mean verbose—Novus has type inference, sensible defaults, and concise syntax. But it means no hidden costs, no action at a distance, no "magic" that obscures what the machine is actually doing.

Coming from C

In C, `process(player)` might modify `player` or it might not—you have to read the function signature.
In Novus, `process(&player)` is read-only. `process(&var player)` mutates. The call site tells you.

1.3.2 Predictable Performance

On a 7 MHz 68000, every cycle matters. On a 50 MHz 68060, you have more headroom—but you still can't afford hidden allocations in a tight loop or cache-hostile memory access patterns.

Novus gives you control. The compiler won't secretly box values, won't insert invisible runtime checks in release builds, won't generate code that's impossible to reason about.

Consider this function:

```
fn sum_array(data: &[i32]) -> i32 {
    var total: i32 = 0
    for value in data {
        total = total + value
    }
    return total
}
```

The generated 68k assembly is exactly what you'd expect:

```
sum_array:
    moveq   #0,d0           ; total = 0
    move.l  d2,-(sp)        ; save d2
    move.l  (a0),d2         ; d2 = slice length
    addq.l  #4,a0           ; a0 = data pointer
.loop:
    subq.l  #1,d2           ; decrement counter
    bmi.s   .done           ; done if negative
    add.l   (a0)+,d0        ; total += *data++
    bra.s   .loop
.done:
    move.l  (sp)+,d2        ; restore d2
    rts
```

No hidden function calls. No allocations. Just a tight loop.

1.3.3 Respect the Machine

The Amiga isn't a Unix workstation. It has custom chips that do things no other hardware can do. Its operating system uses message passing and signals, not files and processes. Its memory model is flat and physical.

Novus doesn't pretend otherwise. The standard library wraps AmigaOS idiomatically—`Result` types for operations that can fail, handles for resources that need cleanup—but it doesn't try to be POSIX. Amiga code should look like Amiga code.

```
// Copper lists, typed and safe
var builder = CopperListBuilder::new()
builder.wait(100, 0)?
builder.move_color(0, rgb12(15, 0, 0))?  // Set background red
let copper_list = builder.build()?
copper_list.activate()

// Blitter operations
blitter.blit_fill(&var bitmap, x, y, width, height, color)?

// Message ports for IPC
let port = MsgPort::create("myport")?
let signal = port.signal_bit()
let sigs = Wait(1 << signal)
```

1.3.4 Fail Fast, Fail Clearly

When something goes wrong, you should know immediately and know exactly what happened. Compile-time errors are better than runtime errors. Runtime errors with clear messages are better than silent corruption.

In debug builds, Novus checks array bounds, validates pointers, and panics with meaningful diagnostics:

```
PANIC at src/player.novus:42
  Array index out of bounds: index 10, length 8

  Stack trace:
    update_score (src/player.novus:42)
    game_loop (src/main.novus:156)
    main (src/main.novus:12)
```

In release builds, those checks can be removed via `-safety-level`—but the type system still prevents the errors that checks would catch.

1.4 A Taste of Novus

Before diving into the full language, let's look at a complete program. This opens an Intuition window, draws a message, and waits for the user to close it:

```
// hello_window.novus - A complete Novus program
from std::core import Result
from std::ffi::intuition import OpenWindowTags, CloseWindow
from std::ffi::graphics import SetAPen, Move, Text
from std::ffi::exec import Wait
from std::ffi::amiga_consts import (
    WA_Width, WA_Height, WA_Title, WA_Flags, WA_IDCMP,
    WFLG_CLOSEGADGET, WFLG_DRAGBAR, WFLG_ACTIVATE,
    IDCMP_CLOSEWINDOW, TAG_DONE
)

pub fn main() -> i32 {
    // Open a window with the close gadget
    var window: *Window = null

    unsafe {
        window = OpenWindowTags(null,
            WA_Width, 320,
            WA_Height, 100,
            WA_Title, "Hello from Novus!",
            WA_Flags, WFLG_CLOSEGADGET | WFLG_DRAGBAR |
                WFLG_ACTIVATE,
            WA_IDCMP, IDCMP_CLOSEWINDOW,
            TAG_DONE)
    }

    if window == null {
        return 1  // Window open failed
    }
```

```
    // Draw text in the window
    unsafe {
        let rp = window.RPort
        SetAPen(rp, 1)            // Color 1 (usually white)
        Move(rp, 20, 50)          // Position cursor
        Text(rp, "Welcome to Novus!", 17)
    }

    // Wait for close gadget
    unsafe {
        let signal = 1u32 << window.UserPort.mp_SigBit
        Wait(signal)
    }

    // Clean up
    unsafe {
        CloseWindow(window)
    }

    return 0
}
```

This is deliberately written in "raw" style, calling AmigaOS functions directly with `unsafe` blocks. Later chapters show the safer stdlib wrappers, but this demonstrates that Novus gives you full access to the platform.

Coming from C

Notice what's different from C:

- No `#include` files—`from...import` brings in what you need
- No semicolons required (they're optional)
- `null` instead of `NULL`
- `unsafe {}` blocks mark where you're bypassing safety checks
- Return type comes after the function name: `fn main() -> i32`

What's the same:

- Same AmigaOS functions: `OpenWindowTags`, `Wait`, etc.
- Same structures: `Window`, `RastPort`
- Same constants: `WA_Width`, `IDCMP_CLOSEWINDOW`
- Same calling convention and memory layout

1.5 What Novus Doesn't Do

Honesty about limitations helps you make informed decisions. Novus is not a general-purpose language. It's designed for Amiga development, and it makes trade-offs accordingly.

1.5.1 No Garbage Collection

Novus uses deterministic resource management via RAII (the `Drop` trait). When a value goes out of scope, its destructor runs immediately. This is predictable and has zero overhead—but it means you must think about ownership.

Cyclic data structures require extra care. Reference counting (via `Rc<T>`) is available when needed, but it's explicit, not automatic.

1.5.2 No Exceptions

Error handling uses `Result<T, E>` and the `?` operator, not exceptions. This means:

- No invisible control flow jumps
- No unwinding tables taking up code space
- Errors are always visible in function signatures

But it also means you can't `catch` an error from deep in a call stack without propagating it explicitly through each level.

1.5.3 No Hidden Threads

The Amiga's multitasking is cooperative and explicit. Novus's `async/await` compiles to state machines driven by Exec signals—there's no hidden thread pool, no automatic parallelism.

If you want parallel execution, you create Exec tasks explicitly. Novus gives you tools to do this safely, but it doesn't do it behind your back.

1.5.4 68020 Minimum

Novus targets 68020+ processors. The 68000/68010 lacks instructions that the compiler relies on (32-bit multiply/divide, bitfield operations). If you're targeting a stock A500 or A1000, Novus isn't the right choice—use C with VBCC directly.

This isn't a significant limitation in practice. Most Amiga users have accelerators, and the A1200/A4000 shipped with 68020/68040.

1.6 Target Audience

This guide assumes you're a working programmer. You should be comfortable with:

- Systems programming concepts: pointers, memory allocation, stack vs. heap
- Basic 68k architecture: registers, addressing modes, the supervisor/user split
- AmigaOS fundamentals: libraries, Exec, DOS, the message-passing model

Prior experience with Rust, Swift, or other modern systems languages will make Novus feel familiar, but it's not required. If you've written C for the Amiga, you have all the background you need.

1.7 How to Read This Guide

The guide is organized in layers:

Part I: Language Fundamentals covers the core language—syntax, types, control flow, functions, and modules. Read this first.

Part II: Standard Library documents the `std` packages: collections, strings, memory management, and Amiga OS bindings.

Part III: Advanced Topics covers async programming, inline assembly, FFI with existing C code, and performance optimization.

Appendices provide quick references, grammar summaries, and migration guides from C.

Code examples are designed to be complete and runnable. When a snippet requires context, we provide it. When we omit boilerplate, we say so.

1.8 Showcase: Copper Rainbow

To close this introduction, here's a program that demonstrates what makes Novus special for Amiga development. It uses the copper coprocessor to create a smooth rainbow gradient across the screen—something that would be impossible on any other platform.

```
// copper_rainbow.novus - A visual showcase
from std::core import Result
from std::graphics::copper import CopperListBuilder,
    CopperListHandle, CopperError
from std::hardware::registers import rgb12
from std::os::timer import delay, Duration
from std::ffi::graphics import WaitTOF

/// Interpolate between two colors
fn lerp_color(c1: u16, c2: u16, t: u16) -> u16 {
    let r1 = (c1 >> 8) & $F
    let r2 = (c2 >> 8) & $F
    let g1 = (c1 >> 4) & $F
    let g2 = (c2 >> 4) & $F
    let b1 = c1 & $F
    let b2 = c2 & $F

    let r = r1 + ((r2 - r1) * t) / 256
    let g = g1 + ((g2 - g1) * t) / 256
    let b = b1 + ((b2 - b1) * t) / 256

    return rgb12((u8)r, (u8)g, (u8)b)
}

/// Get rainbow color for position 0-255
fn rainbow(pos: u8) -> u16 {
    let p = (u16)pos
    let segment = p / 43
    let offset = (p % 43) * 6

    match segment {
        0 => lerp_color(rgb12(15,0,0), rgb12(15,15,0), offset),
        1 => lerp_color(rgb12(15,15,0), rgb12(0,15,0), offset),
```

```
        2 => lerp_color(rgb12(0,15,0), rgb12(0,15,15), offset),
        3 => lerp_color(rgb12(0,15,15), rgb12(0,0,15), offset),
        4 => lerp_color(rgb12(0,0,15), rgb12(15,0,15), offset),
        _ => lerp_color(rgb12(15,0,15), rgb12(15,0,0), offset),
    }
}

/// Build copper list with rainbow gradient
fn build_rainbow() -> Result<CopperListHandle, CopperError> {
    var builder = CopperListBuilder::new()

    var line: u16 = 0
    while line < 200 {
        let scanline = 44 + line
        let color_pos = (line * 256) / 200
        let color = rainbow((u8)color_pos)

        builder.wait(scanline, 0)?
        builder.move_color(0, color)?

        line = line + 2
    }

    builder.wait(244, 0)?
    builder.move_color(0, rgb12(0, 0, 0))?

    return builder.build()
}

pub fn main() -> i32 {
    unsafe { WaitTOF() }

    match build_rainbow() {
        Result::Ok(copper_list) => {
            unsafe { copper_list.activate() }
            let _ = delay(Duration::secs(5))
            return 0  // Copper list cleaned up by Drop
        },
        Result::Err(_) => return 1
    }
}
```

This 70-line program:

- Uses `Result` for error handling throughout
- Relies on RAII—the copper list restores the display when dropped
- Talks directly to the copper hardware through safe abstractions
- Compiles to tight, efficient 68k code

Run it on real hardware or in FS-UAE to see a smooth, colorful gradient scroll across your screen. This is what Novus is for: making the Amiga's unique capabilities accessible with modern safety and ergonomics.

Let's begin.

Chapter 2

Getting Started

> *"The journey of a thousand miles begins with a single step."*
> —Lao Tzu

This chapter walks you through installing the Novus compiler, writing your first program, and understanding the compilation pipeline. By the end, you'll have a working executable running on your Amiga (or emulator).

2.1 Quick Start

If you already have Novus installed, here's the fastest path to "Hello, Amiga!":

1. Create `hello.novus`:

```
from std::io::file import println

pub fn main() -> i32 {
    println("Hello, Amiga!")
    return 0
}
```

2. Compile:

```
novus compile hello.novus -o hello
```

3. Copy to your Amiga and run:

```
1.Workbench:> hello
Hello, Amiga!
```

That's it! The rest of this chapter explains the prerequisites, installation, and toolchain in detail.

2.2 Prerequisites

Before installing Novus, ensure you have:

- **.NET 9.0 or later** — The Novus compiler is written in C# and requires the .NET runtime. Download from `https://dotnet.microsoft.com`
- **VBCC toolchain** — Novus uses VBCC's assembler (`vasmm68k_mot`) and linker (`vlink`) to produce Amiga executables. The toolchain is vendored in the repository under `vendor/vbcc`, or you can specify a custom installation via the `-vbcc-path` option.

- **An Amiga or emulator** — To run your compiled programs. FS-UAE, WinUAE, or real hardware all work. A 68020+ configuration with AmigaOS 2.0 or later is required.

2.3 Installation

2.3.1 Platform-Specific Instructions

macOS

1. Install .NET SDK using Homebrew:

```
brew install dotnet
```

2. Clone and build Novus:

```
git clone https://github.com/barryw/Novus.git
cd Novus
dotnet build
```

3. Add to your PATH (optional):

```
export PATH="$PATH:$(pwd)/Novus/bin/Debug/net9.0"
```

Linux

1. Install .NET SDK:

```
# Ubuntu/Debian
sudo apt install dotnet-sdk-9.0

# Fedora
sudo dnf install dotnet-sdk-9.0
```

2. Clone and build:

```
git clone https://github.com/barryw/Novus.git
cd Novus
dotnet build
```

Windows

1. Download and install .NET SDK from `https://dotnet.microsoft.com`
2. Clone the repository (using Git Bash, PowerShell, or WSL):

```
git clone https://github.com/barryw/Novus.git
cd Novus
dotnet build
```

WSL Note: If you prefer Linux tools, Windows Subsystem for Linux works well. Install the Linux .NET SDK inside WSL and compile there.

2.3.2 Verifying Installation

After building, verify everything works:

```
# Check the Novus compiler
./Novus/bin/Debug/net9.0/novus --version

# Check VBCC tools (vendored)
./vendor/vbcc/bin/vasm --version 2>&1 | head -1
./vendor/vbcc/bin/vlink --version 2>&1 | head -1
```

You should see version information for each tool.

2.3.3 Troubleshooting

Problem: `dotnet: command not found`
Solution: .NET SDK is not installed or not in PATH. Reinstall and ensure the install location is in your PATH.

Problem: `VBCC tools not found`
Solution: The vendored VBCC is under `vendor/vbcc`. If using a custom VBCC installation, pass `-vbcc-path /path/to/vbcc` to the compiler.

Problem: Build fails with missing dependencies
Solution: Run `dotnet restore` to fetch NuGet packages.

Coming from C

Setting up a C cross-compiler for Amiga traditionally involves:

1. Downloading VBCC, vasm, and vlink separately
2. Setting `VBCC` environment variable
3. Creating `vc.config` with include/library paths
4. Finding and installing the AmigaOS NDK
5. Setting up include paths for system headers

Novus bundles everything in the repository—no environment variables or configuration files needed. Just `dotnet build` and compile.

2.4 Your First Program

Create a file named `hello.novus`:

```
from std::io::file import println

pub fn main() -> i32 {
    println("Hello, Amiga!")
    return 0
}
```

2.4.1 Line-by-Line Explanation

Line 1: `from std::io::file import println`

This imports the `println` function from the standard library's I/O module. Unlike some languages where printing is built-in, Novus requires explicit imports. This keeps the language core small and makes dependencies visible.

`std::io::file` is the module path:

- `std` — the standard library
- `io` — the I/O subsystem
- `file` — file and console operations

Line 3: `pub fn main() -> i32`

This declares the program entry point. Every Novus program must have exactly one `main` function that:

- Is marked `pub` (public) so the linker can find it
- Takes no parameters
- Returns `i32` — the exit code passed back to AmigaOS

Line 4: `println("Hello, Amiga!")`

Calls the imported `println` function with a string literal. String literals in Novus are null-terminated C-style strings (`*u8`).

Line 5: `return 0`

Returns the exit code. Zero means success; non-zero indicates an error. AmigaOS and the Shell use this to detect failures.

2.4.2 Compiling

Compile to an Amiga executable:

```
novus compile hello.novus -o hello
```

This produces an executable named `hello` in Amiga HUNK format. Without the `-o` flag, the output defaults to `a.out`.

2.4.3 What Happens During Compilation

When you compile, Novus performs these steps:

1. **Parse** — Read `hello.novus` and build an AST
2. **Analyze** — Check types, resolve names, build IR
3. **Import** — Load `std::io::file` and its dependencies
4. **Generate** — Emit C99 code for your program and used stdlib functions
5. **Compile** — VBCC compiles C to 68k object files
6. **Link** — vlink combines objects into a HUNK executable

Use `-v` (verbose) to watch this process:

```
novus compile hello.novus -o hello -v
```

2.4.4 Running on the Amiga

Transfer the executable to your Amiga and run it from the Shell:

```
1.Workbench:> hello
Hello, Amiga!
```

Congratulations—you've run your first Novus program!

2.5 A More Complete Example

Let's look at a program that does more than print a message. This example queries and displays system memory information:

```
// memory_info.novus - Display memory statistics
from std::io::file import print, println
from std::ffi::exec import AvailMem
from std::ffi::amiga_consts import MEMF_CHIP, MEMF_FAST,
    MEMF_TOTAL

pub fn main() -> i32 {
    println("Memory Information")
    println("==================")

    // Query chip RAM (DMA-accessible memory)
    let chip_mem = unsafe { AvailMem(MEMF_CHIP | MEMF_TOTAL) }

    // Query fast RAM (CPU-only memory)
    let fast_mem = unsafe { AvailMem(MEMF_FAST | MEMF_TOTAL) }

    // Display results
    print("Chip RAM: ")
    print_kb(chip_mem)
    println(" KB")

    print("Fast RAM: ")
    print_kb(fast_mem)
    println(" KB")

    print("Total:    ")
    print_kb(chip_mem + fast_mem)
    println(" KB")

    return 0
}

fn print_kb(bytes: u32) {
    let kb = bytes / 1024u32
    print_number(kb)
}

fn print_number(n: u32) {
    if n == 0 { return }
    if n >= 10 { print_number(n / 10u32) }
    let digit: u8 = (u8)(n % 10u32) + 48u8
    var buf = [0u8; 2]
    buf[0] = digit
```

```
    print(&buf[0])
}
```

This example demonstrates:

- **Multiple imports** from different modules
- **Unsafe FFI calls** to AmigaOS via `AvailMem`
- **Constants** like `MEMF_CHIP` from the FFI layer
- **Helper functions** for formatting output
- **Arithmetic and type casts**

Coming from C

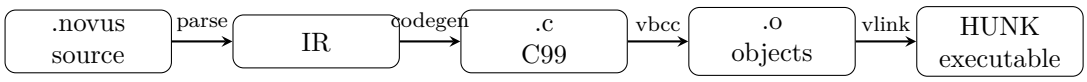
The equivalent C code would look similar, but you'd need to:

- Include `<proto/exec.h>` or set up function prototypes
- Link with `amiga.lib` or use direct library calls
- Deal with library bases (`SysBase`)

In Novus, the FFI layer handles library opening/closing automatically. Just import and call.

2.6 The Compilation Pipeline

Understanding the compilation pipeline helps with debugging and optimization. Here's what happens when you compile:



2.6.1 Stage 1: Parsing and Analysis

The Novus compiler reads your source file, parses it into an Abstract Syntax Tree (AST), then performs semantic analysis:

- Type checking
- Name resolution
- Trait resolution
- Building the Intermediate Representation (IR)

Use `-emit-ir` to see the IR:

```
novus compile hello.novus --emit-ir
```

2.6.2 Stage 2: C Code Generation

The IR is lowered to C99 code. Each function becomes a C function; each type becomes a C struct or typedef. The generated C uses VBCC-specific extensions for 68k calling conventions.

2.6.3 Stage 3: Object File Compilation

VBCC's C compiler (`vbccm68k`) compiles the C code to 68k object files. Each function is compiled separately for incremental builds.

Use `-emit-asm` to see the generated assembly:

```
novus compile hello.novus --emit-asm
```

This is useful for understanding performance characteristics or debugging codegen issues.

2.6.4 Stage 4: Linking

`vlink` combines all object files—your code, the runtime, and stdlib—into a final executable in Amiga HUNK format. Dead code elimination removes unused functions to keep binaries small.

2.7 Development Workflow

2.7.1 Fast Feedback with novus check

For quick syntax and type checking without full compilation:

```
novus check myfile.novus
```

This skips code generation and linking, giving faster feedback during development.

2.7.2 Code Formatting

Keep your code consistent with `novus fmt`:

```
novus fmt myfile.novus
```

This reformats the file in place according to the Novus style guide.

2.7.3 Editor Support

Novus provides a Language Server Protocol (LSP) implementation for IDE integration:

- **VS Code** — Install the Novus extension from the marketplace
- **Other editors** — Configure your LSP client to use `novus lsp`

Features include:

- Syntax highlighting
- Error diagnostics as you type
- Go to definition
- Hover information
- Code completion

2.8 Running on Real Hardware

The ultimate goal is running on real Amiga hardware. Here are several ways to transfer executables.

2.8.1 Shared Folders (Emulator)

If using FS-UAE, WinUAE, or Amiberry, configure a shared folder:

1. Create a directory on your host system (e.g., `~/amiga_share`)
2. Configure the emulator to mount it as a device (e.g., `DEV:`)
3. Compile directly to the shared folder:

```
novus compile hello.novus -o ~/amiga_share/hello
```

4. Access from the Amiga: `DEV:hello`

2.8.2 Network Transfer

With networked Amigas (via Plipbox, A2065, or similar):

```
# Using scp (requires AmiSSH or similar on the Amiga)
scp hello amiga.local:Work/

# Using ftp
ftp amiga.local
put hello
```

2.8.3 CF/SD Card

For accelerator cards with CF/SD storage:

1. Mount the card on your development machine
2. Copy the executable to the card
3. Unmount and return the card to the Amiga

2.8.4 Floppy Disk Images

Create an ADF image with your executable:

```
# Create a new ADF
xdftool myfiles.adf format "MyDisk" + write hello
```

Then use the ADF in your emulator or write it to a real floppy.

2.9 Project Structure

For anything beyond a single file, create a project with `novus new`:

```
novus new myproject --type cli
cd myproject
```

This creates a standard project structure:

```
myproject/
    project.toml         # Project configuration
    src/
        main.novus       # Entry point
```

2.9.1 The project.toml File

The `project.toml` file defines project metadata and build settings:

```
[package]
name = "myproject"
version = "0.1.0"
authors = ["Your Name"]
description = "My first Novus project"

[build]
target_cpu = "68020"
fpu = "auto"
optimization_level = 0
output = "build"
```

2.9.2 Build Settings Reference

Setting	Default	Description
`target_cpu`	`auto`	Target CPU: 68020, 68030, 68040, 68060, or auto
`fpu`	`auto`	FPU mode: soft, 68881, 68040, or auto
`optimization_level`	`0`	VBCC optimization level (0–3)
`output`	`build`	Output directory for compiled files
`stack_size`	`4096`	Stack size in bytes

Build the project with:

```
novus build
```

2.10 Compiler Commands Reference

`novus compile <file>` Compile a single source file to an executable

`novus build` Build a project defined by `project.toml`

`novus check <file>` Type-check without generating code

`novus test <file>` Compile test runner

`novus new <name>` Create a new project from a template

`novus fmt <file>` Format source code

`novus clean` Remove cached build artifacts

`novus lsp` Start the language server

2.10.1 Common Options

Options for `novus compile`:

`-o, -output` Output file name (default: `a.out`)

`-cpu` Target CPU: `68020`, `68030`, `68040`, `68060`, or `auto`

`-fpu` FPU mode: `soft`, `68881`, `68040`, or `auto`

`-release` Enable optimizations and disable debug checks

`-O` Optimization level (0–2)

`-emit-asm` Emit assembly output only

`-emit-ir` Emit IR to stdout

`-v, -verbose` Show detailed progress

2.10.2 Debug vs Release Builds

By default, Novus compiles in debug mode:

- Array bounds checking enabled
- Debug symbols included
- Minimal optimizations

For production, use `-release`:

```
novus compile hello.novus -o hello --release
```

2.11 The Standard Library

Novus includes a comprehensive standard library:

`std::core` Fundamental types: `Option`, `Result`, core traits

`std::collections` Data structures: `Vec`, `HashMap`, `HashSet`

`std::strings` String handling: `String`, `Str`, `StringBuilder`

`std::memory` Memory management, slices

`std::math` Fixed-point arithmetic, trigonometry

`std::io` File and console I/O

`std::ffi` AmigaOS bindings (Exec, DOS, Graphics, Intuition)

`std::hardware` CPU/chipset detection, memory info

Import with `from`:

```
from std::collections::vec import Vec
from std::io::file import println
```

2.12 Showcase: System Information

This program demonstrates the concepts from this chapter by querying and displaying system information:

```
// sysinfo.novus - System Information Display
from std::core import Option
from std::io::file import print, println
from std::ffi::exec import AvailMem
from std::ffi::amiga_consts import MEMF_FAST, MEMF_CHIP
from std::ffi::amiga_consts import MEMF_TOTAL, MEMF_LARGEST

fn print_memory(label: *u8, bytes: u32) {
    print("  ")
    print(label)
    print(": ")
    let kb: u32 = bytes / 1024u32
    if kb == 0 {
        print("0")
    } else {
        print_number(kb)
    }
    println(" KB")
}

fn print_number(n: u32) {
    if n == 0 { return }
    if n >= 10 { print_number(n / 10u32) }
    let digit: u8 = (u8)(n % 10u32) + 48u8
    var buf = [0u8; 2]
    buf[0] = digit
    print(&buf[0])
}

fn show_memory_info() {
    println("")
    println("MEMORY")
    println("----------------------------------------")

    let chip_total = unsafe { AvailMem(MEMF_CHIP | MEMF_TOTAL) }
    let chip_largest = unsafe { AvailMem(MEMF_CHIP |
       MEMF_LARGEST) }
    print_memory("Chip RAM Total", chip_total)
    print_memory("Chip RAM Largest Block", chip_largest)

    println("")

    let fast_total = unsafe { AvailMem(MEMF_FAST | MEMF_TOTAL) }
    if fast_total > 0 {
        let fast_largest = unsafe { AvailMem(MEMF_FAST |
           MEMF_LARGEST) }
        print_memory("Fast RAM Total", fast_total)
        print_memory("Fast RAM Largest Block", fast_largest)
    } else {
        println("  Fast RAM: Not installed")
    }
}
```

```
pub fn main() -> i32 {
    println("============================================")
    println("          NOVUS SYSTEM INFORMATION")
    println("============================================")

    show_memory_info()

    println("")
    println("============================================")
    return 0
}
```

This program demonstrates:

- **Imports** from multiple stdlib modules
- **Unsafe FFI calls** to `AvailMem`
- **Constants** from `amiga_consts`
- **Helper functions** for output formatting
- **Control flow** with conditionals

The full source code is available in `Novus.Tests/Examples/sysinfo_showcase.novus`.

2.13 Running Tests

Novus has built-in test support. Mark test functions with the `@test` attribute:

```
from std::test::test import expect_eq

@test
fn test_addition() {
    expect_eq(2 + 2, 4, "basic addition")
}

@test
fn test_multiplication() {
    expect_eq(3 * 7, 21, "basic multiplication")
}
```

Compile the test runner:

```
novus test mytest.novus
```

Important: The test command builds the test runner but does not execute it. Transfer the executable to your Amiga to run the tests.

2.14 Next Steps

You now have a working Novus installation and understand the basic workflow. The following chapters cover:

- **Chapter 3: Language Basics** — Variables, types, and expressions
- **Chapter 4: Types** — The type system in depth

- **Chapter 5: Memory Management** — Safe and explicit memory handling
- **Chapter 6: Control Flow** — Conditionals, loops, and pattern matching

When you're ready, turn the page and start learning the language itself.

Chapter 3

Language Basics

"Simplicity is prerequisite for reliability."
—Edsger W. Dijkstra

This chapter covers the fundamental building blocks of Novus: how to declare variables, what types are available, how literals work, and how expressions combine values. By the end, you'll understand the core syntax and be ready to write meaningful programs.

Every language has its foundation—the atoms from which programs are built. In Novus, these foundations are designed with systems programming in mind: explicit types, predictable memory layout, and direct mapping to 68k machine operations. Understanding these basics thoroughly will make everything else fall into place.

3.1 Comments

Novus supports two comment styles:

3.1.1 Single-Line Comments

Use `//` for comments that extend to the end of the line:

```
// This is a single-line comment
let x = 42  // Comments can appear after code
```

3.1.2 Multi-Line Comments

Use `/* */` for comments spanning multiple lines:

```
/*
 * This is a multi-line comment.
 * Useful for longer explanations.
 */
let result = compute()
```

Multi-line comments do not nest. The first `*/` closes the comment, regardless of how many `/*` appeared inside.

3.2 Variables and Mutability

Novus provides three keywords for declaring bindings:

3.2.1 Mutable Variables with `var`

Variables declared with `var` can be reassigned:

```
var x = 10
x = 20  // OK - can reassign
x = x + 5  // OK
```

Use `var` for counters, accumulators, temporary values, and any data that changes during execution.

3.2.2 Immutable Bindings with `let`

Bindings declared with `let` cannot be reassigned:

```
let x = 10
x = 20  // ERROR - cannot reassign
```

Immutable bindings make code intent clearer and allow the compiler to optimize more aggressively. Use `let` by default; switch to `var` when you need mutability.

Note that immutability applies to the binding, not necessarily to the data it refers to. For example, a `let` binding to a mutable reference (`&var T`) still allows modifying the referenced data.

3.2.3 Compile-Time Constants with `const`

Constants declared with `const` are evaluated at compile time:

```
pub const SCREEN_WIDTH: u32 = 320
pub const SCREEN_HEIGHT: u32 = 200
pub const SCREEN_SIZE: u32 = SCREEN_WIDTH * SCREEN_HEIGHT
```

Constants must be literals or constant expressions—values that the compiler can evaluate without executing code. They cannot call functions or use runtime values.

Constants are typically declared at module scope with `pub` visibility.

3.2.4 Comparison

Keyword	Reassignable	Evaluation	Typical Scope
`var`	Yes	Runtime	Function/Block
`let`	No	Runtime	Function/Block
`const`	No	Compile-time	Module

Coming from C

In C, all variables are mutable by default. In Novus, prefer `let` (immutable) and only use `var` when you need to reassign.

C	Novus
`int x = 10;`	`var x = 10`
`const int X = 10;`	`let x = 10` or `const X = 10`

Novus's `const` is stricter than C's—it must be a compile-time constant expression, not just a runtime immutable value.

3.2.5 Type Annotations

You can explicitly annotate types:

```
let x: i32 = 42
var count: u32 = 0
```

When the type is obvious from context, annotations are optional:

```
let x = 42        // Inferred as i32
let y = 42u32     // Type suffix makes it u32
```

The compiler performs type inference within function bodies. Module-level constants typically require explicit types.

3.3 Primitive Types

Novus provides several categories of primitive types.

3.3.1 Integer Types

Novus supports signed and unsigned integers in multiple sizes:

`i8` Signed 8-bit integer (−128 to 127)

`i16` Signed 16-bit integer (−32,768 to 32,767)

`i32` Signed 32-bit integer (−2,147,483,648 to 2,147,483,647)

`i64` Signed 64-bit integer (wide range, requires two 32-bit registers on 68k)

`u8` Unsigned 8-bit integer (0 to 255)

`u16` Unsigned 16-bit integer (0 to 65,535)

`u32` Unsigned 32-bit integer (0 to 4,294,967,295)

`u64` Unsigned 64-bit integer (0 to 18,446,744,073,709,551,615)

The default integer type is `i32`. When you write a bare integer literal like `42`, it defaults to `i32` unless context requires otherwise.

3.3.2 Floating-Point Types

`f32` 32-bit IEEE 754 single-precision float

`f64` 64-bit IEEE 754 double-precision float

On 68k systems without an FPU, floating-point operations use software emulation—slow but accurate. For performance-critical math on non-FPU systems, prefer fixed-point types.

3.3.3 Fixed-Point Types

Novus provides fixed-point arithmetic for efficient fractional math on integer hardware:

`fixed16` 16-bit fixed-point (8.8 format: 8 integer bits, 8 fractional bits)

`fixed32` 32-bit fixed-point (16.16 format: 16 integer bits, 16 fractional bits)

Fixed-point types use native 68k integer instructions, making them dramatically faster than soft-float on systems without an FPU.

Fixed-Point Representation

Fixed-point numbers store fractional values by scaling integers. A `fixed16` with 8.8 format stores the value multiplied by 256:

```
// fixed16 (8.8 format)
let half: fixed16 = 0.5        // Stored as 128 (0.5 * 256)
let quarter: fixed16 = 0.25    // Stored as 64 (0.25 * 256)
let pi: fixed16 = 3.14         // Stored as ~804 (3.14 * 256)

// fixed32 (16.16 format) - more precision
let precise_pi: fixed32 = 3.14159  // Stored as 205887 (3.14159 *
    65536)
```

Fixed-Point Arithmetic

All standard arithmetic operations work on fixed-point types:

```
let a: fixed32 = 2.5
let b: fixed32 = 1.5

let sum = a + b          // 4.0
let diff = a - b         // 1.0
let product = a * b      // 3.75
let quotient = a / b     // 1.666...
```

The compiler automatically handles the scaling during multiplication and division, using efficient shift operations where possible.

Converting Between Fixed and Integer

Convert between fixed-point and integers explicitly:

```
let f: fixed32 = 3.75
let i: i32 = f.to_int()        // 3 (truncates)
let rounded: i32 = f.round()   // 4 (rounds to nearest)

let from_int: fixed32 = fixed32::from_int(5)  // 5.0
```

When to Use Fixed-Point

Fixed-point excels in these scenarios:

- **Game physics** — Position, velocity, acceleration calculations
- **Audio processing** — Sample interpolation, volume scaling
- **Graphics** — Texture mapping, rotation, scaling
- **Animation** — Smooth movement, easing functions

Coming from C

C programmers often implement fixed-point manually with macros and shift operations:

C (manual)	Novus
`#define FP_SHIFT 16`	(built-in)
`int fp_mul(int a, int b)`	`a * b`
`{ return (a * b) >> 16; }`	(automatic scaling)
`int x = (3 << 16) / 2;`	`let x: fixed32 = 3.0 / 2.0`

Novus handles the bit manipulation automatically while generating identical machine code.

3.3.4 Boolean Type

The `bool` type represents truth values:

```
let flag: bool = true
let done: bool = false
```

Booleans result from comparison and logical operations. Unlike C, integers and pointers do not implicitly convert to booleans—you must use explicit comparisons.

Coming from C

C allows `if (ptr)` and `if (count)`. Novus requires explicit comparisons:

C	Novus
`if (ptr)`	`if ptr != null`
`if (!ptr)`	`if ptr == null`
`if (count)`	`if count != 0`
`while (n--)`	`while n > 0 { n- ... }`

This catches bugs like `if (x = 10)` which in C silently assigns and evaluates to true.

3.4 Literals

3.4.1 Integer Literals

Integer literals support multiple bases and optional type suffixes.

Decimal

```
let x = 42
let y = 1_000_000  // Underscores improve readability
```

Underscores are ignored; they exist solely to make large numbers easier to read.

Hexadecimal

Novus supports both C-style and Amiga-style hexadecimal:

```
let color1 = 0xFF00AA  // C-style
let color2 = $FF00AA   // Amiga-style (same value)
```

Both forms are equivalent. Use whichever feels natural—Amiga developers often prefer `$`, while those from C backgrounds prefer `0x`.

Binary

Binary literals use the `%` prefix:

```
let mask = %1010    // Binary literal (value: 10)
let flags = %11110000
```

Binary literals are useful for bit patterns and masks when working with hardware registers.

Type Suffixes

Append a type suffix to specify the literal's type explicitly:

```
let a = 42u32   // u32
let b = 42i8    // i8
let c = 100u16  // u16
```

Type suffixes work with all bases:

```
let hex_byte = $FFu8
let binary_word = %1010u16
let decimal_long = 1000u32
```

Without a suffix, integer literals default to `i32`.

3.4.2 Floating-Point Literals

Floating-point literals require a decimal point:

```
let pi = 3.14
let e = 2.71828f32  // f32 with explicit suffix
```

Without a suffix, floating-point literals default to `f64`.

3.4.3 String Literals

String literals use double quotes and support escape sequences:

```
let greeting = "Hello, Amiga!"
let multiline = "Line 1\nLine 2\nLine 3"
let path = "SYS:Utilities\\"  // Escaped backslash
```

Supported escape sequences include:

`\n` Newline (line feed)

`\r` Carriage return

`\t` Horizontal tab

`\0` Null byte

`\\` Backslash

`\"` Double quote

`\'` Single quote

`\xNN` Hexadecimal byte (e.g., `\x1B` for ESC)

3.4.4 F-Strings (Formatted Strings)

F-strings embed expressions directly in string literals:

```
let x = 42
let message = f"The answer is {x}"
// Results in: "The answer is 42"
```

Expressions inside `{...}` are evaluated and converted to strings. F-strings compile to efficient string concatenation or formatting calls.

3.4.5 Character Literals

Character literals use single quotes:

```
let ch: u8 = 'A'
let newline: u8 = '\n'
let escape: u8 = '\x1B'  // ESC character
```

Character literals produce values of type `u8`. Novus does not have a separate `char` type—characters are simply 8-bit unsigned integers.

3.5 Operators

3.5.1 Arithmetic Operators

Standard arithmetic works on integer and floating-point types:

Operator	Meaning
`+`	Addition
`-`	Subtraction
`*`	Multiplication
`/`	Division
`%`	Modulus (remainder)

Division by zero is a runtime error in debug builds and undefined behavior in release builds. Always validate denominators when accepting untrusted input.

```
let sum = 10 + 20      // 30
let product = 5 * 6    // 30
let quotient = 20 / 5  // 4
let remainder = 17 % 5 // 2
```

3.5.2 Comparison Operators

Comparisons produce `bool` values:

Operator	Meaning
`==`	Equal to
`!=`	Not equal to
`<`	Less than
`>`	Greater than
`<=`	Less than or equal to
`>=`	Greater than or equal to

```
let a = 10
let b = 20
let equal = a == b      // false
let less = a < b        // true
let greater_eq = a >= b // false
```

3.5.3 Logical Operators

Logical operators combine boolean values:

Operator	Meaning
`&&`	Logical AND (short-circuiting)
`||`	Logical OR (short-circuiting)
`!`	Logical NOT

```
let x = true
let y = false
let and_result = x && y  // false
let or_result = x || y   // true
let not_result = !x      // false
```

Both `&&` and `||` short-circuit: the right-hand side is not evaluated if the result is determined by the left-hand side alone.

3.5.4 Bitwise Operators

Bitwise operators manipulate individual bits:

Operator	Meaning
`&`	Bitwise AND
`|`	Bitwise OR
`^`	Bitwise XOR
`~`	Bitwise NOT (one's complement)
`«`	Left shift
`»`	Right shift (arithmetic for signed, logical for unsigned)

```
let a = %1111  // 15
let b = %1010  // 10

let and_bits = a & b   // %1010 (10)
let or_bits = a | b    // %1111 (15)
let xor_bits = a ^ b   // %0101 (5)
let not_bits = ~a      // %0000 (-16 in two's complement)

let shifted_left = 1 << 3   // 8
let shifted_right = 16 >> 2 // 4
```

Bitwise operations compile to single 68k instructions and execute in one or two cycles—essential for custom chip programming.

Practical Bitwise Examples

Bitwise operations are fundamental for Amiga programming:

```
// Setting bits (turn ON)
var flags: u16 = 0
flags |= %00000001   // Set bit 0
flags |= %00000100   // Set bit 2
// flags is now %00000101

// Clearing bits (turn OFF)
flags &= ~%00000001  // Clear bit 0
// flags is now %00000100

// Toggling bits (flip)
flags ^= %00001111   // Toggle bottom 4 bits
// flags is now %00001011

// Testing bits
if (flags & %00001000) != 0 {
    // Bit 3 is set
}

// Extracting bit fields
let color: u16 = $0ABC
let red = (color >> 8) & $F    // Extract bits 8-11: $A
let green = (color >> 4) & $F  // Extract bits 4-7: $B
let blue = color & $F          // Extract bits 0-3: $C
```

Hardware Register Manipulation

Custom chip programming relies heavily on bitwise operations:

```
// Enable DMA channels
const DMAF_SETCLR: u16 = $8000
const DMAF_COPPER: u16 = $0080
const DMAF_BITPLANE: u16 = $0100
const DMAF_BLITTER: u16 = $0040

// Turn on Copper, Bitplane, and Blitter DMA
let dmacon_value = DMAF_SETCLR | DMAF_COPPER | DMAF_BITPLANE |
   DMAF_BLITTER

// Mask and shift for sprite position
let x: u16 = 150
let y: u16 = 100
let sprite_pos = ((y & $FF) << 8) | ((x >> 1) & $FF)
```

3.5.5 Compound Assignment Operators

Compound assignment combines an operation with assignment:

```
var x = 10
x += 5   // x = x + 5
x -= 3   // x = x - 3
x *= 2   // x = x * 2
```

```
x /= 4    // x = x / 4
x %= 3    // x = x % 3

x &= $FF    // x = x & $FF
x |= $80    // x = x | $80
x ^= $AA    // x = x ^ $AA
x <<= 2     // x = x << 2
x >>= 1     // x = x >> 1
```

All arithmetic and bitwise operators have corresponding compound forms.

3.5.6 Increment and Decrement Operators

Novus supports both prefix and postfix increment/decrement:

```
var x = 5

++x  // Pre-increment: x becomes 6, evaluates to 6
--x  // Pre-decrement: x becomes 5, evaluates to 5

let a = x++  // Post-increment: a = 5, then x becomes 6
let b = x--  // Post-decrement: b = 6, then x becomes 5
```

Prefix operators modify the variable and return the new value. Postfix operators return the original value and then modify the variable.

3.5.7 Range Operators

Range operators create range values for iteration:

`start..end` Exclusive range (`start` to `end - 1`)

`start..=end` Inclusive range (`start` to `end`)

```
for i in 0..10 {
    // i goes from 0 to 9
}

for j in 0..=10 {
    // j goes from 0 to 10 (inclusive)
}
```

Ranges are covered in detail in Chapter 6.

3.5.8 Operator Precedence

When multiple operators appear in an expression, precedence determines evaluation order. Higher precedence operators bind more tightly:

Precedence	Operators	Associativity
1 (highest)	`() [] . ->`	Left-to-right
2	`! ~ ++ - * &` (unary)	Right-to-left
3	`* / %` (binary)	Left-to-right
4	`+ -` (binary)	Left-to-right
5	`« »`	Left-to-right
6	`< <= > >=`	Left-to-right
7	`== !=`	Left-to-right
8	`&` (bitwise AND)	Left-to-right
9	`^` (bitwise XOR)	Left-to-right
10	`|` (bitwise OR)	Left-to-right
11	`&&`	Left-to-right
12	`||`	Left-to-right
13	`.. ..=`	Left-to-right
14 (lowest)	`= += -=` etc.	Right-to-left

Precedence Examples

```
// Multiplication before addition
let x = 2 + 3 * 4       // 14, not 20

// Shifts before comparison
let y = 1 << 4 > 10     // true (16 > 10)

// Bitwise AND before OR
let z = 1 | 2 & 3       // 3 (2 & 3 = 2, then 1 | 2 = 3)

// Use parentheses for clarity
let clear = (flags & mask) == 0
let combined = (a | b) & (c | d)
```

Coming from C

Novus uses the same operator precedence as C, so your intuition transfers directly. The common gotcha in both languages is that bitwise operators have lower precedence than comparison:

```
// WRONG in both C and Novus:
if flags & MASK == 0 { ... }  // Parsed as: flags & (MASK == 0)

// CORRECT:
if (flags & MASK) == 0 { ... }
```

Always parenthesize bitwise operations in conditions.

3.6 Type Conversions

3.6.1 C-Style Casts

Use C-style cast syntax to convert between numeric types:

```
let x: u32 = 42
let y: i32 = (i32)x
```

```
let z: *u8 = (*u8)y
```

Casts can chain:

```
let w: i64 = (i64)(i32)(u32)x
```

Casts are explicit and potentially unsafe—truncation, sign changes, and pointer conversions are your responsibility. The compiler trusts that you know what you're doing.

3.6.2 The as Keyword

The `as` keyword provides an alternative syntax for type assertions:

```
let x = 42 as u32
```

Both cast styles are valid; choose whichever feels more natural.

3.7 References and Pointers

Novus distinguishes between references (non-null pointers checked at compile time) and raw pointers (nullable, unchecked).

3.7.1 Immutable References

Create a reference with the `&` operator:

```
var x = 10
let ref_x: &i32 = &x
```

The type is `&i32`—a reference to an `i32`. References are guaranteed non-null and automatically dereference in most contexts.

3.7.2 Mutable References

For references that allow modification, use `&var`:

```
var x = 10
let ref_x: &var i32 = &var x
*ref_x = 20  // OK - modifies x
```

The type is `&var i32`. The `&var` syntax appears both in the type annotation and when creating the reference.

Note that Novus uses `var` for mutable, not `mut` as in some other languages.

3.7.3 Raw Pointers

Raw pointers use the `*T` syntax:

```
let ptr: *u8 = (*u8)0xDFF000  // Custom chip register
```

Pointers can be null and require explicit null checks:

```
if ptr == null {
    return  // Handle null case
```

```
}
```

Raw pointers are essential for FFI and hardware access, but prefer references when possible.

Coming from C

Novus distinguishes between references (guaranteed non-null) and pointers (nullable):

C	Novus	Meaning
`int *p`	`*i32`	Nullable pointer
(no equivalent)	`&i32`	Non-null reference
`int * const p`	`let p: *i32`	Immutable pointer
`const int *p`	`*i32` (read-only)	Pointer to const

References eliminate an entire class of null pointer bugs by making non-null the default.

3.7.4 Dereferencing

Use `*` to dereference a pointer or reference:

```
let x = 10
let ref_x = &x
let value = *ref_x  // 10
```

For pointers, dereferencing an invalid or null pointer is undefined behavior.

3.7.5 Pointer Arithmetic

Pointer arithmetic works like C—offsets are scaled by the pointed-to type's size:

```
let arr = [10, 20, 30, 40, 50]
let ptr: *i32 = &arr[0u32]

let first = *ptr            // 10
let second = *(ptr + 1)     // 20 (advances by sizeof(i32) = 4
   bytes)
let third = *(ptr + 2)      // 30
```

Pointer subtraction returns the number of elements between two pointers:

```
let start: *i32 = &arr[0u32]
let end: *i32 = &arr[4u32]
let count = end - start     // 4 elements
```

3.7.6 Auto-Dereferencing

Novus automatically dereferences pointers when accessing struct fields:

```
struct Point { x: i32, y: i32 }

let p = Point { x: 10, y: 20 }
let ptr: *Point = &p

// Both work identically:
let x1 = (*ptr).x  // Explicit dereference
let x2 = ptr.x     // Auto-dereference (preferred)
```

This convenience applies to method calls as well—you don't need `(*ptr).method()`.

3.7.7 Address-of Operator

The `&` operator takes the address of a value:

```
var x: i32 = 42
let ptr: *i32 = &x      // Address of x

// For mutable access:
let mut_ptr: *var i32 = &var x
*mut_ptr = 100          // Modifies x
```

You cannot take the address of a temporary value or literal—only named variables and array elements have stable addresses.

3.8 Arrays

3.8.1 Fixed-Size Arrays

Arrays have a fixed size known at compile time:

```
let numbers = [10, 20, 30]
let primes = [2, 3, 5, 7, 11]
```

The compiler infers both the element type and size from the literal. The type of `numbers` is `[i32; 3]`—an array of 3 `i32` values. The size is part of the type.

3.8.2 Repeat Syntax

Initialize arrays with a repeated value:

```
let zeros = [0; 100]        // 100 zeros
let buffer = [0u8; 1024]    // 1024 bytes
```

The syntax `[value; count]` creates an array of `count` elements, each initialized to `value`.

3.8.3 Indexing

Access array elements with bracket indexing:

```
let numbers = [10, 20, 30]
let first = numbers[0u32]   // 10
let second = numbers[1u32]  // 20
```

Indices must be unsigned integers. In debug builds, out-of-bounds access triggers a panic. In release builds, bounds checking may be elided for performance.

3.9 Tuples

3.9.1 Tuple Types

Tuples group multiple values of potentially different types:

```
let point: (i32, i32) = (100, 200)
let rgb: (u8, u8, u8) = (255, 128, 64)
```

The type `(i32, i32)` means "a tuple of two `i32` values."

3.9.2 Tuple Literals

Create tuples with parenthesized, comma-separated values:

```
let color = (192, 96, 32)
```

3.9.3 Destructuring

Extract tuple elements with destructuring:

```
let (r, g, b) = get_rgb()
```

This binds `r`, `g`, and `b` to the tuple's three elements. Destructuring works in `let` and `var` bindings.

3.9.4 Returning Multiple Values

Tuples enable returning multiple values from functions:

```
fn get_rgb() -> (i32, i32, i32) {
    return (255, 128, 64)
}
```

3.10 Expressions and Statements

In Novus, most constructs are expressions—they produce values.

3.10.1 Expression vs Statement

An expression evaluates to a value:

```
let x = 2 + 2  // Expression: 4
```

A statement performs an action without producing a value:

```
x = 10  // Assignment statement
```

However, even assignments are expressions in Novus, allowing chained assignment:

```
var a = 0
var b = 0
a = b = 5  // Both a and b become 5
```

3.10.2 Expression Contexts

Many control flow constructs in Novus can be used as expressions. For example, `if-else` can produce a value:

```
let max = if a > b { a } else { b }
```

The `match` expression (covered in Chapter 6) similarly produces values based on pattern matching. Both branches must have the same type when used as expressions.

3.11 Intrinsics

Novus provides compile-time intrinsics that look like function calls but are handled specially by the compiler:

```
// Get size of a type in bytes
let size = @sizeof(MyStruct)

// Create a zero-initialized value of a type
let data = @zeroed(MyStruct)
```

Additionally, `@drop_in_place(expr)` can be used to explicitly drop a value before it goes out of scope.

Note: The `alignof()` intrinsic is currently only available in inline assembly blocks. These intrinsics are resolved at compile time and have no runtime overhead.

3.11.1 Available Intrinsics

`@sizeof(T)` Returns the size of type `T` in bytes

`@zeroed(T)` Creates a zero-initialized value of type `T`

`@drop_in_place(expr)` Explicitly drops a value before scope end

`@unreachable()` Marks code as unreachable (optimization hint)

`@type_name(T)` Returns the name of type `T` as a string

3.11.2 Common Uses

The `@sizeof` intrinsic is essential for memory allocation:

```
let size = @sizeof(Player) * count
let mem = AllocMem(size, MEMF_PUBLIC)
```

The `@zeroed` intrinsic creates zero-initialized instances, commonly used in constructors:

```
impl Vec<T> {
    pub fn new() -> Vec<T> {
        return @zeroed(Vec<T>)  // All fields zero/null
    }
}
```

3.12 Best Practices

Following these guidelines will help you write clear, efficient Novus code:

3.12.1 Variable Declarations

- **Prefer `let` over `var`** — Immutability is the default; only use `var` when you need to reassign.
- **Use meaningful names** — `player_count` is better than `pc` or `x`.
- **Initialize at declaration** — Avoid declaring variables without immediate initialization.
- **Use `const` for true constants** — Screen dimensions, hardware addresses, magic numbers.

```
// Good
const SCREEN_WIDTH: u16 = 320
let player_count = get_player_count()
var current_score = 0u32

// Avoid
var x = 0        // What is x?
let BUFFER_SIZE = 1024  // Should be const
```

3.12.2 Type Annotations

- **Omit when obvious** — `let x = 42` is fine; the type is clearly `i32`.
- **Annotate for clarity** — `let mask: u16 = $FFFF` makes the intended type explicit.
- **Always annotate function signatures** — Parameters and return types should be explicit.

3.12.3 Numeric Types

- **Match the hardware** — Use `u16` for hardware registers, `u32` for addresses.
- **Use unsigned for bit manipulation** — Bitwise operations on signed types can surprise you.
- **Prefer `fixed32` over `f32`** — On non-FPU systems, fixed-point is 10-100x faster.

3.13 Common Pitfalls

3.13.1 Integer Overflow

Unlike some languages, Novus does not check for integer overflow in release builds:

```
var x: u8 = 255
x += 1  // x becomes 0, not 256!

var y: i8 = 127
y += 1  // y becomes -128 (overflow wraps)
```

Use wider types or explicit overflow checks when necessary.

3.13.2 Signed vs Unsigned Comparisons

Comparing signed and unsigned integers can produce surprising results:

```
let signed: i32 = -1
let unsigned: u32 = 1

// This requires explicit casting
if (u32)signed > unsigned {  // -1 as u32 is 4294967295
    // This branch executes!
}
```

The compiler warns about mixed signed/unsigned comparisons. Heed these warnings.

3.13.3 Precedence Surprises

Bitwise operators have lower precedence than comparison operators:

```
// WRONG: Parsed as flags & (MASK != 0)
if flags & MASK != 0 { }

// CORRECT: Explicit parentheses
if (flags & MASK) != 0 { }
```

When in doubt, add parentheses.

3.13.4 Reference vs Copy

Understanding when you have a reference vs a copy is crucial:

```
var x = 42
var y = x       // y is a COPY of x
y = 100         // x is still 42

var a = [1, 2, 3]
let b = &a      // b is a REFERENCE to a
// Modifying through b would modify a
```

Primitive types (`i32`, `bool`, etc.) are always copied. Arrays and structs may be copied or referenced depending on context.

3.14 Showcase: Temperature Converter

This example demonstrates many concepts from this chapter in a practical application:

```
// temperature_converter.novus
// Demonstrates: variables, types, operators, fixed-point,
   functions

from std::io import println

// Constants for conversion formulas
const FAHRENHEIT_OFFSET: fixed32 = 32.0
const CELSIUS_MULTIPLIER: fixed32 = 1.8  // 9/5

// Temperature scales as an enum
```

```
enum Scale {
    Celsius,
    Fahrenheit,
    Kelvin,
}

// A temperature value with its scale
struct Temperature {
    value: fixed32,
    scale: Scale,
}

impl Temperature {
    // Create a new temperature
    pub fn new(value: fixed32, scale: Scale) -> Temperature {
        return Temperature { value, scale }
    }

    // Convert to Celsius
    pub fn to_celsius(&self) -> fixed32 {
        match self.scale {
            Scale::Celsius => self.value,
            Scale::Fahrenheit => (self.value - FAHRENHEIT_OFFSET)
                / CELSIUS_MULTIPLIER,
            Scale::Kelvin => self.value - 273.15,
        }
    }

    // Convert to Fahrenheit
    pub fn to_fahrenheit(&self) -> fixed32 {
        let celsius = self.to_celsius()
        return celsius * CELSIUS_MULTIPLIER + FAHRENHEIT_OFFSET
    }

    // Convert to Kelvin
    pub fn to_kelvin(&self) -> fixed32 {
        let celsius = self.to_celsius()
        return celsius + 273.15
    }

    // Check if freezing
    pub fn is_freezing(&self) -> bool {
        return self.to_celsius() <= 0.0
    }

    // Check if boiling (water at sea level)
    pub fn is_boiling(&self) -> bool {
        return self.to_celsius() >= 100.0
    }
}

fn main() -> i32 {
    // Create some temperatures
    let room_temp = Temperature::new(72.0, Scale::Fahrenheit)
    let freezing = Temperature::new(0.0, Scale::Celsius)
    let absolute_zero = Temperature::new(0.0, Scale::Kelvin)

    // Convert and display
```

```
    println(f"Room temperature:
       {room_temp.to_celsius().to_int()}C")
    println(f"Freezing point:
       {freezing.to_fahrenheit().to_int()}F")
    println(f"Absolute zero:
       {absolute_zero.to_celsius().to_int()}C")

    // Boolean operations
    if room_temp.is_freezing() {
        println("Brrr! It's freezing!")
    } else {
        println("Temperature is above freezing.")
    }

    // Array of temperatures
    var temps = [
        Temperature::new(32.0, Scale::Fahrenheit),
        Temperature::new(100.0, Scale::Celsius),
        Temperature::new(273.15, Scale::Kelvin),
    ]

    // Find the hottest
    var max_celsius: fixed32 = -1000.0
    for i in 0u32..3 {
        let c = temps[i].to_celsius()
        if c > max_celsius {
            max_celsius = c
        }
    }

    println(f"Hottest: {max_celsius.to_int()}C")

    // Bit manipulation example: encode temp + scale in u16
    // Format: [scale:2][temp_int:14]
    let temp_int = room_temp.to_celsius().to_int()
    let scale_bits: u16 = 0u16  // Celsius = 0
    let encoded: u16 = (scale_bits << 14) | ((u16)(temp_int) &
       $3FFF)

    // Decode
    let decoded_scale = encoded >> 14
    let decoded_temp = (i16)(encoded & $3FFF)

    println(f"Encoded: ${encoded:04X}, Scale: {decoded_scale},
       Temp: {decoded_temp}")

    return 0
}
```

This showcase demonstrates:

- Constants with `const` and fixed-point values
- Enums for type-safe categories
- Structs with methods via `impl`
- Fixed-point arithmetic for precise calculations

- Boolean expressions and comparisons
- Arrays and iteration
- Bitwise operations for data packing
- F-strings for output formatting

3.15 Summary

This chapter covered Novus's fundamental syntax:

- **Variables:** `var` for mutable, `let` for immutable, `const` for compile-time constants
- **Types:** Integers (`i8`–`i64`, `u8`–`u64`), floats (`f32`, `f64`), fixed-point (`fixed16`, `fixed32`), and `bool`
- **Literals:** Decimal, hex (`0x`/`$`), binary (`%`), strings, f-strings, and characters
- **Operators:** Arithmetic, comparison, logical, bitwise, and compound assignment
- **References:** `&T` (immutable), `&var T` (mutable), `*T` (raw pointers)
- **Arrays:** Fixed-size with type `[T; N]`, indexed with unsigned integers
- **Tuples:** Group values with `(T1, T2, ...)`, destructure with `let (a, b) = ...`

With these building blocks, you can write meaningful programs. The next chapter explores the type system in depth, covering structs, enums, and pattern matching.

Chapter 4

Types

"A type system is a syntactic method for enforcing levels of abstraction in programs."
— John C. Reynolds

Novus features a static, strong type system designed to provide safety without sacrificing the control needed for systems programming on the Amiga. Every value has a known type at compile time, enabling the compiler to catch errors early and generate efficient 68k machine code.

This chapter explores Novus's type system in depth: primitive types, composite types (structs and enums), traits for polymorphism, pattern matching for working with data, and generics for code reuse.

4.1 Type System Overview

Novus's type system is built on these principles:

- **Static typing** — all types known at compile time
- **Strong typing** — no implicit conversions between unrelated types
- **Explicit conversions** — casts must be written explicitly
- **Zero-cost abstractions** — high-level types compile to efficient machine code
- **Null safety** — no null pointers; use `Option<T>` instead
- **Error handling** — errors are values via `Result<T, E>`

Every expression in Novus has a type, and the compiler verifies that types are used correctly throughout your program.

4.2 Primitive Types

Novus provides a set of built-in primitive types (covered in Chapter 2). Here's a quick reference:

4.3 Structs

Structs are composite types that group related data together. They are the primary way to create custom types in Novus.

Type	Description
`i8`, `i16`, `i32`, `i64`	Signed integers
`u8`, `u16`, `u32`, `u64`	Unsigned integers
`bool`	Boolean (`true` or `false`)
`f32`, `f64`	Floating-point (soft-float on 68k without FPU)
`fixed16`, `fixed32`	Fixed-point (8.8 and 16.16 format)
`*T`	Raw pointer to type `T` (can be null)
`&T`, `&var T`	References (guaranteed non-null)

Table 4.1: Novus Primitive Types

4.3.1 Declaring Structs

A struct declaration defines a new type with named fields:

```
struct Point {
    x: i32,
    y: i32,
}

struct Color {
    r: u8,
    g: u8,
    b: u8,
    a: u8,
}
```

Each field has a name and a type. Fields are separated by commas, and a trailing comma is allowed.

Coming from C

No `typedef` needed! In C you often write:
`typedef struct { int x, y; } Point;`
In Novus, the struct name is directly usable as a type:
`struct Point { x: i32, y: i32 }`
Also note: no semicolons after field definitions, and types come *after* names (`x: i32` not `int x`).

4.3.2 Struct Visibility

By default, structs are private to the module where they're defined. Use `pub` to make them public:

```
pub struct Point {
    x: i32,
    y: i32,
}

internal struct Config {
    setting: bool,
}

struct Private {
    data: i32,
}
```

Important: Regardless of struct visibility, *all fields are always private*. Novus does not support field-level visibility modifiers. To expose field values, provide accessor methods in an `impl` block.

4.3.3 Creating Struct Instances

Create instances using struct literal syntax:

```
let origin = Point { x: 0, y: 0 }
let pixel = Point { x: 100, y: 200 }

let red = Color { r: 255, g: 0, b: 0, a: 255 }
```

All fields must be initialized. The order of fields in the literal doesn't matter.

4.3.4 Accessing Fields

Access struct fields using dot notation:

```
let p = Point { x: 10, y: 20 }
let x_coord = p.x  // 10
let y_coord = p.y  // 20
```

To modify fields, the binding must be mutable:

```
var p = Point { x: 10, y: 20 }
p.x = 15
p.y = 25
```

4.3.5 Struct Update Syntax

The spread operator `..` allows creating a new struct from an existing one, updating specific fields:

```
let p1 = Point { x: 10, y: 20 }
let p2 = Point { x: 100, ..p1 }  // { x: 100, y: 20 }
let p3 = Point { y: 200, ..p1 }  // { x: 10, y: 200 }
```

The `..expr` must appear last in the literal and copies all unspecified fields from the given expression. At least one field must be explicitly specified; you cannot use spread alone.

4.3.6 Generic Structs

Structs can be parameterized with type parameters:

```
struct Pair<T, U> {
    first: T,
    second: U,
}
```

Usage:

```
// ... (with Pair defined above)
let int_pair = Pair { first: 42, second: 100 }
let mixed = Pair { first: 10, second: "hello" }
```

The compiler infers type arguments from usage, or you can specify them explicitly:

```
let explicit: Pair<i32, i32> = Pair { first: 1, second: 2 }
```

4.3.7 Const Generic Parameters

Structs can also have compile-time constant parameters:

```
struct Buffer<const N: u32> {
    data: [u8; N],
    len: u32,
}
```

Usage:

```
// ... (with Buffer defined above)
let small_buf: Buffer<64> = Buffer {
    data: [0; 64],
    len: 0
}
let large_buf: Buffer<1024> = Buffer {
    data: [0; 1024],
    len: 0
}
```

Const parameters must be integer types and are known at compile time.

4.3.8 Where Clauses

Struct definitions can have `where` clauses to constrain generic parameters:

```
struct Container<T> where T: Clone {
    value: T,
    backup: T,
}
```

This requires that any type `T` used with `Container` must implement the `Clone` trait.

4.4 Implementation Blocks

Methods and associated functions are defined in `impl` blocks:

```
impl Point {
    fn new(x: i32, y: i32) -> Point {
        Point { x: x, y: y }
    }

    fn distance(&self) -> f32 {
        let dx = self.x as f32
        let dy = self.y as f32
```

```
        sqrt(dx * dx + dy * dy)
    }

    fn translate(&var self, dx: i32, dy: i32) {
        self.x = self.x + dx
        self.y = self.y + dy
    }
}
```

4.4.1 Self Parameter Variants

Methods can take `self` in different forms:

Form	Meaning
`&self`	Immutable borrow; method reads but doesn't modify
`&var self`	Mutable borrow; method can modify the instance
`self`	Takes ownership; instance consumed by call
`consuming self`	Explicit ownership transfer; semantically identical to `self`

Table 4.2: Self Parameter Forms

```
impl Point {
    // Immutable borrow - reads only
    fn magnitude(&self) -> i32 {
        self.x * self.x + self.y * self.y
    }

    // Mutable borrow - modifies in place
    fn scale(&var self, factor: i32) {
        self.x = self.x * factor
        self.y = self.y * factor
    }

    // Consumes self - takes ownership
    fn into_tuple(self) -> (i32, i32) {
        (self.x, self.y)
    }

    // Explicit consuming - same as self
    fn consume(consuming self) {
        // self is consumed
    }
}
```

4.4.2 Associated Functions

Functions without a `self` parameter are associated functions (like static methods):

```
impl Point {
    fn origin() -> Point {
        Point { x: 0, y: 0 }
    }
```

```
    fn from_polar(r: f32, theta: f32) -> Point {
        Point {
            x: (r * cos(theta)) as i32,
            y: (r * sin(theta)) as i32,
        }
    }
}
```

Usage:

```
// ... (with Point defined above)
let origin = Point::origin()
let p = Point::from_polar(10.0, 3.14159 / 4.0)
```

4.4.3 Generic Implementations

Implementations can be generic:

```
// ... (illustrative - with Pair defined above)
impl<T, U> Pair<T, U> {
    fn new(first: T, second: U) -> Pair<T, U> {
        Pair { first: first, second: second }
    }

    fn get_first(&self) -> &T {
        &self.first
    }
}
```

Specialized implementations can use where clauses:

```
// ... (with Pair defined above)
impl<T> Pair<T, T> where T: Clone {
    fn duplicate_first(self) -> Option<Pair<T, T>> {
        match self.first.clone() {
            Option::Some(copy) => {
                return Option::Some(Pair { first: self.first,
                    second: copy })
            },
            Option::None => {
                return Option::None
            },
        }
    }
}
```

4.4.4 Multiple Impl Blocks

A type can have multiple `impl` blocks:

```
impl Point {
    fn new(x: i32, y: i32) -> Point {
        Point { x: x, y: y }
    }
```

```
}

impl Point {
    fn distance(&self) -> f32 {
        sqrt((self.x * self.x + self.y * self.y) as f32)
    }
}
```

This is useful for organizing code or when implementing different traits.

4.5 Enums

Enumerations define a type by enumerating its possible variants. Each variant can carry associated data.

4.5.1 Unit Variants

Simple enums list named constants:

```
enum Direction {
    North,
    South,
    East,
    West,
}
```

Usage:

```
// ... (with Direction defined above)
let heading = Direction::North
```

4.5.2 Tuple Variants

Variants can carry data as tuples:

```
enum Command {
    Quit,
    Move(i32, i32),
    Write(i32),
    ChangeColor(u8, u8, u8),
}
```

Usage:

```
// ... (with Command defined above)
let cmd1 = Command::Quit
let cmd2 = Command::Move(10, 20)
let cmd3 = Command::ChangeColor(255, 0, 0)
```

Each variant can have a different number and type of fields.

4.5.3 Generic Enums

Enums can be parameterized with type parameters:

```
enum Option<T> {
    Some(T),
    None,
}

enum Result<T, E> {
    Ok(T),
    Err(E),
}
```

These are the foundation of Novus's error handling and null safety (covered in detail later).

4.5.4 Enum Limitations

Important: Novus enums support only *unit variants* and *tuple variants*. Struct-style variants are **not supported**:

```
// NOT SUPPORTED - struct variants (example only - invalid syntax)
enum Message {
    Move { x: i32, y: i32 },  // ... (compile error!)
}
```

Instead, use tuple variants:

```
enum Message {
    Move(i32, i32),
    Quit,
}
```

If you need named fields, define a separate struct:

```
struct MoveData {
    x: i32,
    y: i32,
}

enum Message {
    Move(MoveData),
    Quit,
}
```

4.6 Traits

Traits define shared behavior across types. They're similar to interfaces in other languages but more powerful.

4.6.1 Defining Traits

A trait declares a set of methods that implementing types must provide:

```
trait Clone {
    fn clone(&self) -> Option<Self>  // Returns Option because
        allocation can fail
}
```

```
trait Eq {
    fn eq(&self, other: &Self) -> bool
}

trait Hash {
    fn hash(&self) -> u32
}

trait Display {
    fn fmt(&self, f: &var Formatter) -> bool
}
```

4.6.2 Default Implementations

Traits can provide default implementations:

```
trait Printable {
    fn print(&self) {
        // Default implementation
        println("Printable object")
    }

    fn debug_print(&self)  // Must be implemented
}
```

Types implementing the trait can use the default or override it.

4.6.3 Implementing Traits

Use `impl TraitName for TypeName` to implement a trait:

```
impl Clone for Point {
    fn clone(&self) -> Option<Point> {
        return Option::Some(Point { x: self.x, y: self.y })
    }
}

impl Eq for Point {
    fn eq(&self, other: &Point) -> bool {
        return self.x == other.x && self.y == other.y
    }
}
```

Once implemented, you can call trait methods:

```
let p1 = Point { x: 10, y: 20 }
let p2 = Point { x: 30, y: 40 }

// Clone returns Option<T> because allocation can fail
match p1.clone() {
    Option::Some(copy) => {
        // use copy
    },
    Option::None => {
        // handle allocation failure
```

```
    }
}

// Eq provides equality comparison
if p1.eq(&p2) {
    // points are equal
}
```

4.6.4 Built-in Traits

Novus provides several important built-in traits:

Trait	Purpose
`Drop`	Custom cleanup when value goes out of scope
`Clone`	Explicit duplication (returns `Option<Self>`)
`Copy`	Marker trait for implicitly copyable types
`Eq`	Equality comparison
`Hash`	Hashing for use in collections
`From<T>`	Type conversion from `T` (method: `convert`)
`Default`	Provides a default value
`Display`	User-facing formatted output
`Debug`	Programmer-facing debug output

Table 4.3: Core Built-in Traits

The Drop Trait

`Drop` provides custom cleanup logic:

```
struct FileHandle {
    fd: i32,
}

impl Drop for FileHandle {
    fn drop(&var self) {
        close_file(self.fd)
    }
}
```

When a `FileHandle` goes out of scope, `drop` is called automatically.

The From Trait

`From<T>` enables type conversions. The method is named `convert`:

```
impl From<i32> for Point {
    fn convert(value: i32) -> Point {
        return Point { x: value, y: value }
    }
}
```

Usage:

```
// ... (with Point and From impl defined above)
```

```
let p = Point::convert(42)
```

4.6.5 Trait Bounds

Traits can be used as bounds on generic parameters:

```
fn are_equal<T>(a: &T, b: &T) -> bool where T: Eq {
    return a.eq(b)
}

fn clone_if_equal<T>(a: &T, b: &T) -> Option<T> where T: Clone +
    Eq {
    if a.eq(b) {
        return a.clone()  // clone() already returns Option<T>
    }
    return Option::None
}
```

The first function requires `T` to implement `Eq`. The second requires both `Clone` and `Eq`, using `+` to combine bounds.

4.7 Pattern Matching

Pattern matching is a powerful feature for destructuring and inspecting data. It's the primary way to work with enums and extract data from composite types.

4.7.1 Pattern Types

Novus supports several pattern forms:

Literal Patterns

Match exact values:

```
42          // Matches the number 42
true        // Matches boolean true
"hello"     // Matches the string "hello"
null        // Matches null pointer
```

Identifier Patterns

Bind a value to a name:

```
x           // Binds value to variable x
count       // Binds value to variable count
```

Wildcard Pattern

Ignore a value:

```
_           // Matches anything, discards value
```

Variant Patterns

Destructure enum variants:

```
// Pattern syntax examples (illustrative)
Option::Some(x)            // Matches Some, binds inner value to x
Option::None               // Matches None
Command::Move(x, y)        // Matches Move, binds coordinates
Result::Err(e)             // Matches Err, binds error value to e
```

Or Patterns

Match multiple patterns:

```
1 | 2 | 3                  // Matches 1, 2, or 3
Direction::North | Direction::South  // Matches either variant
```

Reference Patterns

Match through references:

```
&x          // Matches a reference, binds referent to x
&42         // Matches reference to 42
```

Guarded Patterns

Add conditional guards:

```
x if x > 0               // Matches any value > 0
Some(x) if x < 100       // Matches Some with value < 100
```

4.7.2 Match Expressions

The `match` expression compares a value against patterns:

```
match value {
    Pattern1 => expression1,
    Pattern2 => expression2,
    Pattern3 if guard => expression3,
    _ => default_expression,
}
```

Each arm consists of a pattern, optional guard, and an expression. The first matching pattern's expression is evaluated.

Basic Matching

```
let number = 42
let description = match number {
    0 => "zero",
    1 => "one",
    2 | 3 | 5 | 7 => "small prime",
```

```
    42 => "the answer",
    _ => "something else",
}
```

Matching Enums

```
let cmd = Command::Move(10, 20)

match cmd {
    Command::Quit => {
        println("Exiting")
        exit()
    },
    Command::Move(x, y) => {
        println("Moving to ({}, {})", x, y)
        move_cursor(x, y)
    },
    Command::Write(ch) => {
        println("Writing character {}", ch)
        write_char(ch)
    },
    Command::ChangeColor(r, g, b) => {
        set_color(r, g, b)
    },
}
```

Pattern Guards

Guards add conditional logic to patterns:

```
let number = 15

match number {
    x if x < 0 => println("negative"),
    x if x == 0 => println("zero"),
    x if x < 10 => println("small positive"),
    x if x < 100 => println("medium positive"),
    _ => println("large positive"),
}
```

Exhaustiveness

Match expressions must be *exhaustive* — they must cover all possible values:

```
// ERROR: non-exhaustive
match direction {
    Direction::North => "north",
    Direction::South => "south",
    // Missing East and West!
}

// OK: exhaustive with wildcard
match direction {
    Direction::North => "north",
```

```
    Direction::South => "south",
    _ => "other",
}

// OK: exhaustive with all variants
match direction {
    Direction::North => "north",
    Direction::South => "south",
    Direction::East => "east",
    Direction::West => "west",
}
```

The compiler verifies exhaustiveness and reports errors if any case is unhandled.

4.7.3 If Let Expressions

Novus provides `if let` for binding values that pass a truthiness check. This is useful for checking pointers for null or integers for zero:

```
let ptr: *u8 = get_pointer()

if let p = ptr {
    // p is bound to ptr (non-null)
    use_pointer(p)
} else {
    // ptr was null
}
```

The condition succeeds if the value is "truthy" (non-null for pointers, non-zero for integers). The bound variable is available in the `if` block.

For `Option` and `Result` types, use `match` for pattern matching:

```
match opt {
    Option::Some(x) => {
        println("Got value: {}", x)
    },
    Option::None => {
        println("No value")
    },
}
```

4.8 Option and Result

Novus eliminates null pointer errors and exception-based error handling by using two core types: `Option<T>` and `Result<T, E>`.

4.8.1 Option<T>

`Option<T>` represents an optional value — either `Some(T)` or `None`:

```
enum Option<T> {
    Some(T),
    None,
}
```

Use `Option` when a value might be absent:

```
fn find_user(id: u32) -> Option<User> {
    if user_exists(id) {
        Option::Some(get_user(id))
    } else {
        Option::None
    }
}
```

Working with Option

The preferred way to work with `Option` is pattern matching:

```
let opt = Option::Some(42)

// Pattern matching (preferred - safe and explicit)
match opt {
    Option::Some(x) => println("Value: %ld", x),
    Option::None => println("No value"),
}
```

For quick checks, use the `is_some()` and `is_none()` methods. Note that these methods **consume** the `Option`:

```
let opt1 = Option::Some(42)
if opt1.is_some() {
    println("Has value")
}
// opt1 is consumed - cannot use it again

let opt2 = Option::None
if opt2.is_none() {
    println("No value")
}
```

To extract the value directly:

```
let opt = Option::Some(42)

// unwrap_or (safe - provides default)
let x = opt.unwrap_or(0)  // Returns 42, or 0 if None

// unwrap (panics on None - use sparingly)
let opt2 = Option::Some(100)
let y = opt2.unwrap()  // Returns 100
```

Option Methods

Common `Option` methods. All methods consume `self` unless otherwise noted:

Note: Because these methods consume `self`, you cannot call multiple methods on the same `Option` value. Use pattern matching when you need to inspect and then use the value.

Method	Description
`is_some(self) -> bool`	Returns `true` if `Some` (consumes)
`is_none(self) -> bool`	Returns `true` if `None` (consumes)
`unwrap(self) -> T`	Returns value, panics on `None`
`unwrap_or(self, T) -> T`	Returns value or default
`ok_or(self, E) -> Result<T,E>`	Converts to `Result`

Table 4.4: Common Option Methods

4.8.2 Result<T, E>

`Result<T, E>` represents either success (`Ok(T)`) or failure (`Err(E)`):

```
enum Result<T, E> {
    Ok(T),
    Err(E),
}
```

Use `Result` for operations that can fail:

```
fn parse_number(s: &str) -> Result<i32, ParseError> {
    if is_valid_number(s) {
        Result::Ok(parse(s))
    } else {
        Result::Err(ParseError::InvalidFormat)
    }
}
```

Working with Result

The preferred way to work with `Result` is pattern matching:

```
let result = parse_number("42")

// Pattern matching (preferred - handles both cases)
match result {
    Result::Ok(n) => println("Number: %ld", n),
    Result::Err(e) => println("Parse failed"),
}

// Another match if you only care about Ok
match result {
    Result::Ok(n) => println("Got number: %ld", n),
    Result::Err(_) => {},
}
```

For quick checks, `is_ok()` and `is_err()` consume the result:

```
let r1 = parse_number("42")
if r1.is_ok() {
    println("Parse succeeded")
}
// r1 is consumed

let r2 = parse_number("bad")
if r2.is_err() {
```

```
    println("Parse failed")
}
```

To extract values directly:

```
let result = parse_number("42")

// unwrap_or (safe - provides default)
let n = result.unwrap_or(0)  // Returns parsed value or 0

// unwrap (panics on Err - use sparingly)
let result2 = parse_number("100")
let m = result2.unwrap()
```

Result Methods

Common `Result` methods. All methods consume `self`:

Method	Description
`is_ok(self) -> bool`	Returns `true` if `Ok` (consumes)
`is_err(self) -> bool`	Returns `true` if `Err` (consumes)
`unwrap(self) -> T`	Returns value, panics on `Err`
`unwrap_or(self, T) -> T`	Returns value or default
`ok(self) -> Option<T>`	Converts to `Option`, discards error
`err(self) -> Option<E>`	Gets error as `Option`

Table 4.5: Common Result Methods

4.8.3 The ? Operator

The `?` operator provides early return on error:

```
fn read_number_from_file(path: &str) -> Result<i32, Error> {
    let file = open_file(path)?         // Returns Err early
    let contents = read_file(&file)?    // Returns Err early
    let number = parse_number(&contents)?  // Returns Err early
    Result::Ok(number)
}
```

The `?` operator:

1. Checks if the `Result` is `Err`
2. If `Err`, returns the error immediately
3. If `Ok`, unwraps the value and continues

This is equivalent to:

```
fn read_number_from_file(path: &str) -> Result<i32, Error> {
    let file = match open_file(path) {
        Result::Ok(f) => f,
        Result::Err(e) => return Result::Err(e),
    }

    let contents = match read_file(&file) {
```

```
        Result::Ok(c) => c,
        Result::Err(e) => return Result::Err(e),
    }

    let number = match parse_number(&contents) {
        Result::Ok(n) => n,
        Result::Err(e) => return Result::Err(e),
    }

    Result::Ok(number)
}
```

The `?` operator works with both `Result` and `Option`:

```
fn get_first_element(vec: &Vec<i32>) -> Option<i32> {
    let first = vec.get(0)?  // Returns None early if empty
    Option::Some(*first * 2)
}
```

4.9 Generics

Generics enable code reuse by parameterizing types and functions with type variables.

4.9.1 Generic Functions

Functions can be generic over types:

```
fn identity<T>(value: T) -> T {
    value
}
```

Usage:

```
// ... (with identity defined above)
let x = identity(42)        // T = i32
let s = identity("hello")   // T = &str
```

Multiple type parameters:

```
fn pair<T, U>(first: T, second: U) -> (T, U) {
    (first, second)
}
```

Usage:

```
// ... (with pair defined above)
let p = pair(42, "hello")  // T = i32, U = &str
```

4.9.2 Generic Structs

Structs can be generic (as shown earlier):

```
struct Vec<T> {
    ptr: *T,
```

```
    len: u32,
    capacity: u32,
}
```

Implementation:

```
// ... (with Vec defined above)
impl<T> Vec<T> {
    fn new() -> Vec<T> {
        return @zeroed(Vec<T>)
    }

    fn push(&var self, value: T) {
        // ... Implementation
    }

    fn get(&self, index: u32) -> Option<&T> {
        if index < self.len {
            Option::Some(&self.ptr[index])
        } else {
            Option::None
        }
    }
}
```

4.9.3 Const Generic Parameters

Functions and types can be parameterized with compile-time constants:

```
fn create_array<const N: u32>() -> [i32; N] {
    [0; N]
}
```

Usage:

```
// ... (with create_array defined above)
let arr1 = create_array::<10>()   // [i32; 10]
let arr2 = create_array::<100>()  // [i32; 100]
```

Structs with const generics:

```
struct Matrix<const ROWS: u32, const COLS: u32> {
    data: [f32; ROWS * COLS],
}
```

Const parameters must be integer types and known at compile time.

4.9.4 Where Clauses

Generic parameters can be constrained with `where` clauses:

```
fn hash_clone<T>(value: &T) -> u32 where T: Clone + Hash {
    let copy = value.clone()
    return copy.hash()
}
```

Where clauses can appear on:

- Function definitions
- Struct definitions
- Impl blocks
- Trait definitions

Multiple constraints with +:

```
fn process<T>(value: T) where T: Clone + Eq + Hash {
    // T must implement Clone, Eq, and Hash
}
```

4.9.5 Turbofish Syntax

Type arguments can be specified explicitly using turbofish syntax ::<>:

```
let vec = Vec::<i32>::new()
let result = parse::<u32>("42")
let pair = Pair::<i32, bool>::new(10, true)
```

This is necessary when the compiler cannot infer type arguments:

```
// Ambiguous - compiler doesn't know T
let vec = Vec::new()  // ERROR

// Explicit - compiler knows T = i32
let vec = Vec::<i32>::new()  // OK
```

4.9.6 Generic Constraints in Practice

Here's a complete example showing generic constraints:

```
trait Comparable {
    fn compare(&self, other: &Self) -> i32
}
```

Function with trait constraint:

```
fn find_max<T>(items: &[T]) -> Option<&T> where T: Comparable {
    if items.len() == 0 {
        return Option::None
    }

    var max = &items[0]
    var i = 1

    while i < items.len() {
        if items[i].compare(max) > 0 {
            max = &items[i]
        }
        i = i + 1
    }
```

```
    Option::Some(max)
}
```

Implementing the trait:

```
// ... (with Comparable defined above)
impl Comparable for i32 {
    fn compare(&self, other: &i32) -> i32 {
        if *self < *other {
            return -1
        } else if *self > *other {
            return 1
        } else {
            return 0
        }
    }
}
```

Usage:

```
// ... (with Comparable, find_max, and impl above)
let numbers = [10, 5, 20, 15]
let max = find_max(&numbers)  // Some(&20)
```

4.10 Type Coercions and Casts

Novus has strict type checking with limited implicit conversions.

4.10.1 Integer Widening

Smaller integer types can be implicitly widened to larger ones in some contexts:

```
let x: i8 = 42
let y: i32 = x  // Implicit widening OK (i8 -> i32)
```

4.10.2 Integer Narrowing

Narrowing requires explicit casts:

```
let x: i32 = 1000
let y: i8 = x as i8  // Explicit cast required (truncates)
```

4.10.3 Reference to Pointer

References can be converted to raw pointers:

```
let x = 42
let r: &i32 = &x
let p: *i32 = r as *i32  // Reference to pointer
```

4.10.4 Array to Pointer

Arrays decay to pointers in some contexts:

```
let arr = [1, 2, 3, 4, 5]
let ptr: *i32 = arr as *i32  // Array to pointer
```

4.10.5 Explicit Casts

Use `as` for explicit type conversions:

```
let x: i32 = 42
let y: f32 = x as f32    // Int to float
let z: u32 = x as u32    // Signed to unsigned
let b: u8 = (x & 0xFF) as u8  // Truncation
```

Always prefer explicit casts to make your intent clear.

4.11 Type Inference

Novus uses type inference to reduce verbosity while maintaining static typing.

4.11.1 Local Variable Inference

The `let` keyword allows the compiler to infer types:

```
let x = 42           // Inferred as i32
let s = "hello"      // Inferred as &str
let v = Vec::new()   // ERROR: cannot infer T
let v = Vec::<i32>::new()  // OK: explicit type argument
```

You can always specify types explicitly:

```
let x: i32 = 42
let s: &str = "hello"
let v: Vec<i32> = Vec::new()
```

4.11.2 Function Return Type Inference

Function return types must be explicitly declared:

```
// ERROR: missing return type
fn add(a: i32, b: i32) {
    a + b
}

// OK: explicit return type
fn add(a: i32, b: i32) -> i32 {
    a + b
}
```

4.11.3 Generic Type Inference

Generic type arguments are often inferred from usage:

```
var vec = Vec::<i32>::new()
vec.push(42)  // T inferred as i32

let opt = Option::Some(42)  // T inferred as i32
```

When inference fails, use turbofish syntax:

```
let result = parse::<i32>("42")
```

4.12 Summary

Novus's type system provides:

- **Structs** for grouping related data with private fields
- **Enums** for sum types with unit and tuple variants
- **Traits** for shared behavior and polymorphism
- **Pattern matching** for safe data inspection and destructuring
- **Option and Result** for null safety and error handling
- **Generics** for code reuse with type and const parameters
- **Type inference** to reduce verbosity
- **Explicit casts** for type conversions

These features combine to create a type system that catches errors at compile time while remaining expressive and efficient. The emphasis on explicitness (no hidden allocations, no implicit conversions) and safety (no null, errors as values) makes Novus code predictable and robust.

In the next chapter, we'll explore Novus's memory management model, including ownership, borrowing, and the interaction with AmigaOS memory pools.

Chapter 5

Memory Management

"Memory management is the heart of every systems program." — Brian Kernighan

The Amiga's cooperative multitasking environment demands careful memory management. Unlike modern systems with virtual memory and garbage collection, the Amiga uses flat physical memory shared between all tasks. A memory leak or dangling pointer affects the entire system, not just one process.

Novus provides a memory management model that combines the safety of modern languages with the control needed for 68k systems programming. This chapter covers ownership and move semantics, references and borrowing, the Drop trait for automatic cleanup, defer blocks for explicit resource management, and the standard library's memory abstractions.

5.1 The Amiga Memory Model

Before diving into Novus specifics, it's essential to understand AmigaOS memory management.

5.1.1 Memory Types

The Amiga has distinct memory types with different characteristics:

Chip RAM Accessible by both CPU and custom chips (Agnus, Denise, Paula). Required for graphics, audio, and DMA operations. Limited to 2MB on most systems.

Fast RAM Accessible only by CPU. Faster access since no DMA contention. Can be much larger (up to 2GB on 68040/060 systems).

Public Memory Safe for inter-task communication. Other tasks may access it.

When allocating memory, you specify flags to request the appropriate type:

```
// ... (with memory flags imported from std::ffi::amiga_consts)
// Chip RAM for graphics buffers
let bitmap = MemoryBlock::alloc(size, MEMF_CHIP | MEMF_CLEAR)

// Fast RAM for general data (falls back to Chip if unavailable)
let buffer = MemoryBlock::alloc(size, MEMF_FAST | MEMF_PUBLIC)

// Any memory (system chooses)
let data = MemoryBlock::alloc(size, MEMF_PUBLIC | MEMF_CLEAR)
```

5.1.2 Memory Flag Reference

5.1.3 No Size Parameter on Free

A critical feature of Novus memory types: you don't pass the size when freeing memory. Unlike raw `FreeMem(ptr, size)` calls, Novus types track size automatically:

Flag	Description
MEMF_ANY	Any memory type (value: 0)
MEMF_PUBLIC	Memory safe for inter-task access
MEMF_CHIP	Chip RAM (required for DMA)
MEMF_FAST	Fast RAM (CPU-only, faster)
MEMF_CLEAR	Zero-fill the allocated memory
MEMF_LARGEST	Allocate largest available block
MEMF_LOCAL	Memory local to the current task
MEMF_24BITDMA	Memory in 24-bit address space
MEMF_KICK	Memory for Kickstart/resident modules
MEMF_REVERSE	Allocate from high addresses down
MEMF_NO_EXPUNGE	Prevent memory from being expunged
MEMF_TOTAL	Query total memory (not for allocation)

Table 5.1: AmigaOS Memory Flags

```
// AmigaOS C style - error-prone
void* ptr = AllocMem(1024, MEMF_PUBLIC);
// ... must remember size is 1024 ...
FreeMem(ptr, 1024);  // Wrong size = memory corruption!

// Novus style - size tracked automatically
var block = MemoryBlock::alloc(1024, MEMF_PUBLIC)?
// ... no need to track size ...
block.free()  // Size remembered internally!
```

This eliminates an entire class of bugs common in AmigaOS programming.

5.2 Ownership and Move Semantics

Every value in Novus has a single *owner*—the variable that holds it. When the owner goes out of scope, the value is dropped (cleaned up). This ownership model prevents double-free and use-after-free bugs.

5.2.1 Move by Default

When you pass a value to a function that takes ownership, the value is *moved*:

```
fn consume(consuming s: String) {
    // s is owned by this function
    // it will be dropped when consume() returns
}

fn main() -> i32 {
    let x = String::new()

    consume(x)  // x is moved into consume()

    // ERROR: x has been moved, cannot use it
    // let len = x.len()  // Compile error!

    return 0
}
```

The `consuming` keyword explicitly marks parameters that take ownership. After a move, the original variable is invalid—the compiler prevents you from using it.

5.2.2 Borrowing to Avoid Moves

To use a value without transferring ownership, *borrow* it with a reference:

```
fn print_length(s: &String) {
    // s is borrowed immutably
    let len = s.len()
    println("Length: {}", len)
}

fn main() -> i32 {
    let x = String::new_from_str("Hello")?

    print_length(&x)  // Borrow x

    // x is still valid here
    let len = x.len()  // OK!

    return 0
}
```

The `&` operator creates a reference—a pointer that borrows the value without taking ownership.

5.2.3 The Borrow Checker

Novus includes a borrow checker that tracks moves and prevents use-after-move errors:

```
fn test_move_tracking() {
    let s = String::new()

    if some_condition {
        consume(s)  // s moved in this branch
    }

    // ERROR: s may have been moved
    // let len = s.len()  // Compile error!
}
```

The borrow checker is control-flow sensitive—it tracks moves across `if`, `match`, and `while` branches.

Important: Novus's borrow checker tracks *moves* and *reference lifetimes*. References cannot outlive their source, and converting references to raw pointers requires `unsafe`. Novus does not enforce reference *exclusivity* (you can have multiple mutable references), but it does prevent dangling references. See Section 5.3.5 for details.

5.3 References

References are non-null pointers that borrow values without taking ownership.

5.3.1 Immutable References

An immutable reference `&T` provides read-only access:

```
fn read_point(p: &Point) -> i32 {
    return p.x + p.y  // Can read fields
}
```

Usage:

```
// ... (with Point and read_point defined above)
let pt = Point { x: 10, y: 20 }
let sum = read_point(&pt)  // Borrow pt immutably
```

5.3.2 Mutable References

A mutable reference `&var T` allows modification:

```
fn scale_point(p: &var Point, factor: i32) {
    p.x = p.x * factor
    p.y = p.y * factor
}
```

Usage:

```
// ... (with Point and scale_point defined above)
var pt = Point { x: 10, y: 20 }
scale_point(&var pt, 2)  // Borrow pt mutably
// pt is now { x: 20, y: 40 }
```

Note that the variable being borrowed must be declared with `var` (mutable).

5.3.3 Self Parameter Forms

Methods use special self parameter syntax:

```
impl Point {
    // Immutable borrow - reads only
    fn magnitude(&self) -> i32 {
        return self.x * self.x + self.y * self.y
    }

    // Mutable borrow - can modify
    fn translate(&var self, dx: i32, dy: i32) {
        self.x = self.x + dx
        self.y = self.y + dy
    }

    // Consuming - takes ownership
    fn into_tuple(consuming self) -> (i32, i32) {
        return (self.x, self.y)
    }
}
```

5.3.4 References vs Raw Pointers

References (`&T`, `&var T`) are guaranteed non-null. Raw pointers (`*T`) can be null and require `unsafe` for dereferencing:

```
let x = 42
let ref: &i32 = &x          // Reference - guaranteed valid
let ptr: *i32 = &x as *i32  // Raw pointer - could be null

// References can be used directly
let y = *ref  // OK

// Raw pointers require unsafe
unsafe {
    let z = *ptr  // Must be in unsafe block
}
```

Use references when possible. Reserve raw pointers for FFI and low-level operations.

5.3.5 Reference Lifetimes

References in Novus have *lifetimes*—they are tied to the scope of the value they borrow. The compiler tracks these lifetimes and prevents you from using a reference after its source has been dropped. This eliminates an entire class of bugs: dangling pointers.

Why Lifetimes Matter

Consider this common pattern when working with AmigaOS screens:

```
fn dangerous() {
    let rp: &RastPort
    {
        let screen = ScreenHandle::lores("Demo", 5)?
        rp = screen.rastport()  // rp borrows from screen
    }  // screen dropped here!
    SetAPen(rp, 2)  // BUG: rp is dangling!
}
```

Without lifetime checking, this code would compile and crash at runtime—a Guru Meditation. With Novus's lifetime tracking, the compiler catches this at compile time:

```
error[E0597]: ‘screen‘ does not live long enough
  --> example.novus:5:14
   |
 4 |         let screen = ScreenHandle::lores("Demo", 5)?
   |             ^^^^^^ borrowed value
 5 |         rp = screen.rastport()
   |              ----------------- borrow occurs here
 6 |     }
   |     ^ ‘screen‘ dropped here while still borrowed
```

Lifetime Rules

Novus enforces these lifetime rules at compile time:

1. **References cannot outlive their source.** A reference must not be used after the value it borrows from goes out of scope.

2. **References cannot be stored in struct fields.** This is a v1 limitation—lifetime parameters on structs are planned for a future version. Use raw pointers if you need to store references in structs.

3. **Method return lifetimes are inferred.** When a method returns a reference, it's tied to `&self` (or the single reference parameter if no self). Multiple reference parameters without `&self` is ambiguous and rejected.

4. **Converting reference to pointer requires unsafe.** To escape lifetime tracking, you must explicitly use an `unsafe` block.

Error Reference

E0597: Does not live long enough Triggered when a reference outlives the value it borrows from.

```
fn bad() {
    let r: &i32
    {
        let x: i32 = 42
        r = &x       // x is in inner scope
    }                // x dropped here
    let y = *r       // ERROR: x doesn't live long enough
}
```

Fix: Move the reference into the same scope as its source, or restructure to avoid storing the reference.

E0106: Cannot contain reference / Cannot infer lifetime Triggered when storing a reference in a struct field, or when lifetime inference is ambiguous.

```
// ERROR: struct cannot contain reference
struct BadCache {
    rp: &RastPort  // Not allowed in v1
}

// ERROR: cannot infer lifetime
fn pick(a: &i32, b: &i32) -> &i32 {  // Which input?
    return a
}
```

Fix for structs: Use a raw pointer (`*RastPort`) and manage safety manually.
Fix for functions: Add `&self` parameter, or reduce to single reference parameter.

E0133: Requires unsafe Triggered when converting a reference to a raw pointer outside an `unsafe` block.

```
let x: i32 = 42
let r: &i32 = &x
let p: *i32 = (*i32)r  // ERROR: requires unsafe
```

Fix: Wrap in `unsafe` if you can guarantee pointer validity:

```
let p: *i32 = unsafe { (*i32)r }  // OK
```

E0515: Cannot return reference to local Triggered when returning a reference to a local variable.

```
fn bad() -> &i32 {
    let x: i32 = 42
    return &x  // ERROR: x will be dropped when function returns
}
```

Fix: Return an owned value, or take the value as a reference parameter and return a reference tied to it.

Escape Hatches

Sometimes you need to break the rules—especially when interfacing with AmigaOS libraries that manage their own memory. Novus provides explicit escape hatches:

Raw pointers (`*T`) have no lifetime tracking. Use them for FFI and cases where you manually guarantee validity:

```
// FFI function returns pointer with unknown lifetime
extern fn GetSomePointer() -> *Thing

fn use_ffi() {
    let ptr: *Thing = GetSomePointer()
    unsafe {
        // You guarantee ptr is valid
        do_something(*ptr)
    }
}
```

Unsafe blocks let you convert references to pointers when needed:

```
fn pass_to_legacy_api(data: &MyData) {
    let ptr: *MyData = unsafe { (*MyData)data }
    LegacyFunction(ptr)  // You guarantee data outlives this call
}
```

The philosophy is *power with guardrails*: safe by default, explicit opt-out when you need control. The `unsafe` keyword marks exactly where you're taking responsibility for memory safety.

5.4 The Drop Trait

The `Drop` trait enables automatic cleanup when values go out of scope—Novus's form of RAII (Resource Acquisition Is Initialization).

5.4.1 Implementing Drop

```
from std::core import Drop

struct FileHandle {
    fd: i32,
}

impl Drop for FileHandle {
    fn drop(&var self) {
        // Called automatically when FileHandle goes out of scope
        close_file(self.fd)
```

```
    }
}

fn main() -> i32 {
    {
        let file = FileHandle { fd: open_file("data.txt") }
        // ... use file ...
    }  // file.drop() called automatically here

    return 0
}
```

5.4.2 Drop Execution Order

Drop is called in reverse declaration order (LIFO):

```
fn main() -> i32 {
    let a = Resource::new(1)  // Created first
    let b = Resource::new(2)  // Created second
    let c = Resource::new(3)  // Created third

    return 0
    // Drop order: c, b, a (reverse)
}
```

5.4.3 Move Prevents Double Drop

When a value is moved, the compiler deactivates its drop:

```
fn take_ownership(consuming r: Resource) {
    // r is dropped here when function returns
}

fn main() -> i32 {
    let r = Resource::new(42)
    take_ownership(r)  // r is moved

    return 0
    // r's drop is NOT called - it was moved
}
```

This prevents double-free bugs.

5.4.4 Drop for Enums

Enums with data-carrying variants have Drop called on their payloads:

```
fn main() -> i32 {
    {
        let opt: Option<String> = Option::Some(String::new())
        // opt goes out of scope
    }  // String inside is automatically dropped

    return 0
}
```

5.5 Defer Blocks

While Drop provides automatic cleanup, sometimes you need explicit control over cleanup order or want to clean up resources that don't implement Drop. The `defer` statement handles this.

5.5.1 Basic Defer

A defer block executes when the enclosing function returns:

```
fn process_file() -> i32 {
    let fd = open_file("data.txt")

    defer {
        close_file(fd)  // Guaranteed to run
    }

    // ... process file ...
    // Even if we return early, defer runs

    if error_occurred {
        return -1  // defer still runs
    }

    return 0  // defer runs here too
}
```

5.5.2 LIFO Execution Order

Multiple defers execute in Last-In-First-Out order:

```
fn main() -> i32 {
    var resource1 = open_resource(1)
    defer {
        cleanup_resource(1)  // Runs SECOND
    }

    var resource2 = open_resource(2)
    defer {
        cleanup_resource(2)  // Runs FIRST
    }

    return 0
    // Execution: cleanup(2), then cleanup(1)
}
```

This matches the natural pattern of "last opened, first closed."

5.5.3 Return Value Computed Before Defer

The return value is computed *before* defer blocks execute:

```
fn compute() -> i32 {
    var result = 42

    defer {
```

```
        result = 0  // This does NOT change the return value!
    }

    return result  // Returns 42, not 0
}
```

This is important: defer blocks cannot modify the return value.

5.5.4 Defer vs Drop

Use Drop when:

- Cleanup is tied to a type's lifetime
- You want automatic cleanup without explicit code
- The resource is always cleaned up the same way

Use defer when:

- Cleanup depends on runtime conditions
- You need cleanup for values that don't implement Drop
- You want explicit control over cleanup order
- The cleanup logic is function-specific

5.6 The Using Statement

The `using` statement combines resource acquisition with guaranteed cleanup:

```
struct Resource {
    value: i32
}

impl Drop for Resource {
    fn drop(&var self) {
        // Cleanup
    }
}

fn create_resource(v: i32) -> Resource {
    return Resource { value: v }
}

fn main() -> i32 {
    using create_resource(42) {
        // Resource exists in this scope
        let x = 10
    }  // Resource is dropped here

    return 0
}
```

This is syntactic sugar for creating a resource and ensuring it's dropped at block end.

5.7 RAII: Resource Acquisition Is Initialization

RAII is a programming pattern where resource lifetime is tied to object lifetime. When you acquire a resource (memory, file handle, hardware access), you wrap it in an object. When that object is destroyed, the resource is automatically released. Novus implements RAII through the `Drop` trait.

5.7.1 Why RAII Matters on the Amiga

The Amiga's cooperative multitasking makes RAII especially valuable:

- **No memory protection** — a leaked resource affects the entire system
- **Limited resources** — only 8 sprites, 4 audio channels, finite Chip RAM
- **Complex cleanup sequences** — closing a window requires freeing menus, gadgets, and screen locks in the right order
- **Early return paths** — error handling often needs cleanup at multiple exit points

RAII ensures resources are released even when functions return early due to errors. The standard library uses RAII extensively—over 70 types implement `Drop`.

5.7.2 The Three RAII Patterns

Novus uses three complementary RAII patterns throughout the standard library:

Handle Types

Handle types wrap OS resources and manage their complete lifecycle:

```
from std::ui::window import WindowHandle, WindowBuilder
from std::ui::screen import ScreenHandle

fn show_window() -> Result<(), IntuitionError> {
    // WindowHandle manages the Intuition window
    var window = WindowBuilder::new()
        .title("My App")
        .size(640, 480)
        .build()?

    // Use the window...
    window.draw_text(10, 20, "Hello, Amiga!")

    return Result::Ok(())
}  // window.drop() closes the window automatically
```

Common handle types in the standard library:

Guard Types

Guard types provide temporary exclusive access to a shared resource. When the guard goes out of scope, the resource is released for others to use:

```
from std::graphics::blitter import BlitterGuard, own_blitter
from std::os::exec import SemaphoreHandle, SemaphoreGuard
```

Module	Handle Types
std::ui	WindowHandle, ScreenHandle, BufferedWindow
std::graphics	BitMapHandle, FontHandle, SpriteHandle
std::audio	AudioChannel, SampleHandle, ModPlayer
std::os	TimerHandle, MsgPortHandle, SignalHandle
std::net	TcpStream, TcpListener, UdpSocket
std::thread	ThreadHandle, ProcessHandle

Table 5.2: Common Handle Types

```
fn blit_safely() {
    // BlitterGuard gives exclusive blitter access
    match own_blitter() {
        Option::Some(guard) => {
            // Blitter is ours exclusively
            blit_copy(src, dst, width, height)

            // guard.drop() calls DisownBlitter()
        },
        Option::None => {
            // Couldn't get blitter
        }
    }
}

fn critical_update(sem: &SemaphoreHandle, data: &var SharedData) {
    // SemaphoreGuard holds the lock
    var lock = sem.obtain()

    // Safe to modify shared data
    data.counter = data.counter + 1

}  // lock.drop() calls ReleaseSemaphore()
```

Guard types follow a common pattern:

- Created by calling an "obtain" or "lock" function
- Hold exclusive (or shared) access while they exist
- Release access in their `Drop` implementation
- Cannot be copied or cloned

Common guard types:

Builder Types

Builder types accumulate configuration, then create a resource. They clean up intermediate allocations if building fails:

```
from std::ui::mui import MUIAppBuilder, MUIApp

fn create_app() -> Option<MUIApp> {
    // Builder accumulates configuration
    var builder = MUIAppBuilder::new()
```

Guard Type	Purpose
`BlitterGuard`	Exclusive blitter access (OwnBlitter/DisownBlitter)
`SemaphoreGuard`	Exclusive semaphore lock (ObtainSemaphore/ReleaseSemaphore)
`SharedSemaphoreGuard`	Shared semaphore lock (read access)
`CriticalSection`	Task switching disabled (Forbid/Permit)
`InterruptSection`	Interrupts disabled (Disable/Enable)
`RefreshGuard`	Window refresh lock (BeginRefresh/EndRefresh)

Table 5.3: Guard Types in the Standard Library

```
    builder.title("My MUI Application")
    builder.version("$VER: MyApp 1.0")
    builder.author("Developer")

    // Build creates the MUI application
    // Returns None if creation fails
    let app = builder.build()?

    return Option::Some(app)
}  // If build() fails, builder.drop() cleans up

// Alternative: method chaining with @chain attribute
fn create_app_chained() -> Option<MUIApp> {
    return MUIAppBuilder::new()
        .title("My App")
        .version("1.0")
        .build()
}
```

Builder types are especially useful for AmigaOS APIs that require many configuration options (MUI, GadTools, Intuition).

Common builder types:

Builder Type	Creates
`MUIAppBuilder`	MUI application objects
`MUIGroupBuilder`	MUI group objects
`GadToolsMenuBuilder`	GadTools menu strips
`CopperListBuilder`	Hardware copper lists
`WindowBuilder`	Intuition windows
`ReActionBuilder`	ReAction GUI elements

Table 5.4: Builder Types in the Standard Library

5.7.3 Implementing Your Own RAII Types

When wrapping AmigaOS resources, follow these patterns:

```
from std::core import Drop, Option
from std::ffi::exec import FreeMem
from std::ffi::amiga_consts import MEMF_ANY

// Handle pattern: wrap a resource pointer
```

```
struct MyResourceHandle {
    ptr: *MyResource,
    owned: bool,
}

impl MyResourceHandle {
    // Constructor acquires the resource
    pub fn new() -> Option<MyResourceHandle> {
        let ptr = allocate_my_resource()
        if ptr == null {
            return Option::None
        }
        return Option::Some(MyResourceHandle {
            ptr: ptr,
            owned: true,
        })
    }

    // into_raw() transfers ownership out
    pub fn into_raw(consuming self) -> *MyResource {
        // Caller now owns the pointer
        self.owned = false
        return self.ptr
    }
}

impl Drop for MyResourceHandle {
    fn drop(&var self) {
        // Only free if we still own it
        if self.owned && self.ptr != null {
            free_my_resource(self.ptr)
        }
    }
}
```

Key implementation points:

- Track ownership with a boolean flag
- Provide `into_raw()` to transfer ownership out
- Check for null before freeing
- Make constructors return `Option` or `Result`

5.7.4 RAII with Error Handling

RAII and the `?` operator work together beautifully:

```
fn complex_operation() -> Result<(), Error> {
    // Each resource is cleaned up if later steps fail
    var window = open_window()?      // Auto-closed on error
    var screen = open_screen()?      // Auto-closed on error
    var bitmap = alloc_bitmap()?     // Auto-freed on error

    // If this fails, all three are cleaned up
    do_work(&window, &screen, &bitmap)?
```

```
    return Result::Ok(())
}
// Normal exit: all three dropped in reverse order
// Error exit: all acquired resources dropped
```

Without RAII, this would require deeply nested error handling or goto-based cleanup.

5.7.5 When NOT to Use RAII

RAII isn't always appropriate:

- **Borrowed resources** — if you're given a pointer you don't own, don't wrap it in a handle that frees it
- **Shared ownership** — if multiple owners need to keep a resource alive, use explicit reference counting
- **Async cleanup** — if cleanup must happen in a different context (like a callback), use explicit management
- **Performance-critical paths** — very hot loops might benefit from manual pooling

Coming from C

In C, you typically use `goto cleanup` patterns or deeply nested `if` statements to handle resource cleanup. Novus's RAII eliminates these patterns:

```
// C: Manual cleanup with goto
int do_work(void) {
    void *a = NULL, *b = NULL;
    int result = -1;

    a = AllocMem(100, MEMF_ANY);
    if (!a) goto cleanup;

    b = AllocMem(200, MEMF_ANY);
    if (!b) goto cleanup;

    // ... actual work ...
    result = 0;

cleanup:
    if (b) FreeMem(b, 200);
    if (a) FreeMem(a, 100);
    return result;
}
```

In Novus, the same code is much simpler:

```
// Novus: RAII handles cleanup
fn do_work() -> Result<(), ExecError> {
    var a = MemoryBlock::try_alloc(100, MEMF_ANY)?
    var b = MemoryBlock::try_alloc(200, MEMF_ANY)?

    // ... actual work ...

    return Result::Ok(())
```

```
}  // Both freed automatically, in correct order
```

The `?` operator propagates errors, and `Drop` ensures cleanup—no goto, no nested ifs, no size tracking.

5.8 Standard Library Memory Types

Novus provides several memory types in `std::memory` that wrap AmigaOS allocation with safety and convenience.

5.8.1 MemoryBlock

`MemoryBlock` is the fundamental building block—raw bytes with automatic size tracking:

```
from std::memory::block import MemoryBlock
from std::ffi::amiga_consts import MEMF_FAST, MEMF_CLEAR

fn main() -> i32 {
    // Allocate 1024 bytes
    match MemoryBlock::alloc(1024, MEMF_FAST | MEMF_CLEAR) {
        Option::Some(var block) => {
            defer {
                block.free()
            }

            let ptr: *u8 = block.ptr()
            let size: u32 = block.size()

            // Use the memory...
            ptr[0] = 42
        },
        Option::None => {
            // Allocation failed
            return 1
        }
    }

    return 0
}
```

MemoryBlock Methods

`MemoryBlock` implements `Drop`, so it's freed automatically when it goes out of scope.

5.8.2 MemHandle

For common allocation patterns, `MemHandle` provides convenient factory methods with preset flags:

```
from std::memory::amiga import MemHandle

fn main() -> i32 {
    // Chip RAM for graphics/audio (MEMF_CHIP | MEMF_CLEAR |
       MEMF_PUBLIC)
```

Method	Description
`alloc(size, flags)`	Allocate raw memory, returns `Option<MemoryBlock>`
`try_alloc(size, flags)`	Allocate with `Result<MemoryBlock, ExecError>`
`ptr()`	Get pointer to memory
`size()`	Get size in bytes
`is_empty()`	Check if zero-size block
`into_raw()`	Take ownership of pointer, returns `(*u8, u32)`
`free()`	Free the memory
`resize(new_size, flags)`	Resize block, returns success `bool`

Table 5.5: MemoryBlock Methods

```
    match MemHandle::new_chip(1024) {
        Option::Some(var chip_mem) => {
            defer { chip_mem.drop() }
            // Use chip memory for DMA operations
        },
        Option::None => return 1
    }

    // Fast RAM for general data (MEMF_FAST | MEMF_CLEAR |
       MEMF_PUBLIC)
    match MemHandle::new_fast(2048) {
        Option::Some(var fast_mem) => {
            defer { fast_mem.drop() }
            // Use fast memory for CPU operations
        },
        Option::None => return 1
    }

    // Any available memory
    match MemHandle::new_any(512) {
        Option::Some(var any_mem) => {
            // System chooses chip or fast
        },
        Option::None => return 1
    }

    return 0
}
```

`MemHandle` uses `AllocVec`/`FreeVec` internally, which stores the size in the allocation header. This is slightly less memory-efficient than `MemoryBlock` (4 bytes overhead) but provides the same convenience.

Use `MemHandle` for common cases; use `MemoryBlock` when you need full control over memory flags.

5.8.3 Allocation<T>

`Allocation<T>` provides type-safe bulk allocation with element count tracking:

```
from std::memory::block import Allocation
from std::ffi::amiga_consts import MEMF_CHIP, MEMF_CLEAR
```

```
fn main() -> i32 {
    // Allocate space for 100 u32 values
    match Allocation::<u32>::new(100, MEMF_CHIP | MEMF_CLEAR) {
        Option::Some(var alloc) => {
            defer {
                alloc.free()
            }

            let count: u32 = alloc.count()         // 100
            let bytes: u32 = alloc.size_bytes()    // 400
            let ptr: *u32 = alloc.as_mut_ptr()

            // Initialize elements
            var i: u32 = 0
            while i < count {
                ptr[i] = i * 2
                i = i + 1
            }
        },
        Option::None => {
            return 1
        }
    }

    return 0
}
```

`Allocation<T>` checks for overflow when computing `count * sizeof(T)`, preventing a common security vulnerability.

5.8.4 Box<T>

`Box<T>` allocates a single value on the heap:

```
from std::memory::block import Box

struct BigData {
    buffer: [u8; 4096],
}

fn main() -> i32 {
    // Allocate BigData on heap instead of stack
    match Box::new(BigData { buffer: [0; 4096] }) {
        Option::Some(var boxed) => {
            defer {
                boxed.free()
            }

            let ptr: *BigData = boxed.get_mut()
            ptr.buffer[0] = 42
        },
        Option::None => {
            return 1
        }
    }

    return 0
```

```
}
```

Use `Box<T>` for:

- Large structs that don't fit on the stack
- Recursive data structures
- Values that need heap allocation with automatic cleanup

5.8.5 Slice<T>

`Slice<T>` is a fat pointer—a pointer plus length—for passing arrays without losing size information:

```
from std::memory::slice import Slice

fn sum(numbers: &Slice<i32>) -> i32 {
    var total: i32 = 0
    var i: u32 = 0

    while i < numbers.len() {
        match numbers.get(i) {
            Option::Some(n) => {
                total = total + n
            },
            Option::None => {}
        }
        i = i + 1
    }

    return total
}

fn main() -> i32 {
    let arr = [1, 2, 3, 4, 5]
    let slice = Slice::new(&arr as *i32, 5)

    let result = sum(&slice)  // No need to pass count separately!

    return result
}
```

Slice Methods

Method	Description
`new(ptr, len)`	Create slice from pointer and length
`len()`	Get length
`is_empty()`	Check if empty
`get(index)`	Get element with bounds check, returns `Option<T>`
`get_unchecked(index)`	Get element without bounds check (unsafe)

Table 5.6: Slice Methods

5.9 Copy and Clone Traits

Some types can be duplicated safely. Novus provides two traits for this.

5.9.1 The Copy Trait

`Copy` is a marker trait for types that can be implicitly copied:
Primitive types are implicitly Copy:

```
// Primitive types are Copy
let x: i32 = 42
let y = x   // x is copied, both x and y are valid
```

Structs can be marked as Copy if all fields are Copy:

```
struct Point {
    x: i32,
    y: i32,
}

impl Copy for Point {}
```

Usage:

```
// ... (with Point defined as Copy above)
let p1 = Point { x: 1, y: 2 }
let p2 = p1  // p1 is copied, both valid
```

Types that manage resources (like `String`, `Vec`, `MemoryBlock`) should NOT implement `Copy`.

5.9.2 The Clone Trait

`Clone` provides explicit deep copying. It returns `Option<Self>` because cloning types that allocate memory (like `Vec` or `String`) can fail:

```
// ... (with Point defined above)
impl Clone for Point {
    fn clone(&self) -> Option<Point> {
        return Option::Some(Point { x: self.x, y: self.y })
    }
}
```

Usage:

```
// ... (with Point and Clone impl above)
let p1 = Point { x: 1, y: 2 }

match p1.clone() {
    Option::Some(p2) => {
        // p2 is a deep copy of p1
    },
    Option::None => {
        // Clone failed - only happens for types that allocate
    },
}
```

For types that manage resources, `clone()` must allocate new memory, which can fail on a memory-constrained Amiga:

```
// Vec::clone() allocates new memory for the copy
let original = Vec::<i32>::new()?
// ... populate original ...

match original.clone() {
    Option::Some(copy) => {
        // copy is independent - modifying it doesn't affect
           original
    },
    Option::None => {
        // Allocation failed - handle gracefully
    }
}
```

5.10 Common Patterns

5.10.1 Resource Acquisition with Error Handling

```
fn process_with_resources() -> Result<i32, Error> {
    // Acquire first resource
    var mem = MemoryBlock::try_alloc(1024, MEMF_PUBLIC)?
    defer {
        mem.free()
    }

    // Acquire second resource
    var file = open_file("data.txt")?
    defer {
        close_file(file)
    }

    // Use resources...

    return Result::Ok(0)
}
```

The `?` operator propagates errors, and defer ensures cleanup regardless of how the function exits.

5.10.2 Passing Arrays to Functions

```
fn process_buffer(data: &Slice<u8>) {
    var i: u32 = 0
    while i < data.len() {
        let byte = data.get_unchecked(i)
        // Process byte...
        i = i + 1
    }
}

fn main() -> i32 {
```

```
    let buffer = [1u8, 2, 3, 4, 5]
    let slice = Slice::new(&buffer as *u8, 5)
    process_buffer(&slice)
    return 0
}
```

5.10.3 Moving Out of a Container

```
fn take_from_block() -> (*u8, u32) {
    var block = MemoryBlock::alloc(1024, MEMF_PUBLIC)?

    // Transfer ownership to caller
    let (ptr, size) = block.into_raw()

    // block is now empty - Drop won't free anything
    return (ptr, size)

    // Caller must manually: FreeMem(ptr, size)
}
```

Use `into_raw()` when you need to transfer ownership to code that manages memory manually.

5.11 Memory Safety Guarantees

Novus provides several safety guarantees:

No Double-Free The borrow checker deactivates drop for moved values.

No Use-After-Move The compiler prevents using variables after they've been moved.

Automatic Size Tracking `MemoryBlock` and `Allocation<T>` eliminate size-tracking bugs.

Overflow Protection `Allocation<T>` checks for integer overflow before allocation.

Bounds Checking `Slice::get()` returns `Option<T>`, forcing explicit handling of out-of-bounds access.

Non-Null References References (`&T`, `&var T`) are guaranteed non-null.

5.11.1 What Is NOT Guaranteed

For full transparency, Novus does NOT prevent:

- Multiple simultaneous mutable references (no exclusivity enforcement)
- Use-after-free with raw pointers (`*T`)
- Data races (no thread-safety guarantees beyond Exec primitives)
- Memory leaks from forgotten cleanup (though Drop helps)

These trade-offs keep the language simple while providing significant safety improvements over C.

5.12 Summary

Novus memory management provides:

- **Ownership** — every value has a single owner
- **Move semantics** — `consuming` parameters take ownership
- **References** — `&T` (immutable) and `&var T` (mutable) borrow without ownership transfer
- **Borrow checker** — prevents use-after-move at compile time
- **Drop trait** — automatic cleanup when values go out of scope
- **RAII patterns** — Handle, Guard, and Builder types for safe resource management
- **Defer blocks** — explicit cleanup with LIFO execution order
- **Using statement** — scoped resource management
- **Standard library types** — `MemoryBlock`, `MemHandle`, `Allocation<T>`, `Box<T>`, `Slice<T>`
- **AmigaOS integration** — proper memory flags for Chip/Fast/Public RAM

The combination of RAII, ownership tracking, and size-safe abstractions makes Novus significantly safer than C for AmigaOS programming. The standard library's 70+ types with `Drop` implementations ensure that resources are properly cleaned up even in the presence of errors.

In the next chapter, we'll explore control flow in depth: conditionals, loops, pattern matching, and error handling with the `?` operator.

Chapter 6

Control Flow

Control flow constructs determine the order in which statements are executed. Novus provides a comprehensive set of control flow mechanisms, from basic conditionals to sophisticated pattern matching. This chapter covers all the ways you can direct program execution in Novus.

6.1 Conditional Statements

6.1.1 if and else

The `if` statement executes code based on a boolean condition:

```
fn classify_temperature(temp: i32) -> i32 {
    if temp < 0 {
        return -1  // Freezing
    } else if temp < 20 {
        return 0   // Cold
    } else if temp < 30 {
        return 1   // Comfortable
    } else {
        return 2   // Hot
    }
}
```

The condition must be of type `bool`. Unlike C, Novus does not implicitly convert integers to booleans—you must use explicit comparisons.

Blocks are required. Unlike C, the braces around the body are mandatory, even for single statements:

```
// Correct
if x > 0 {
    return x
}

// Syntax error: missing braces
// if x > 0 return x
```

6.1.2 if as an Expression

In Novus, `if` can be used as an expression when both branches return a value:

```
let max = if a > b { a } else { b }

// Also valid with blocks
let result = if condition {
```

```
    compute_something()
} else {
    compute_fallback()
}
```

When used as an expression, both the `if` and `else` branches must be present and must return the same type:

```
// Error: branches return different types
let bad = if condition { 42 } else { "hello" }
```

6.1.3 Short-Circuit Evaluation

The logical operators `&&` (and) and `||` (or) use short-circuit evaluation:

- `a && b`: If `a` is false, `b` is not evaluated
- `a || b`: If `a` is true, `b` is not evaluated

This allows safe idioms like:

```
// ptr is only dereferenced if non-null
if ptr != 0 && (*ptr).value > 0 {
    // Safe to use ptr here
}

// check_signal is only called if running is true
if running && check_signal() {
    handle_signal()
}
```

6.1.4 if let

The `if let` construct checks whether a value is non-null (for pointers) or non-zero (for integers) and binds it to a variable. This is primarily used when working with raw pointers from AmigaOS FFI calls:

```
fn process_pointer(ptr: *u8) -> i32 {
    if let p = ptr {
        // p is bound to ptr and is guaranteed non-null here
        return (i32)p
    } else {
        // ptr was null
        return -1
    }
}
```

Note: For pattern matching on `Option<T>` and `Result<T, E>` types, use `let-else` (Section 6.1.5) or `match` (Section 6.4) instead.

The `if let` construct is particularly useful with pointers returned from AmigaOS functions that may return null on failure:

```
fn open_library(name: *u8) -> i32 {
    let lib = OpenLibrary(name, 0)
```

```
    if let base = lib {
        // Library opened successfully, base is non-null
        CloseLibrary(base)
        return 0
    } else {
        // Library not found
        return 1
    }
}
```

6.1.5 let-else

The `let-else` statement handles pattern matching where the `else` block must diverge (i.e., return, break, continue, or panic). This is ideal for early-exit patterns:

```
enum MaybeValue {
    Some(i32),
    None
}

fn process(input: MaybeValue) -> i32 {
    let MaybeValue::Some(value) = input else {
        // This block runs if pattern doesn't match
        // Must diverge - cannot fall through
        return -1
    }

    // value is now bound and available here
    return value * 2
}
```

The `else` block *must* diverge. The compiler will reject code where the else block could fall through to the subsequent statements.

This pattern works well with `Result` and `Option` types:

```
from std::core import Result
from std::error::errors import DosError

fn read_file(path: *u8) -> Result<i32, DosError> {
    let Result::Ok(handle) = open_file(path) else {
        return Result::Err(DosError::NotFound)
    }

    // handle is now available for use
    let bytes = read_data(handle)
    close_file(handle)
    return Result::Ok(bytes)
}
```

6.2 Loops

Novus provides several loop constructs, each suited to different use cases.

6.2.1 while Loops

The `while` loop executes its body as long as the condition is true:

```
var count = 0
while count < 10 {
    count++
}
// count is now 10
```

The condition is evaluated before each iteration. If the condition is initially false, the body never executes.

```
fn find_first_even(start: i32, max: i32) -> i32 {
    var n = start
    while n < max {
        if n % 2 == 0 {
            return n  // Found an even number
        }
        n++
    }
    return -1  // None found
}
```

6.2.2 forever Loops

The `forever` loop creates an infinite loop that must be exited with `break` or `return`:

```
fn wait_for_signal() -> i32 {
    var iterations = 0
    forever {
        iterations++
        if check_signal() {
            break  // Exit the loop
        }
        if iterations > 1000 {
            return -1  // Timeout
        }
    }
    return iterations
}
```

The `forever` keyword is clearer than `while true` and signals the programmer's intent that the loop is designed to be exited via control flow rather than a condition becoming false.

This is particularly useful for event loops in Amiga applications:

```
fn main_event_loop(window: *Window) {
    forever {
        let msg = GetMsg(window.UserPort)
        if let m = msg {
            let class = m.Class
            ReplyMsg(m)

            if class == IDCMP_CLOSEWINDOW {
                break  // User closed the window
            }
```

```
            handle_message(class)
        }

        WaitPort(window.UserPort)
    }
}
```

6.2.3 C-Style for Loops

Novus supports traditional C-style `for` loops with initialization, condition, and update expressions:

```
// Sum numbers 0 through 9
var sum = 0
for var i = 0; i < 10; i++ {
    sum = sum + i
}
// sum is now 45
```

The initialization can declare a new variable (with `var` or `let`) or assign to an existing one. The loop variable declared in the initialization is scoped to the loop.

```
// Skip even numbers with continue
var odd_sum = 0
for var j = 0; j < 10; j++ {
    if j % 2 == 0 {
        continue  // Skip even numbers
    }
    odd_sum = odd_sum + j
}
// odd_sum is 1 + 3 + 5 + 7 + 9 = 25
```

6.2.4 Range-Based for Loops

Novus provides syntactic sugar for iterating over numeric ranges:

```
// Exclusive range: 0, 1, 2, 3, 4 (excludes 5)
var sum = 0
for i in 0..5 {
    sum = sum + i
}
// sum is 10
```

```
// Inclusive range: 1, 2, 3, 4, 5 (includes 5)
var sum = 0
for i in 1..=5 {
    sum = sum + i
}
// sum is 15
```

Ranges can use variables:

```
let start = 2
```

```
let end = 7
var sum = 0
for i in start..end {
    sum = sum + i
}
// sum is 2 + 3 + 4 + 5 + 6 = 20
```

An empty range (where start equals end) executes zero iterations:

```
var count = 0
for i in 5..5 {
    count++  // Never executes
}
// count is 0
```

6.2.5 for-in Loops with Collections

The `for-in` loop iterates over collections that provide `get()` and `len()` methods:

```
// ... (with Vec imported from std::collections::vec)
let vec: Vec<i32> = Vec { [10, 20, 30, 40, 50] }

var sum: i32 = 0
for value in vec {
    sum += value
}
// sum is 150
```

The loop variable receives a copy of each element. The collection itself remains usable after the loop completes. For collections containing non-copyable types, the iteration behavior depends on the collection's implementation.

Any type that provides `get(index: i32)` and `len()` methods can be used with `for-in`. The compiler generates code equivalent to a C-style for loop that calls these methods.

You can declare the loop variable as mutable if you need to modify the local copy:

```
for var item in collection {
    item = transform(item)  // Modifies local copy
    process(item)
}
```

6.3 Loop Control Statements

6.3.1 break and continue

The `break` statement immediately exits the innermost loop:

```
var i = 0
while i < 100 {
    if i == 42 {
        break  // Exit immediately
    }
    i++
}
// i is 42
```

The `continue` statement skips the rest of the current iteration and proceeds to the next:

```
var sum = 0
var i = 0
while i < 10 {
    i++
    if i == 5 {
        continue  // Skip adding 5
    }
    sum = sum + i
}
// sum is 1+2+3+4+6+7+8+9+10 = 50 (5 is skipped)
```

6.3.2 Labeled Loops

When working with nested loops, you can label loops and use those labels with `break` and `continue` to control outer loops:

```
var count = 0
outer: while true {
    var inner = 0
    while inner < 5 {
        count++
        if count == 7 {
            break outer  // Exit the outer loop
        }
        inner++
    }
}
// count is 7
```

Labels work with all loop types:

```
outer: forever {
    for i in 0..10 {
        for j in 0..10 {
            if i * j > 50 {
                break outer  // Exit all three loops
            }
        }
    }
}
```

The `continue` statement also works with labels:

```
var sum = 0
var i = 0
outer: while i < 5 {
    i++
    var j = 0
    while j < 3 {
        j++
        if i == 3 && j == 2 {
            continue outer  // Skip to next i iteration
        }
        sum++
```

```
    }
}
```

Note: Unlike C labels, Novus labels are only valid for loop control. Novus does not support `goto` statements—labels can only be used with `break` and `continue`.

6.3.3 Postfix Conditions

Novus supports postfix `if` and `unless` for concise conditional statements:

```
fn classify(x: i32) -> i32 {
    return -1 if x < 0     // Return -1 if negative
    return 1 if x > 0      // Return 1 if positive
    return 0               // Zero
}
```

The `unless` keyword is the logical inverse of `if`:

```
fn check(valid: bool) -> i32 {
    return -1 unless valid  // Return -1 if NOT valid
    return 0
}
```

Postfix conditions work with `break` and `continue`:

```
var sum = 0
var m = 0
while m < 10 {
    m++
    continue if m % 3 == 0  // Skip multiples of 3
    sum = sum + m
}
// sum is 1+2+4+5+7+8+10 = 37

outer: while true {
    var k = 0
    while k < 100 {
        k++
        break outer if k == 5  // Exit outer when k reaches 5
    }
}
```

6.4 Pattern Matching with match

The `match` expression provides exhaustive pattern matching over values. It's more powerful than a simple switch statement and is the idiomatic way to handle enums in Novus.

6.4.1 Basic match Expressions

```
fn http_status_category(code: i32) -> i32 {
    match code {
        200 => return 1,  // Success
        201 => return 1,
        204 => return 1,
```

```
        400 => return 2,   // Client Error
        401 => return 2,
        404 => return 2,
        500 => return 3,   // Server Error
        503 => return 3,
        _   => return 0    // Unknown (wildcard)
    }
}
```

The underscore (_) is a wildcard pattern that matches any value not handled by previous arms.

Coming from C

`match` is similar to C's `switch`, but safer and more powerful:

- **No fallthrough**: Each arm is independent; no `break` needed
- **Exhaustive**: The compiler ensures all cases are handled
- **Expressions**: `match` returns a value, not just executes code
- **Patterns**: Can destructure enums, tuples, and structs

C switch	Novus match
`case 1: ...; break;`	`1 => ...,`
`default: ...`	`_ => ...`

Forget `break`—the absence of fallthrough eliminates a whole class of bugs.

6.4.2 Matching on Enums

Pattern matching is essential for working with enums, especially those with associated data:

```
enum Status {
    Success,
    Pending,
    Error(i32)
}

fn handle_status(status: Status) -> i32 {
    match status {
        Status::Success => return 0,
        Status::Pending => return 1,
        Status::Error(code) => return code  // Bind the error code
    }
}
```

When an enum variant carries data, the pattern can bind that data to a variable for use in the match arm.

6.4.3 Match with Generic Enums

Pattern matching works seamlessly with generic enums like `Option<T>` and `Result<T, E>`:

```
from std::core import Option

fn get_value_or_default(opt: Option<i32>) -> i32 {
```

```
    match opt {
        Option::Some(val) => return val,
        Option::None => return 0
    }
}
```

```
from std::core import Result
from std::error::errors import DosError

fn process_result(res: Result<i32, DosError>) -> i32 {
    match res {
        Result::Ok(value) => return value,
        Result::Err(_) => return -1  // Ignore error details
    }
}
```

6.4.4 Match with Blocks

When a match arm requires multiple statements, use a block:

```
fn sign(n: i32) -> i32 {
    match n {
        0 => return 0,
        _ => {
            if n > 0 {
                return 1    // Positive
            }
            return -1       // Negative
        }
    }
}
```

6.4.5 Match with Guards

Work in Progress

Match guards are parsed but code generation has known issues. Use `if`/`else` chains as an alternative until this is resolved.

Match arms can include guard clauses with `if` to add additional conditions:

```
fn classify_number(n: i32) -> i32 {
    match n {
        x if x < 0 => return -1,       // Negative
        0 => return 0,                  // Zero
        x if x <= 10 => return 1,      // Small positive
        _ => return 2                   // Large positive
    }
}
```

The guard is evaluated after the pattern matches but before the arm is executed. If the guard evaluates to false, matching continues with the next arm.

6.4.6 Matching Integer Literals

Novus supports matching on decimal, hexadecimal, and binary integer literals:

```
fn parse_permission(bits: i32) -> i32 {
    match bits {
        %111 => return 7,  // rwx (binary)
        %110 => return 6,  // rw-
        %100 => return 4,  // r--
        _    => return 0
    }
}

fn hex_component(hex: i32) -> i32 {
    match hex {
        $00 => return 0,    // Hexadecimal
        $FF => return 255,
        $80 => return 128,
        _   => return hex
    }
}
```

6.4.7 Exhaustiveness Checking

The Novus compiler verifies that match expressions are exhaustive—every possible value must be handled. For enums, this means every variant must have a corresponding arm (or a wildcard):

```
enum Direction {
    North,
    South,
    East,
    West
}

fn direction_to_int(dir: Direction) -> i32 {
    match dir {
        Direction::North => return 0,
        Direction::South => return 1,
        Direction::East => return 2
        // Error: non-exhaustive pattern, Direction::West not
           covered
    }
}
```

Either add a case for `Direction::West` or use a wildcard pattern.

6.5 Error Propagation with the Try Operator

The try operator (`?`) provides concise error propagation for functions returning `Result<T, E>`.

6.5.1 Basic Usage

Without the try operator, error handling requires verbose match expressions:

```
fn process_file_manual(path: *u8) -> Result<i32, DosError> {
    let handle_result = open_file(path)
    let handle = match handle_result {
        Result::Ok(h) => h,
        Result::Err(e) => return Result::Err(e)
    }

    let bytes_result = read_file(handle)
    let bytes = match bytes_result {
        Result::Ok(b) => b,
        Result::Err(e) => return Result::Err(e)
    }

    close_file(handle)
    return Result::Ok(bytes)
}
```

With the try operator, this becomes:

```
fn process_file(path: *u8) -> Result<i32, DosError> {
    let handle = open_file(path)?
    let bytes = read_file(handle)?
    close_file(handle)?

    return Result::Ok(bytes)
}
```

The `?` operator:

1. Unwraps `Result::Ok(value)` and yields `value`
2. On `Result::Err(e)`, immediately returns `Result::Err(e)` from the current function

6.5.2 Try Operator in Expressions

The try operator can be used within expressions:

```
fn get_file_size(path: *u8) -> Result<i32, DosError> {
    let handle = open_file(path)?
    let bytes = read_file(handle)?
    return Result::Ok(bytes + 100)  // Add header overhead
}
```

6.5.3 Chaining Operations

The try operator excels at chaining multiple fallible operations:

```
fn complex_operation() -> Result<i32, DosError> {
    let file = open_file("data.txt")?
    let header = read_header(file)?
    let body = read_body(file, header.length)?
    let result = process_data(body)?
    close_file(file)?

    return Result::Ok(result)
}
```

Each `?` acts as an early-exit point—if any operation fails, the error propagates immediately to the caller.

6.5.4 Combining with defer

The try operator pairs well with `defer` for resource cleanup:

```
fn safe_file_operation(path: *u8) -> Result<i32, DosError> {
    let handle = open_file(path)?
    defer {
        close_file(handle)
    }

    let header = read_header(handle)?   // If this fails...
    let body = read_body(handle)?       // ...or this...

    // The defer block still runs, closing the file
    return Result::Ok(process(body))
}
```

6.6 The defer Statement

The `defer` statement schedules code to run when the current scope exits, regardless of how it exits (normal completion, early return, or error propagation).

6.6.1 Basic Usage

```
fn example() -> i32 {
    let buffer = allocate_buffer(1024)
    defer {
        free_buffer(buffer)
    }

    // Use buffer...
    if some_condition() {
        return -1  // defer block still runs
    }

    return 0  // defer block runs before returning
}
```

6.6.2 Multiple Defers

Multiple `defer` statements execute in reverse order (last-in, first-out):

```
fn multi_resource() {
    let resource1 = acquire_first()
    defer { release_first(resource1) }

    let resource2 = acquire_second()
    defer { release_second(resource2) }

    let resource3 = acquire_third()
    defer { release_third(resource3) }
```

```
    // Use resources...

    // On exit: release_third, then release_second, then
       release_first
}
```

This LIFO order ensures resources are released in the reverse order of acquisition, which is typically the correct order for dependent resources.

6.6.3 Defer with AmigaOS Resources

The `defer` statement is particularly useful for managing AmigaOS resources:

```
fn open_window_safe() -> i32 {
    let screen = LockPubScreen(0)
    if screen == 0 {
        return -1
    }
    defer { UnlockPubScreen(0, screen) }

    let visual_info = GetVisualInfo(screen, 0)
    if visual_info == 0 {
        return -2
    }
    defer { FreeVisualInfo(visual_info) }

    let window = OpenWindowTags(screen)
    if window == 0 {
        return -3
    }
    defer { CloseWindow(window) }

    // Use window...
    process_events(window)

    return 0
    // Resources released in order: window, visual_info, screen
}
```

6.7 The using Statement

The `using` statement provides scoped resource management for types that implement the `Drop` trait. When the block exits, the resource's `drop` method is called automatically.

```
pub struct Resource {
    value: i32
}

impl Drop for Resource {
    fn drop(&var self) {
        // Cleanup happens here automatically
    }
}

fn create_resource(v: i32) -> Resource {
```

```
    return Resource { value: v }
}

fn process_data() -> i32 {
    using create_resource(42) {
        // The resource exists for this block
        let x: i32 = 10
        // ... use x ...
    }
    // Resource's drop() called automatically here

    return 0
}
```

The `using` statement takes an expression that returns a type implementing `Drop`. The resource is created, the block executes, and then `drop()` is called—regardless of how the block exits.

6.7.1 using vs defer

Both `using` and `defer` provide deterministic cleanup, but they serve different purposes:

Use `defer` when:

- You need to bind the resource to a variable for use in the scope
- The resource must live for the entire function scope
- You need cleanup after early returns from anywhere in the function
- You're working with raw FFI resources that don't implement `Drop`

Use `using` when:

- The resource is only needed to exist during a specific block
- You don't need to access the resource directly (e.g., a lock or guard)
- The resource type implements the `Drop` trait

Example showing the difference:

```
fn process_files() -> i32 {
    // defer: resource is bound to a variable, cleanup at
       function exit
    let handle = open_file("log.txt")
    defer { close_file(handle) }

    // using: resource exists for block scope only
    // Useful for locks, guards, or temporary resources
    using acquire_lock() {
        // Lock is held for this block
        do_protected_work()
    }
    // Lock released here

    // We can still use handle
    write_to_file(handle, "Done")
    return 0
    // handle closed here by defer
}
```

6.8 Summary

Novus provides a complete set of control flow constructs:

- **Conditionals**: `if`/`else`, `if let`, `let-else` for pattern matching with early exit
- **Short-circuit evaluation**: `&&` and `||` operators for efficient boolean logic
- **Loops**: `while`, `forever`, C-style `for`, range-based `for`, and `for-in`
- **Loop control**: `break`, `continue`, labeled loops for nested control
- **Postfix conditions**: `if` and `unless` for concise conditional statements
- **Pattern matching**: `match` with exhaustiveness checking, guards, and literal patterns
- **Error propagation**: The `?` operator for concise `Result` handling
- **Resource management**: `defer` and `using` for deterministic cleanup

These constructs work together to enable clear, safe, and expressive code. The emphasis on explicit control flow and exhaustive pattern matching helps prevent bugs while keeping the code readable.

In the next chapter, we'll explore functions and methods, including function definitions, parameter passing, generic functions, and impl blocks.

Chapter 7

Functions and Methods

> *"Functions should do one thing. They should do it well. They should do it only."*
> —Robert C. Martin

Functions are the fundamental building blocks of Novus programs. This chapter covers everything from basic function definitions to generic methods, closures, and compile-time evaluation. By the end, you'll understand how to write reusable, type-safe code that integrates seamlessly with the Novus type system.

7.1 Function Basics

7.1.1 Defining Functions

Functions are declared with the `fn` keyword:

```
fn add(a: i32, b: i32) -> i32 {
    return a + b
}

fn greet(name: *u8) {
    // Prints the name to console
    println(name)
}
```

A function declaration consists of:

- The `fn` keyword
- The function name (must be a valid identifier)
- Parameters in parentheses, each with name and type
- An optional return type after `->`
- The function body in braces

If no return type is specified, the function returns the unit type `()`.

7.1.2 Implicit Returns

When a function body ends with an expression (not a statement), that expression becomes the function's return value:

```
fn square(x: i32) -> i32 {
    x * x  // Implicit return - no return keyword needed
}
```

```
fn add(a: i32, b: i32) -> i32 {
    a + b
}
```

You can also use explicit `return` statements, which is required for early returns:

```
fn safe_sqrt(x: i32) -> i32 {
    if x < 0 {
        return 0  // Early return requires 'return'
    }
    // ... compute sqrt ...
    result  // Implicit return at end
}
```

Both styles are valid. Use implicit returns for simple functions and explicit returns when you need early exit or want to be more explicit about control flow.

7.1.3 Parameters

Parameters are passed by value by default. Each parameter requires an explicit type:

```
fn calculate(x: i32, y: i32, z: i32) -> i32 {
    return x + y * z
}
```

Important: Parameters are always immutable within the function body. To work with a mutable copy, assign to a local variable:

```
fn process(x: i32) -> i32 {
    // x = x + 1  // Error: cannot mutate parameter
    var local = x  // Create mutable copy
    local = local + 1
    return local * 2
}
```

7.1.4 Reference Parameters

Pass data by reference to avoid copying large structures:

```
struct Player {
    x: i32,
    y: i32,
    health: i32,
    score: u32,
}

// Pass by immutable reference - cannot modify
fn get_position(player: &Player) -> (i32, i32) {
    return (player.x, player.y)
}

// Pass by mutable reference - can modify
fn damage(player: &var Player, amount: i32) {
    player.health = player.health - amount
    if player.health < 0 {
```

```
        player.health = 0
    }
}

fn main() -> i32 {
    var p = Player { x: 100, y: 200, health: 100, score: 0 }

    let (x, y) = get_position(&p)
    damage(&var p, 25)

    return 0
}
```

Coming from C

In C, you use pointers for pass-by-reference and must manually track whether a function modifies data:

```
// C: Is player modified? Must check implementation!
void process(Player *player);
void print_info(const Player *player);
```

Novus makes mutability explicit at the call site:

```
// Novus: Call site shows intent clearly
process(&var player)    // Will modify player
print_info(&player)     // Won't modify player
```

The key differences:

- `&T` (immutable reference) ≈ `const T*`
- `&var T` (mutable reference) ≈ `T*`
- References are never null; use `*T` for nullable pointers
- The call site `&var` makes modifications visible

The reference type syntax mirrors the declaration:

- `&T` — immutable reference, pass with `&value`
- `&var T` — mutable reference, pass with `&var value`

7.1.5 The consuming Keyword

The `consuming` keyword indicates that a function takes ownership of a value, consuming it so it cannot be used afterward:

```
fn take_ownership(consuming s: String) {
    // s is now owned by this function
    // ... (The original variable is no longer usable)
}

fn use_string() {
    var s = String::new()
    take_ownership(s)
```

```
    // s is no longer valid here - compilation error if used
}
```

This is particularly useful for builder patterns and type-state transformations:

```
impl StringBuilder {
    // Consumes the builder and returns the final String
    pub fn build(consuming self) -> String {
        // Convert builder to String, consuming self
    }
}
```

Consuming Parameters in AmigaOS FFI

Many AmigaOS system calls use `consuming` to indicate ownership transfer. For example, `FreeMem` consumes its pointer parameter:

```
from std::ffi::exec import AllocMem, FreeMem
from std::ffi::amiga_consts import MEMF_FAST

fn example() {
    let ptr = unsafe { AllocMem(1024, MEMF_FAST) }
    if ptr != null {
        // ... use ptr ...
        // ptr is consumed by FreeMem - cannot use it afterward
        unsafe { FreeMem(ptr, 1024) }
        // ptr is now invalid
    }
}
```

7.1.6 Early Returns

Use explicit `return` for early exit from functions:

```
fn safe_divide(a: i32, b: i32) -> i32 {
    if b == 0 {
        return 0  // Early return for error case
    }
    a / b  // Implicit return for normal case
}
```

7.1.7 Returning Multiple Values

Use tuples to return multiple values:

```
fn divmod(a: i32, b: i32) -> (i32, i32) {
    let quotient = a / b
    let remainder = a % b
    (quotient, remainder)  // Return tuple
}

fn main() -> i32 {
    let (q, r) = divmod(17, 5)
    // q = 3, r = 2
```

```
    return 0
}
```

7.2 Visibility

7.2.1 Public Functions

By default, functions are private to their module. Use `pub` to make them accessible from other modules:

```
// Private - only accessible within this module
fn helper() -> i32 {
    return 42
}

// Public - accessible from other modules
pub fn api_function() -> i32 {
    return helper()
}
```

7.2.2 Internal Visibility

The `internal` modifier makes a function visible within the same package but not to external code:

```
// ... (illustrative - internal visibility modifier)
internal fn package_utility() -> i32 {
    return 100
}
```

7.3 Traits

Traits define shared behavior that types can implement. Understanding traits is essential before discussing generic functions, as generics often use trait bounds.

7.3.1 Defining Traits

A trait declares method signatures that implementing types must provide:

```
pub trait Eq {
    fn eq(&self, other: &Self) -> bool
}
```

The `Self` type (capitalized) refers to the implementing type. In the example above, when implementing `Eq` for `Point`, `Self` becomes `Point`.

7.3.2 Implementing Traits

Use `impl TraitName for Type` to implement a trait:

```
struct Point {
    x: i32,
```

```
    y: i32,
}

impl Eq for Point {
    fn eq(&self, other: &Point) -> bool {
        return self.x == other.x && self.y == other.y
    }
}
```

7.3.3 Common Standard Library Traits

Novus defines several important traits in `std::core`:

`Drop` Destructor called when value goes out of scope

`Clone` Deep copy with potential allocation (returns `Option<Self>`)

`Copy` Marker trait for bitwise-copyable types

`Eq` Equality comparison

`Ord` Total ordering for comparison

`Hash` Hashing for use in `HashMap`

`Display` Format value to a `Formatter`; returns `bool` for success

`From<T>` Type conversion from `T`

`Into<T>` Type conversion to `T`

`Default` Provide a default value for a type

Example implementing `Drop`:

```
from std::ffi::exec import FreeMem

impl Drop for Buffer {
    fn drop(&var self) {
        if self.size > 0 {
            unsafe { FreeMem(self.ptr, self.size) }
            self.ptr = null
            self.size = 0
        }
    }
}
```

7.4 Methods and Impl Blocks

Methods are functions associated with a type, defined within `impl` blocks.

7.4.1 Defining Methods

```
struct Rectangle {
    width: u32,
    height: u32,
}

impl Rectangle {
    // Associated function (no self) - called as Rectangle::new()
    pub fn new(width: u32, height: u32) -> Rectangle {
        return Rectangle { width: width, height: height }
    }

    // Method with immutable self reference
    pub fn area(&self) -> u32 {
        return self.width * self.height
    }

    // Method with mutable self reference
    pub fn scale(&var self, factor: u32) {
        self.width = self.width * factor
        self.height = self.height * factor
    }

    // Method that consumes self
    pub fn into_tuple(self) -> (u32, u32) {
        return (self.width, self.height)
    }
}
```

7.4.2 Self Parameter Types

Methods can receive `self` in different ways:

Receiver	Meaning
`&self`	Immutable reference; can read but not modify
`&var self`	Mutable reference; can read and modify
`self`	Takes ownership by value; consumes the instance
`consuming self`	Explicit ownership transfer (clearer intent)

Both `self` and `consuming self` have the same semantics—they consume the value. The `consuming` keyword makes the intent more explicit and is recommended for clarity.

```
impl Resource {
    // Read-only access
    fn info(&self) -> i32 {
        return self.value
    }

    // Modify in place
    fn update(&var self, new_value: i32) {
        self.value = new_value
    }

    // Consume and transform - explicit consuming
    fn into_raw(consuming self) -> *u8 {
```

```
        let ptr = self.ptr
        // Don't run destructor - ownership transferred
        return ptr
    }
}
```

7.4.3 Associated Functions

Functions without a `self` parameter are called *associated functions*. They're called using the type name:

```
// ... (with Vec defined elsewhere)
impl<T> Vec<T> {
    // Associated function - no self
    // @zeroed creates a zero-initialized instance
    pub fn new() -> Vec<T> {
        return @zeroed(Vec<T>)
    }

    // Associated function with parameters
    pub fn with_capacity(cap: u32) -> Result<Vec<T>, ExecError> {
        // ... Allocate memory and return Vec
    }
}
```

Calling associated functions:

```
// ... (with Vec and impl defined above)
var v = Vec::<i32>::new()
var v2 = Vec::<i32>::with_capacity(100)?
```

Associated functions are commonly used for constructors (`new`, `with_capacity`) and factory methods.

7.4.4 Calling Methods

Methods are called using dot notation:

```
var rect = Rectangle::new(10, 20)
let a = rect.area()          // Call method with &self
rect.scale(2)                // Call method with &var self
let (w, h) = rect.into_tuple()  // Consumes rect
```

The compiler automatically borrows `self` as needed. You don't need to write `(&rect).area()`—the compiler inserts the borrow for you.

Coming from C

In C, you pass structs to functions explicitly and use `->` for pointer access:

```
// C: Explicit struct passing
struct Rectangle *r = create_rect(10, 20);
int area = rect_area(r);      // Pass pointer explicitly
rect_scale(r, 2);             // No indication it modifies r
int w = r->width;             // Arrow for pointer access
```

Novus uses method syntax with automatic borrowing:

```
// Novus: Method syntax, uniform dot access
var rect = Rectangle::new(10, 20)
let area = rect.area()      // Auto-borrow as &self
rect.scale(2)               // Auto-borrow as &var self
let w = rect.width          // Dot works for values too
```

Key differences:

- No `->` operator—dot (`.`) works for both values and pointers
- Methods are called `value.method()`, not `method(&value)`
- Auto-deref: `ptr.field` works like C's `ptr->field`
- Constructors are `Type::new()`, not separate `create_type()` functions

7.5 Generic Functions

Generic functions work with multiple types using type parameters.

7.5.1 Type Parameters

```
fn identity<T>(value: T) -> T {
    return value
}

fn swap<T>(a: &var T, b: &var T) {
    let temp = *a
    *a = *b
    *b = temp
}
```

Type parameters are declared in angle brackets after the function name.

7.5.2 Trait Bounds

Use `where` clauses to constrain type parameters to types that implement specific traits:

```
// ... (with Eq and Display imported)
pub fn expect_eq<T>(actual: T, expected: T, message: Str) where
   T: Eq + Display {
    if actual != expected {
        // ... Report failure - can use Eq and Display methods on
           T
    }
}
```

Multiple bounds are combined with `+`:

```
fn sort<T>(items: &var [T], count: u32) where T: Ord + Clone {
    // Can use Ord for comparison, Clone for copying
}
```

7.5.3 Turbofish Syntax

When the compiler cannot infer generic types, use the turbofish syntax (`::<T>`) to specify them explicitly:

```
// Type inferred from variable declaration
let v: Vec<i32> = Vec::new()

// Explicit type with turbofish
let v = Vec::<i32>::new()

// Turbofish on function calls
let chan = bounded_channel::<i32>()?
```

The turbofish is especially useful when calling generic functions where the type cannot be inferred from context.

7.5.4 Generic Trait Implementations

Implement traits for generic types:

```
// ... (with HashMap defined elsewhere)
impl<K, V> Drop for HashMap<K, V> where K: Hash + Eq {
    fn drop(&var self) {
        // ... Free all buckets and entries
    }
}
```

7.6 Function Overloading

Novus supports function overloading—multiple functions with the same name but different parameter signatures.

7.6.1 Overloading Rules

Functions can be overloaded when they differ by:

- Parameter types (`i32` vs `i64`)
- Parameter count
- Reference vs value (`T` vs `&T`)
- Mutability (`&T` vs `&var T`)
- Pointer vs value (`T` vs `*T`)

```
fn abs(x: i32) -> i32 {
    if x < 0 { return -x }
    return x
}

fn abs(x: i16) -> i16 {
    if x < 0 { return -x }
    return x
}
```

```
fn print_value(x: i32) {
    // Print i32
}

fn print_value(x: i32, y: i32) {
    // Print two values
}

fn print_value(x: i32, y: i32, z: i32) {
    // Print three values
}
```

7.6.2 Overloading Restrictions

Overloading is **not** allowed when functions differ only by:

- Return type (with identical parameters)
- Parameter names (with identical types)

```
// ERROR: Same parameter types, different return types
fn get() -> i32 { return 42 }
fn get() -> i64 { return 42 }  // Error: duplicate signature

// ERROR: Same types, different names
fn test(x: i32) -> i32 { return x }
fn test(y: i32) -> i32 { return y }  // Error: duplicate signature
```

7.7 Const Functions

Const functions can be evaluated at compile time when called with constant arguments.

7.7.1 Defining Const Functions

```
const fn times_two(x: i32) -> i32 {
    return x * 2
}

const fn max(a: i32, b: i32) -> i32 {
    if a > b {
        return a
    }
    return b
}

// Const fn calling another const fn
const fn clamp(value: i32, low: i32, high: i32) -> i32 {
    if value < low { return low }
    if value > high { return high }
    return value
}
```

7.7.2 Using Const Functions

Const functions enable compile-time computation for constants:

```
const WIDTH: i32 = 320
const HEIGHT: i32 = 200
const DOUBLED_WIDTH: i32 = times_two(WIDTH)     // 640 at compile
    time
const SCREEN_SIZE: i32 = WIDTH * HEIGHT         // 64000 at
    compile time
const CLAMPED: i32 = clamp(999, 0, 100)         // 100 at compile
    time
```

Const functions can also be called at runtime with non-constant arguments:

```
fn process(x: i32) -> i32 {
    return times_two(x)  // Runtime call
}
```

7.7.3 Const Function Restrictions

Const functions have limitations:

- No heap allocation
- No I/O operations
- No external function calls
- Only call other const functions
- Limited recursion depth

7.8 Closures

Closures are anonymous functions that can capture variables from their environment.

7.8.1 Closure Syntax

```
// Basic closure with type annotations
let add = |x: i32, y: i32| -> i32 { x + y }

// Closure capturing environment
let multiplier = 10
let scale = |x: i32| -> i32 { x * multiplier }
let result = scale(5)  // 50

// No parameters (note: space between pipes required)
let get_value = | | -> i32 { 42 }
```

Closures use pipes (|) to delimit parameters:

```
|parameters| -> ReturnType { body }
```

7.8.2 Capturing Variables

Closures capture variables from their enclosing scope. By default, captures are by value (copied):

```
fn example() -> i32 {
    let offset = 10
    let add_offset = |x: i32| -> i32 {
        x + offset  // Captures 'offset' by value
    }
    add_offset(5)  // 15
}
```

7.9 External Functions

External functions declare interfaces to C code or AmigaOS system libraries.

7.9.1 Extern Declarations

External functions declare interfaces to AmigaOS system libraries. Here are common examples from exec.library:

```
// Memory allocation
extern pub fn AllocMem(byteSize: u32, requirements: u32) -> *u8
extern pub fn FreeMem(consuming memoryBlock: *u8, byteSize: u32)

// Task synchronization
extern pub fn Wait(signalSet: u32) -> u32
extern pub fn Signal(task: *Task, signals: u32)

// Message passing
extern pub fn PutMsg(port: *MsgPort, message: *Message)
extern pub fn GetMsg(port: *MsgPort) -> *Message
```

From dos.library (note BCPL pointer conventions—file handles are `i32`):

```
// File operations (using BCPL pointers)
extern pub fn Open(name: *u8, accessMode: i32) -> i32
extern pub fn Close(consuming file: i32) -> i32
extern pub fn Read(file: i32, buffer: *u8, length: i32) -> i32
extern pub fn Write(file: i32, buffer: *u8, length: i32) -> i32
```

External functions have no body—they're implemented in ROM or disk-based libraries and linked at runtime.

7.9.2 Calling External Functions

AmigaOS system calls require `unsafe` blocks because they bypass Novus's safety guarantees:

```
from std::ffi::exec import AllocMem, FreeMem
from std::ffi::amiga_consts import MEMF_FAST, MEMF_CLEAR

fn allocate_buffer(size: u32) -> *u8 {
    var ptr: *u8 = null
    unsafe {
```

```
        ptr = AllocMem(size, MEMF_FAST | MEMF_CLEAR)
    }
    return ptr
}

fn free_buffer(ptr: *u8, size: u32) {
    unsafe {
        FreeMem(ptr, size)
    }
}
```

7.9.3 AmigaOS Library Bindings

Novus provides pre-generated FFI bindings for all AmigaOS libraries. These are auto-generated from SFD (System Function Definition) files:

```
// Import from pre-generated FFI modules
from std::ffi::exec import AllocMem, FreeMem, Wait, Signal
from std::ffi::dos import Open, Close, Read, Write
from std::ffi::graphics import BltBitMap, BltTemplate
from std::ffi::intuition import OpenWindow, CloseWindow

// Import structure definitions
from std::ffi::amiga_structs import Task, MsgPort, BitMap,
    RastPort

// Import constants
from std::ffi::amiga_consts import MEMF_CHIP, MEMF_FAST,
    MODE_OLDFILE
```

Developers should import from these modules rather than writing `extern fn` declarations manually.

7.9.4 Variadic Functions

External functions can accept variable arguments using `...args`:

```
extern fn VPrintf(format: *u8, ...args) -> i32
```

Note that variadic functions are only supported for external declarations; Novus code cannot define variadic functions.

7.10 Best Practices

7.10.1 Parameter Passing Guidelines

Situation	Recommendation
Small values (integers, bools)	Pass by value
Large structs, read-only	Pass by reference (`&T`)
Large structs, modify	Pass by mutable reference (`&var T`)
Transfer ownership	Pass by value or use `consuming`

7.10.2 Method Receiver Guidelines

Receiver	Use When
`&self`	Reading data, computing derived values
`&var self`	Modifying internal state
`self` / `consuming self`	Transforming into another type, builder finalization

7.10.3 AmigaOS FFI Best Practices

When calling AmigaOS system libraries:

- Always wrap library calls in `unsafe` blocks
- Check return values—many functions return `NULL`/`0` on failure
- Track allocation sizes—`FreeMem` requires the original size
- Match memory flags—free with the same size you allocated
- Respect `consuming` parameters—don't use pointers/handles after consumption
- Use stdlib wrappers (`MemoryBlock`, `Vec`, etc.) when possible for safety
- BCPL pointers (DOS.library) are `i32` values, not `*u8`

Example of safe memory management:

```
from std::memory::block import MemoryBlock
from std::ffi::amiga_consts import MEMF_FAST

// Prefer MemoryBlock over raw AllocMem/FreeMem
match MemoryBlock::alloc(1024, MEMF_FAST) {
    Option::Some(var block) => {
        // Use block.ptr() to access raw pointer
        let ptr = block.ptr()
        // ... use ptr ...
        // Automatic cleanup when block goes out of scope
    },
    Option::None => {
        // Handle allocation failure
    }
}
```

7.10.4 Naming Conventions

- Use `snake_case` for function names: `get_value`, `set_position`
- Use `new` for constructors: `Vec::new()`, `String::new()`
- Use `with_*` for constructors with parameters: `with_capacity`
- Use `is_*` for boolean queries: `is_empty`, `is_valid`
- Use `into_*` for consuming conversions: `into_raw`, `into_string`
- Use `as_*` for borrowing conversions: `as_str`, `as_slice`
- Use `try_*` for fallible operations returning `Result`: `try_alloc`

7.11 Summary

This chapter covered the complete function system in Novus:

- **Function basics**: Declaration, parameters, return values
- **Reference parameters**: `&T` for reading, `&var T` for modifying
- **Traits**: Defining and implementing shared behavior
- **Methods**: Impl blocks, self receivers, associated functions
- **Generics**: Type parameters, trait bounds, turbofish syntax
- **Overloading**: Multiple functions with the same name
- **Const functions**: Compile-time evaluation
- **Closures**: Anonymous functions capturing environment
- **External functions**: AmigaOS FFI and system calls

Functions in Novus are designed for clarity and safety. Explicit parameter passing, trait bounds, and ownership semantics help prevent bugs while keeping the code readable.

The next chapter explores modules and project organization, showing how to structure larger Novus programs.

Chapter 8

Modules and Project Organization

> *"The purpose of abstraction is not to be vague, but to create a new semantic level in which one can be absolutely precise."*
> —Edsger W. Dijkstra

Novus provides a module system for organizing code into logical units with explicit visibility control. This chapter covers imports, visibility modifiers, project structure, and the standard library organization.

8.1 Module Basics

In Novus, each source file (`.novus`) is a module. The module's namespace is determined by its location in the directory structure.

8.1.1 Module Naming

Module paths use `::` as the separator:

File Path	Module Path
`src/main.novus`	`main`
`src/renderer.novus`	`renderer`
`src/graphics/bitmap.novus`	`graphics::bitmap`
`src/sound/player.novus`	`sound::player`

The standard library uses the `std` prefix:

File Path	Module Path
`std/core.novus`	`std::core`
`std/collections/vec.novus`	`std::collections::vec`
`std/ffi/exec.novus`	`std::ffi::exec`

8.2 Imports

Use the `from ... import ...` syntax to bring items from other modules into scope.

8.2.1 Basic Imports

Import specific items by name:

```
// Import specific types
from std::core import Option, Result

// Import functions and constants
from std::ffi::exec import AllocMem, FreeMem
from std::ffi::amiga_consts import MEMF_FAST, MEMF_CLEAR
```

Coming from C

C's `#include` literally copies header files into your source, leading to:

- Include order dependencies
- Include guards (`#ifndef`/`#define`/`#endif`)
- Slow compilation from re-parsing headers
- Polluted namespaces with everything in headers

```
// C: Headers are textually included
#include <exec/memory.h>
#include <proto/exec.h>     // Order matters!
```

Novus imports are semantic, not textual:

```
// Novus: Import specific symbols
from std::ffi::exec import AllocMem, FreeMem
from std::ffi::amiga_consts import MEMF_FAST
```

Benefits:

- No include guards needed
- Order doesn't matter
- Only imported names are in scope
- Faster compilation (no re-parsing)
- No "header file" vs "source file" distinction

8.2.2 Wildcard Imports

Import all public items from a module:

```
// Import everything from amiga_structs
from std::ffi::amiga_structs import *

// Now Task, MsgPort, Library, etc. are all in scope
```

Use wildcards sparingly—they can make it unclear where symbols come from.

8.2.3 Pattern Imports

Import multiple items matching a prefix or suffix pattern:

```
// Import all IDCMP_* constants (prefix pattern)
from std::ffi::amiga_consts import IDCMP_*

// This imports IDCMP_CLOSEWINDOW, IDCMP_MOUSEBUTTONS, etc.

// Import all *Error types (suffix pattern)
from std::error::errors import *Error
```

```
// This imports DosError, ExecError, GraphicsError, etc.
```

8.2.4 Import Aliases

Rename imports to avoid conflicts or for convenience:

```
from std::collections::vec import Vec as DynamicArray
from std::collections::hashmap import HashMap as Map
```

Using the aliases:

```
// ... (with DynamicArray and Map imported above)
var items = DynamicArray::<i32>::new()
var lookup = Map::<Str, i32>::new()
```

8.2.5 Multiple Imports

Import from multiple modules at the top of your file:

```
// Recommended import order for stdlib files:
// 1. std::core - fundamental types
from std::core import Option, Result, Drop, Clone

// 2. std::collections - data structures
from std::collections::vec import Vec

// 3. std::memory - memory management
from std::memory::block import MemoryBlock

// 4. std::strings - string types
from std::strings::core import Str

// 5. std::error - error types
from std::error::errors import ExecError

// 6. std::ffi - AmigaOS bindings
from std::ffi::amiga_structs import Task, MsgPort
from std::ffi::amiga_consts import MEMF_PUBLIC
from std::ffi::exec import AllocMem, FreeMem
```

8.3 Visibility Modifiers

Novus has four visibility levels that control where items can be accessed.

8.3.1 Private (Default)

Items without a visibility modifier are private to their module:

```
// Only visible in this file
const BUFFER_SIZE: u32 = 1024

fn helper() -> i32 {
    42
```

```
}

struct InternalData {
    value: i32,
}
```

Private items cannot be imported by other modules.

8.3.2 Public (pub)

The `pub` keyword exports items, making them visible to other modules:

```
// Exported - can be imported by other modules
pub const VERSION: u32 = 1

pub fn create_bitmap(w: u32, h: u32) -> Result<Bitmap,
    GraphicsError> {
    // ...
}

pub struct Bitmap {
    width: u32,
    height: u32,
    data: *u8,
}
```

Use `pub` for your module's public API—functions, types, and constants that other code should use.

Coming from C

C uses `static` for file-local symbols and relies on headers for "public" APIs:

```
// C: static = private to this file
static int helper(void) { return 42; }

// C: non-static = visible (if declared in header)
int public_api(void) { return helper(); }
```

Novus inverts this: private by default, explicit `pub` for public:

```
// Novus: private by default
fn helper() -> i32 { return 42 }

// Novus: pub = exported
pub fn public_api() -> i32 { return helper() }
```

The mapping:

- C `static` ≈ Novus (default, no modifier)
- C non-static + header ≈ Novus `pub`
- C `extern` ≈ Novus `extern` (for FFI only)

8.3.3 Internal (`internal`)

The `internal` modifier makes items visible within the same package (workspace) but not exported to external code:

```
// ... (internal visibility - illustrative)
internal const SHARED_BUFFER_SIZE: u32 = 4096

internal fn shared_helper() -> i32 {
    return 100
}
```

Use `internal` for implementation details shared between modules in a library.

8.3.4 Extern (`extern`)

The `extern` keyword declares items defined elsewhere, resolved at link time:

```
// ... (extern declarations - illustrative)
extern var SysBase: *ExecBase

// Hardware register at specific address
extern var CUSTOM: *Custom at 0xdff000

// Function from another object file or assembly
extern fn asm_memcpy(dest: *u8, src: *u8, len: u32)
```

The `at ADDRESS` syntax is used for memory-mapped hardware registers. Common examples include `CUSTOM` at 0xdff000 (custom chips), `CIA_A` at 0xbfe001, and `CIA_B` at 0xbfd000.

Note: AmigaOS library functions are imported from the standard library FFI modules, not declared as `extern` in user code:

```
// Import AmigaOS functions from std::ffi
from std::ffi::exec import AllocMem, FreeMem, Wait, Signal
from std::ffi::dos import Open, Close, Read, Write
```

External declarations have no implementation in Novus—they're provided by AmigaOS, assembly files, or C libraries.

8.4 Re-exports

Re-exports allow modules to expose items from other modules as part of their own API. This is useful for creating convenient public interfaces.

8.4.1 The `pub use` Syntax

```
// ... (example from std/prelude.novus - illustrative)

// Re-export core types
pub use std::core::Option, Result, Drop, From

// Re-export string types
pub use std::strings::core::Str

// Re-export error types
```

```
pub use std::error::errors::DosError, ExecError, IntuitionError
```

Now users can import from `std::prelude` instead of multiple modules:

```
// Instead of:
from std::core import Option, Result
from std::strings::core import Str
from std::error::errors import DosError

// Users can write:
from std::prelude import Option, Result, Str, DosError
```

8.4.2 Multiple Re-exports

Re-export multiple items to create a convenient API:

```
// ... (example from std/async/mod.novus - illustrative)

// Re-export future types
pub use std::async::future::Future, Poll, Context, Waker
pub use std::async::future::Ready, Pending, ready, pending

// Re-export executor functions
pub use std::async::executor::block_on_sleep, poll_sleep_once

// Re-export sleep utilities
pub use std::async::sleep::Sleep, sleep, sleep_secs, sleep_millis
```

8.5 Constants and Static Variables

8.5.1 Constants (`const`)

Constants are compile-time values that are inlined at every use site:

```
const MAX_SPRITES: u32 = 8
const PI: i32 = 3_14159  // Fixed-point representation
pub const VERSION: u32 = 1

// Constants can use const functions
const fn double(x: i32) -> i32 { x * 2 }
const DOUBLED: i32 = double(21)  // 42 at compile time
```

Properties of constants:

- Must be computable at compile time
- No memory address (inlined at use sites)
- Always immutable
- Preferred for configuration values

8.5.2 Static Variables (`static`)

Static variables have a fixed memory location and live for the entire program:

```
// Immutable static - initialized data (size inferred from
    elements)
static SINE_TABLE: [i16] = [
    0, 804, 1608, 2410, 3212, 4011, 4808, // ...
]

// Mutable static - use with caution
static var frame_count: u32 = 0
static var initialized: bool = false
```

Properties of static variables:

- Have a memory address (stored in `.data` or `.bss`)
- Live for the entire program lifetime
- Immutable by default (`static var` for mutable)
- Must be initialized with a constant expression

8.5.3 Mutable Statics

Mutable static variables are declared with `static var`:

```
static var g_chip_pool: ChipPool = ChipPool {
    blocks: null,
    count: 0,
}

pub fn get_pool() -> *ChipPool {
    return &var g_chip_pool
}
```

Warning: Mutable statics are dangerous in multitasking environments. For thread-safe global state, consider using:

- Atomic types for simple counters
- Semaphores for complex data
- Immutable statics with interior mutability

8.6 Project Structure

Novus uses TOML configuration files to define projects and workspaces.

8.6.1 Single Project

A simple project structure:

```
myapp/
  project.toml       # Project configuration
  src/
    main.novus       # Entry point
    renderer.novus   # Additional module
    sound/
      player.novus   # Submodule
      mixer.novus
```

The `project.toml` file:

```
[package]
name = "myapp"
version = "0.1.0"
type = "cli"              # cli, workbench, dual, library, device
entry = "src/main.novus" # Entry point (optional, defaults to
    main.novus)
description = "My Amiga application"
authors = ["Your Name"]

[build]
target_cpu = "68020"      # 68000, 68020, 68040, 68060
fpu = "auto"              # soft, 68881, 68040, auto
output = "build"
optimization_level = 2

[paths]
src = "src"
```

8.6.2 Workspaces

Workspaces contain multiple related projects:

```
myworkspace/
  workspace.toml       # Workspace configuration
  mylib/
    project.toml
    src/
      lib.novus
  myapp/
    project.toml
    src/
      main.novus
  examples/
    project.toml
    src/
      demo.novus
```

The `workspace.toml` file:

```
[workspace]
name = "myworkspace"
version = "0.1.0"
members = ["mylib", "myapp", "examples"]  # Directory names of
    member projects

[workspace.build]
target_cpu = "68020"
fpu = "auto"
optimization_level = 2
```

The `members` array lists directory names of projects within the workspace.

8.6.3 Project Types

Novus supports several project types, each with different startup behavior:

`cli` Command-line application launched from Shell with `argc`/`argv` arguments

`workbench` Application launched by double-clicking icons; receives WBStartup message with file arguments

`dual` Application that handles both CLI and Workbench launches

`library` AmigaOS shared library (`.library`) with ROMTag and vector table

`device` AmigaOS device driver (`.device`) with BeginIO/AbortIO interface

Important: Workbench applications *must* reply to the WBStartup message using `ReplyMsg()` when exiting, or Workbench will hang. Use `Input() == 0` to detect Workbench launches.

8.6.4 Creating Projects

Use the `novus new` command to create projects:

```
# Create a new workspace
novus new myworkspace

# Create a CLI project in the workspace
cd myworkspace
novus new mytool --type cli

# Create a Workbench application
novus new mygui --type workbench

# Create a library
novus new mylib --type library
```

8.6.5 Building Projects

Build commands:

```
# Build all projects in workspace
novus build

# Build a specific project
novus build mytool

# Build with specific CPU target
novus build --cpu 68040
```

8.7 Standard Library Organization

The Novus standard library is organized into focused modules:

8.7.1 Core Modules

`std::core` Fundamental types: `Option`, `Result`, `Drop`, `Clone`, `Eq`, etc.

`std::prelude` Common re-exports for convenience

8.7.2 Data Structures

`std::collections::vec` Dynamic array (`Vec<T>`)

`std::collections::hashmap` Hash map (`HashMap<K, V>`)

`std::collections::hashset` Hash set (`HashSet<T>`)

`std::collections::vecdeque` Double-ended queue

`std::collections::ringbuffer` Fixed-size ring buffer

`std::collections::bitset` Bit set

`std::collections::slotmap` Slot map for entity storage

`std::collections::freelist` Free list allocator

`std::collections::smallvec` Small-buffer-optimized vector

`std::collections::intrusive_list` Intrusive doubly-linked list

8.7.3 Memory Management

`std::memory::block` Raw memory blocks (`MemoryBlock`)

`std::memory::chip_pool` Chip memory pool

`std::memory::chip_cache` Chip memory cache

8.7.4 Strings

`std::strings::core` String slice type (`Str`)

`std::strings::format` Formatting and `Display` trait

8.7.5 Error Handling

`std::error::errors` Error types: `DosError`, `ExecError`, `GraphicsError`, etc.

8.7.6 I/O and Networking

`std::io` Basic I/O functions

`std::net::socket` BSD socket interface

`std::net::tcp` TCP client/server

`std::net::udp` UDP sockets

`std::net::dns` DNS resolution

8.7.7 Concurrency

std::async Async/await runtime

std::sync Synchronization primitives

std::thread Task management

8.7.8 Graphics and UI

std::graphics::bitmap Bitmap handling

std::graphics::sprite Hardware sprites

std::graphics::copper Copper list generation

std::graphics::blitter Blitter operations

std::graphics::intuition Intuition window/screen wrappers

std::ui::window Window management

std::ui::screen Screen management

std::ui::menu Menu creation

std::ui::events Event handling

8.7.9 AmigaOS FFI

The FFI modules provide direct bindings to AmigaOS libraries:

std::ffi::exec Exec library (memory, tasks, signals)

std::ffi::dos DOS library (files, paths, CLI)

std::ffi::graphics Graphics library (rendering, text)

std::ffi::intuition Intuition library (windows, screens)

std::ffi::layers Layers library (layer management)

std::ffi::gadtools GadTools library (GUI toolkit)

std::ffi::utility Utility library (tags, hooks)

std::ffi::workbench Workbench library (icons, AppIcons)

std::ffi::icon Icon library (icon handling)

std::ffi::asl ASL library (file/font requesters)

std::ffi::diskfont Diskfont library (font loading)

std::ffi::commodities Commodities library (hotkeys)

std::ffi::iffparse IFFParse library (IFF files)

std::ffi::timer_device Timer device (delays, timing)

std::ffi::audio_device Audio device (Paula access)

std::ffi::bsdsocket BSD sockets (networking)

std::ffi::amiga_structs AmigaOS structure definitions

std::ffi::amiga_consts AmigaOS constants

8.7.10 Other Modules

std::math Fixed-point math, trigonometry

std::audio Paula audio, ProTracker player

std::test Test framework

std::os Operating system utilities

8.8 The Prelude

The prelude (`std::prelude`) re-exports commonly used types:

```
// ... (contents of std/prelude.novus - illustrative)
pub use std::core::Option, Result, Drop, From
pub use std::strings::core::Str
pub use std::error::errors::DosError, ExecError, IntuitionError,
    GraphicsError
```

Import the prelude for quick access to common types:

```
from std::prelude import *

pub fn main() -> i32 {
    let result: Result<i32, DosError> = do_something()
    match result {
        Ok(value) => return value,
        Err(_) => return 1
    }
}
```

8.9 Best Practices

8.9.1 Visibility Guidelines

1. **Default to private** — only make items `pub` when needed for external API
2. **Use `internal` for shared implementation** — better than making everything `pub`
3. **Document public APIs** — anything `pub` should have doc comments
4. **Keep modules focused** — one module, one responsibility

8.9.2 Import Guidelines

1. **Prefer specific imports** over wildcards for clarity
2. **Use wildcards sparingly** — only for well-known modules like `amiga_consts`

3. **Group imports logically** — core types, then collections, then FFI
4. **Use aliases** to avoid name conflicts

8.9.3 Project Organization

1. **Use workspaces** for multi-project development
2. **Separate library and application code** into different projects
3. **Keep tests in a separate project** that depends on the library
4. **Use meaningful directory structure** that matches module paths

8.9.4 Constants vs Statics

Use	When
`const`	Configuration values, magic numbers, version strings
`static`	Lookup tables, large data that shouldn't be inlined
`static var`	Global state (use sparingly, prefer safer alternatives)

8.10 Summary

This chapter covered the Novus module system:

- **Imports**: `from module::path import items`
- **Visibility**: Private (default), `pub`, `internal`, `extern`
- **Re-exports**: `pub use` for API convenience
- **Constants**: `const` for compile-time values
- **Statics**: `static` for fixed-location data
- **Projects**: `project.toml` and `workspace.toml`
- **Standard library**: Well-organized modules for all needs

Good module organization makes code easier to understand, maintain, and reuse. Start with private visibility and only expose what's needed for your public API.

The next chapter covers error handling with `Result` and `Option`.

Chapter 9

Attributes

"Metadata is a love note to the future." — Jason Scott

Attributes provide a way to attach metadata to code elements—functions, structs, enums, and variables. The compiler uses this metadata to alter behavior, generate code, or produce warnings. This chapter covers all implemented attributes with practical examples.

9.1 Understanding Attributes

In systems programming, we often need to tell the compiler things that can't be expressed in the code itself. Should this function be inlined? Is this struct packed for hardware access? Is this function safe to call from an interrupt handler?

Attributes provide these hints and directives. Unlike comments, attributes are part of the language—the compiler parses them, validates them, and acts on them.

Coming from C

GCC/Clang Attributes: C has `__attribute__((packed))`, `__attribute__((always_inline))`, etc. Novus uses cleaner `@packed` and `@inline` syntax. Same ideas, better ergonomics.

9.2 Attribute Syntax

Novus supports two equivalent attribute syntaxes. Choose whichever feels more natural for your codebase.

9.2.1 At-Sign Syntax (Recommended)

The primary syntax uses `@` followed by the attribute name:

```
@test
fn test_addition() {
    expect_eq(2 + 2, 4, "basic addition")
}

@inline
fn fast_add(a: i32, b: i32) -> i32 {
    return a + b
}
```

This syntax is concise and visually distinct from code.

9.2.2 Bracket Syntax

An alternative bracket syntax is also supported:

```
#[test]
fn test_multiplication() {
    expect_eq(3 * 7, 21, "multiplication")
}

#[derive(Eq, Hash)]
struct Point {
    x: i32,
    y: i32,
}
```

The bracket syntax is useful when attributes have complex arguments.

9.2.3 Attribute Arguments

Attributes can accept arguments—both named and positional:

```
// Named arguments
@library(name = "example.library", version = 1, revision = 0)
pub struct ExampleLibrary {
    counter: u32,
}

// Positional arguments
@test("Verify sorting performance")
fn test_sort_perf() {
    // ...
}

// Flag arguments
@test(skip = "Not yet implemented")
fn test_future() {
    // Skipped during test runs
}

// Mixed arguments
@libfunc(offset = -30)  // LVO offset -30 bytes
pub fn my_library_function() -> i32 {
    return 42
}
```

9.2.4 Multiple Attributes

Multiple attributes can be stacked on a single item:

```
@test
@inline
pub fn test_fast_path() {
    // This function is both a test case and should be inlined
}

@interrupt
@export
```

```
pub fn vblank_handler() {
    // Interrupt handler that must be preserved in the binary
}
```

The order of attributes generally doesn't matter, though some combinations are invalid (the compiler will warn you).

9.3 Testing Attributes

Novus has first-class support for testing through attributes.

9.3.1 @test

Marks a function as a test case. The `novus test` command discovers and compiles these into a test runner.

```
@test
fn test_basic() {
    expect_eq(1 + 1, 2, "one plus one")
}

@test("Descriptive test name")
fn test_with_description() {
    expect_eq(true, true, "truth is true")
}
```

Arguments:

positional string Optional test description

`skip = "reason"` Skip this test with an explanation

```
@test(skip = "Hardware not available in CI")
fn test_blitter_operations() {
    // This test requires real Amiga hardware
}

@test(skip)
fn test_known_broken() {
    // Skipped without explanation (not recommended)
}
```

9.3.2 @bench / @benchmark

Marks a function for performance testing. Both names are equivalent:

```
@bench
fn bench_sort() {
    var data = generate_random_data(1000)
    sort(&var data)
}

@benchmark
fn bench_hash() {
    var map = HashMap::new()
```

```
    for i in 0..1000 {
        map.insert(i, i * 2)
    }
}
```

Benchmark functions are included when you run `novus test -b`. The runner executes each benchmark multiple times and reports timing statistics.

9.4 Optimization Attributes

These attributes guide the compiler's optimization decisions.

9.4.1 @inline

Hints that the compiler should inline this function at call sites:

```
@inline
fn min(a: i32, b: i32) -> i32 {
    if a < b { return a } else { return b }
}

@inline
fn abs(x: i32) -> i32 {
    if x < 0 { return -x } else { return x }
}
```

Small, frequently-called functions benefit most from inlining. Benefits include:

- Elimination of function call overhead
- Better register allocation across the inlined code
- Opportunities for constant folding when arguments are known

The compiler may ignore this hint for:

- Recursive functions
- Very large functions
- Functions with complex control flow

9.4.2 @noinline

Prevents inlining even when the compiler might otherwise choose to inline:

```
@noinline
fn error_handler() {
    // Keep error handling out of hot paths
    // to improve instruction cache locality
    log_error("Something went wrong")
    cleanup_resources()
}

@noinline
fn cold_path_check(value: i32) -> bool {
    // Rarely-taken branch - don't pollute hot code
```

```
    return validate_complex_condition(value)
}
```

Use `@noinline` for:

- Error handling code (rarely executed)
- Large functions called from tight loops
- Functions where you need predictable stack usage

9.5 Layout Attributes

9.5.1 @packed

Removes padding between struct fields:

```
// Without @packed: 8 bytes (a=1, padding=3, b=4)
struct Normal {
    a: u8,
    b: u32,
}

// With @packed: 5 bytes (a=1, b=4, no padding)
@packed
struct Packed {
    a: u8,
    b: u32,
}
```

Coming from C

C Equivalent: `#pragma pack(1)` or `__attribute__((packed))`. Same effect, but Novus makes it explicit per-struct.

Warning: Packed structs may have unaligned fields, which can cause:

- Slower memory access (multiple bus cycles)
- Address errors on 68000/68010 (crashes!)
- Correct but slow access on 68020+

Use when:

- Matching external data formats (file formats, network protocols)
- Mapping hardware registers
- Memory-constrained situations where every byte matters

9.6 Trait Derivation

9.6.1 @derive / #[derive]

Automatically generates trait implementations for structs:

```
#[derive(Eq)]
struct Point {
    x: i32,
    y: i32,
}

#[derive(Eq, Hash)]
struct UserId {
    id: u32,
}
```

Supported traits:

`Eq` Generates `eq(&self, &Self) -> bool` comparing all fields

`Hash` Generates `hash(&self) -> u32` combining field hashes

You can derive multiple traits at once:

```
#[derive(Eq, Hash)]
struct CacheKey {
    module: u32,
    offset: u32,
}

// Now CacheKey can be used as HashMap key
var cache = HashMap::<CacheKey, Data>::new()
```

9.7 Method Chaining

9.7.1 @chain

Enables fluent method chaining by making a method return `&var Self`:

```
struct WindowBuilder {
    width: i32,
    height: i32,
    title: *u8,
    flags: u32,
}

impl WindowBuilder {
    pub fn new() -> WindowBuilder {
        return WindowBuilder {
            width: 640,
            height: 480,
            title: "Window",
            flags: 0,
        }
    }

    @chain
    pub fn width(&var self, w: i32) {
        self.width = w
    }
```

```
    @chain
    pub fn height(&var self, h: i32) {
        self.height = h
    }

    @chain
    pub fn title(&var self, t: *u8) {
        self.title = t
    }

    pub fn build(&self) -> Window {
        // Create actual window...
    }
}
```

Usage:

```
var window = WindowBuilder::new()
    .width(800)
    .height(600)
    .title("My Application")
    .build()
```

The `@chain` attribute automatically adds an implicit `return self` at the end of the method.

9.8 Interoperability Attributes

9.8.1 @export

Marks a function for export with an unmangled name, preventing dead code elimination:

```
@export
pub fn my_callback(data: *u8) -> i32 {
    // Can be called from external C code or AmigaOS
    return process(data)
}

@export
pub fn _UserLibInit() -> i32 {
    // Entry point called by AmigaOS
    return initialize_library()
}
```

Use this when:

- Creating callback functions for AmigaOS (hooks, interrupt handlers)
- Functions that need to be called from assembly or C
- Entry points that might appear unused to the optimizer

9.8.2 @extern_type

Marks a struct as an external C type, preventing Novus from emitting its definition:

```
@extern_type
struct timeval {
    tv_sec: i32,
    tv_usec: i32,
}

@extern_type
struct Library {
    node: Node,
    flags: u8,
    // ... matches C header definition
}
```

This tells the compiler: "This type is defined elsewhere (in C headers or runtime); just use it, don't emit a definition."

9.9 Synchronization Attributes

These attributes provide automatic synchronization wrappers.

9.9.1 @atomic

Wraps the function body in a Forbid/Permit critical section:

```
@atomic
pub fn update_shared_counter(counter: *u32) {
    // Forbid() called on entry
    *counter = *counter + 1
    // Permit() called on exit (even if returning early)
}

@atomic
pub fn swap_values(a: *i32, b: *i32) {
    let temp = *a
    *a = *b
    *b = temp
    // No task can switch during this operation
}
```

This prevents task switching while the function executes. The compiler ensures Permit() is called on all exit paths.

Use for:

- Short, fast operations on shared data
- Lock-free data structure updates
- Counter increments visible to multiple tasks

Avoid for:

- Functions that might block (I/O, waiting)
- Long-running computations
- Anything that allocates memory

9.9.2 @no_interrupts

Wraps the function body in Disable/Enable:

```
@no_interrupts
pub fn poke_hardware_register(addr: *u16, value: u16) {
    // Disable() called on entry - ALL interrupts blocked
    *addr = value
    // Enable() called on exit
}

@no_interrupts
pub fn read_modify_write(reg: *u16, mask: u16, value: u16) {
    let current = *reg
    *reg = (current & ~mask) | (value & mask)
}
```

Warning: Use sparingly! This blocks *all* interrupts including:

- Audio DMA (causes audio glitches)
- Disk I/O (can corrupt data)
- Vertical blank (freezes display updates)
- Timer interrupts (breaks timing)

Only use for timing-critical hardware access that must be atomic.

Coming from C

AmigaOS: Disable() disables all hardware interrupts by setting the processor interrupt mask. Enable() restores it. These are Exec functions, not direct hardware access.

9.10 Interrupt Handling Attributes

These attributes are essential for writing interrupt-driven code.

9.10.1 @interrupt

Marks a function as an interrupt handler. The compiler generates appropriate prologue/epilogue code:

```
@interrupt
pub fn vblank_handler() {
    // Called during vertical blank interrupt
    // All registers properly saved/restored
    update_frame_counter()
}

@interrupt
pub fn audio_handler() {
    // Called when audio channel needs more data
    feed_next_sample()
}
```

The compiler generates:

- Register save (MOVEM.L to stack)
- Your function body
- Register restore (MOVEM.L from stack)
- RTE (Return from Exception)

9.10.2 @interrupt_safe

Documents that a function is safe to call from interrupt context:

```
@interrupt_safe
pub fn increment_counter(counter: *u32) {
    // Safe: no allocations, no blocking calls
    *counter = *counter + 1
}

@interrupt_safe
fn calculate_checksum(data: *u8, len: u32) -> u32 {
    // Safe: pure computation, no system calls
    var sum: u32 = 0
    for i in 0..len {
        sum = sum + (data[i] as u32)
    }
    return sum
}
```

Functions marked `@interrupt_safe` must:

- Never allocate memory
- Never call blocking functions (Wait, WaitIO, etc.)
- Never call Permit() without matching Forbid()
- Complete quickly (microseconds, not milliseconds)

This is currently documentation-only but helps communicate intent and catch errors during code review.

9.11 Diagnostic Attributes

9.11.1 @suppress

Suppresses specific compiler warnings:

```
@suppress("W0002", "Known safe: buffer size validated externally")
pub fn write_unchecked(buffer: *u8, data: *u8, len: i32) {
    // Warning W0002 (unchecked buffer access) suppressed
    // We accept responsibility for bounds checking
}

@suppress("W0001")  // Minimum suppression - just the code
pub fn use_deprecated_api() {
    call_old_function()
}
```

Arguments:

warning code The warning ID to suppress (e.g., `"W0001"`)

justification Explanation of why the warning is safe to ignore (recommended)

Always prefer fixing warnings over suppressing them. When suppression is necessary, document why.

9.12 Program Configuration

9.12.1 #[stack_size]

Sets the stack size for the compiled program:

```
#[stack_size(131072)]  // 128KB stack

fn main() -> i32 {
    // Programs with deep recursion need more stack
    return 0
}
```

Stack Size Guidelines:

Use Case	Recommended Size
Simple programs	8KB – 16KB
Typical applications	32KB – 64KB (default)
Deep recursion (parsers, tree traversal)	128KB – 256KB
Large stack arrays	Array size + 32KB buffer

Table 9.1: Stack Size Recommendations

Note: AmigaOS CLI default stack is only 4KB, far too small for most Novus programs. The Novus runtime automatically checks and increases stack size if needed.

9.13 Library Attributes

These attributes are used when creating AmigaOS shared libraries.

9.13.1 @library

Marks a struct as an Amiga shared library:

```
@library(name = "example.library", version = 1, revision = 0)
pub struct ExampleLibrary {
    base: Library,  // Must include Library base
    counter: u32,
    initialized: bool,
}
```

Arguments:

`name` Library name (must end with `.library`)

`version` Major version number

revision Minor revision number

The compiler generates:

- ROMTag structure
- Library vector table
- Open/Close/Expunge handlers
- Proper HUNK structure for library loading

9.13.2 @libfunc

Marks a method as a library vector entry point:

```
impl ExampleLibrary {
    @libfunc
    pub fn add_numbers(&self, a: i32, b: i32) -> i32 {
        return a + b
    }

    @libfunc
    pub fn get_counter(&self) -> u32 {
        return self.counter
    }
}
```

Library functions are called through the library base pointer, following AmigaOS calling conventions.

9.13.3 Library Lifecycle Hooks

```
impl ExampleLibrary {
    @libinit
    pub fn init() -> bool {
        // One-time initialization when library loads
        // Allocate resources, open dependencies
        return true
    }

    @libopen
    pub fn open(version: u32) -> bool {
        // Called each time library is opened
        if version > 1 { return false }  // Version check
        self.counter = self.counter + 1
        return true
    }

    @libclose
    pub fn close() {
        // Called each time library is closed
        self.counter = self.counter - 1
    }

    @libexpunge
    pub fn expunge() -> bool {
        // Return true if library can be unloaded
```

```
        // Return false to keep library resident
        return self.counter == 0
    }
}
```

9.14 Device Attributes

These attributes are used when creating AmigaOS devices.

9.14.1 @device

Marks a struct as an Amiga device:

```
@device(name = "example.device", version = 1, revision = 0)
pub struct ExampleDevice {
    base: Device,
    unit: u32,
    active: bool,
}
```

9.14.2 @devicecmd

Marks a method as a device command handler:

```
impl ExampleDevice {
    @devicecmd(CMD_READ)
    pub fn handle_read(&self, io: *IORequest) {
        // Handle read command
        io.io_Actual = read_data(io.io_Data, io.io_Length)
    }

    @devicecmd(CMD_WRITE)
    pub fn handle_write(&self, io: *IORequest) {
        // Handle write command
        write_data(io.io_Data, io.io_Length)
        io.io_Actual = io.io_Length
    }
}
```

9.14.3 @beginio / @abortio

Standard device I/O handlers:

```
impl ExampleDevice {
    @beginio
    pub fn begin_io(&self, io: *IORequest) {
        // Start I/O operation
        let cmd = io.io_Command
        match cmd {
            CMD_READ => self.handle_read(io),
            CMD_WRITE => self.handle_write(io),
            _ => io.io_Error = IOERR_NOCMD,
        }
```

```
    }

    @abortio
    pub fn abort_io(&self, io: *IORequest) -> i32 {
        // Abort pending I/O
        // Return 0 if aborted, error otherwise
        return 0
    }
}
```

9.14.4 Device Lifecycle Hooks

Similar to libraries: `@deviceinit`, `@deviceopen`, `@deviceclose`, `@deviceexpunge`.

9.15 Common Attribute Patterns

9.15.1 Interrupt-Safe State Updates

Combining `@atomic` with `@interrupt_safe`:

```
static var frame_count: u32 = 0
static var frames_per_second: u32 = 0

@interrupt_safe
@inline
fn increment_frame_count() {
    frame_count = frame_count + 1
}

@atomic
pub fn get_and_reset_fps() -> u32 {
    let fps = frames_per_second
    frames_per_second = 0
    return fps
}

@interrupt
@export
pub fn vblank_interrupt() {
    increment_frame_count()
    if frame_count >= 50 {  // PAL: 50 frames = 1 second
        frames_per_second = frame_count
        frame_count = 0
    }
}
```

9.15.2 Builder Pattern with Validation

```
struct ConfigBuilder {
    width: Option<i32>,
    height: Option<i32>,
    depth: i32,
}
```

```
impl ConfigBuilder {
    pub fn new() -> ConfigBuilder {
        return ConfigBuilder {
            width: Option::None,
            height: Option::None,
            depth: 8,
        }
    }

    @chain
    pub fn width(&var self, w: i32) {
        self.width = Option::Some(w)
    }

    @chain
    pub fn height(&var self, h: i32) {
        self.height = Option::Some(h)
    }

    @chain
    pub fn depth(&var self, d: i32) {
        self.depth = d
    }

    pub fn build(&self) -> Result<Config, ConfigError> {
        // Validate all required fields are set
        let w = match self.width {
            Option::Some(v) => v,
            Option::None => return
                Result::Err(ConfigError::MissingWidth),
        }
        let h = match self.height {
            Option::Some(v) => v,
            Option::None => return
                Result::Err(ConfigError::MissingHeight),
        }
        return Result::Ok(Config { width: w, height: h, depth:
            self.depth })
    }
}
```

9.16 Showcase: Interrupt-Driven Music Player

This showcase demonstrates interrupt and synchronization attributes for audio playback.

```
// music_player_showcase.novus - Demonstrates interrupt attributes

from std::core import Option
from std::io::file import print, println
from std::ffi::exec import Forbid, Permit, Disable, Enable

// Player state stored as globals for interrupt access
static var g_is_playing: bool = false
static var g_current_position: u32 = 0
static var g_sample_length: u32 = 0
static var g_tempo_counter: u32 = 0
```

```
static var g_tempo_divider: u32 = 6  // ~8 ticks/sec at PAL
static var g_volume: u16 = 64

// @interrupt_safe: Documents this is safe to call from interrupt
// @inline: Hint to inline for performance
@interrupt_safe
@inline
fn advance_position() -> bool {
    if g_current_position >= g_sample_length {
        return false
    }
    g_current_position = g_current_position + 1u32
    return true
}

// @no_interrupts: Wraps in Disable/Enable for hardware access
@no_interrupts
fn write_audio_volume(volume: u16) {
    g_volume = volume
    // In real code: write to Paula register here
}

// @atomic: Wraps in Forbid/Permit for task-safe state access
@atomic
fn start_playback(length: u32, tempo: u32) -> bool {
    if g_is_playing { return false }
    g_sample_length = length
    g_current_position = 0u32
    g_tempo_divider = tempo
    g_is_playing = true
    return true
}

@atomic
fn stop_playback() {
    if g_is_playing {
        g_is_playing = false
        write_audio_volume(0u16)
    }
}

// @interrupt: Generates proper save/restore and RTE
// @export: Prevents dead code elimination
@interrupt
@export
pub fn vblank_music_handler() {
    if !g_is_playing { return }

    g_tempo_counter = g_tempo_counter + 1u32
    if g_tempo_counter < g_tempo_divider { return }
    g_tempo_counter = 0u32

    if !advance_position() {
        g_is_playing = false
    }
}

pub fn main() -> i32 {
```

```
    println("Key Attributes Demonstrated:")
    println("  @interrupt      - VBlank handler")
    println("  @interrupt_safe - Safe helper functions")
    println("  @atomic         - Critical sections")
    println("  @no_interrupts  - Hardware access")
    println("  @inline         - Performance hints")
    println("  @export         - Callback preservation")
    return 0
}
```

This pattern is typical for:

- MOD/XM music players
- Sample playback engines
- Sound effect systems
- Any audio requiring precise timing

9.17 Attribute Reference

Attribute	Applies To	Purpose
`@test`	functions	Mark as test case
`@bench`	functions	Mark for benchmarking
`@inline`	functions	Hint to inline
`@noinline`	functions	Prevent inlining
`@packed`	structs	Remove field padding
`@derive`	structs	Auto-implement traits
`@chain`	methods	Enable method chaining
`@export`	functions	Keep and don't mangle name
`@extern_type`	structs	External C type
`@atomic`	functions	Wrap in Forbid/Permit
`@no_interrupts`	functions	Wrap in Disable/Enable
`@interrupt`	functions	Mark as interrupt handler
`@interrupt_safe`	functions	Safe for interrupt context
`@suppress`	functions	Suppress warning
`#[stack_size]`	program	Set stack size
`@library`	structs	Define Amiga library
`@libfunc`	methods	Library vector entry
`@device`	structs	Define Amiga device
`@devicecmd`	methods	Device command handler

Table 9.2: Complete Attribute Reference

9.18 Unknown Attributes

If you use an unknown attribute, the compiler emits a warning:

```
warning[W2001]: unknown attribute 'foo'
  --> test.novus:3:1
   |
 3 | @foo
```

```
   | ^^^
   |
help: This attribute is not recognized and will be ignored
help: Known attributes: library, libfunc, inline, test, ...
```

This helps catch typos while allowing forward compatibility with future attributes.

9.19 Future Attributes

The following attributes are planned for future versions:

`@deprecated` Mark functions as deprecated with migration notes

`@must_use` Require return value to be used

`@cold` Mark function as unlikely to be called (affects code layout)

`@hot` Mark function as frequently called (affects optimization)

`@pure` Function has no side effects (enables memoization)

Chapter 10

Error Handling

> *"Errors should never pass silently. Unless explicitly silenced."* — The Zen of Python

Novus treats errors as values, not exceptions. Every operation that can fail returns a `Result` type, making error handling explicit and impossible to ignore. This chapter covers Novus's error handling philosophy, the standard error types, and patterns for writing robust code.

10.1 Philosophy: Errors as Values

Unlike languages with exceptions, Novus requires explicit handling of errors:

- **No hidden control flow** — You always know when a function can fail
- **No uncaught exceptions** — Errors can't silently propagate up the stack
- **Compile-time enforcement** — Ignoring a `Result` produces a warning
- **Zero runtime cost** — No exception tables or unwinding machinery

10.1.1 Why Not Exceptions?

Many languages use exceptions for error handling. While convenient in some ways, exceptions have significant drawbacks for systems programming:

1. Stack Unwinding Cost: Exception handling requires runtime machinery to unwind the stack, search for handlers, and call destructors. On a 68020 running at 14 MHz, this overhead is unacceptable.

2. Invisible Control Flow: Any function call might throw, making control flow analysis difficult. In Novus, functions that can fail are marked by their return type.

3. Unpredictable Cleanup: With exceptions, you need `try/finally` blocks to ensure resources are released. In Novus, `Result` combinators and pattern matching make cleanup explicit.

4. Binary Size: Exception tables and unwinding code add significant size to executables. Every byte counts on classic Amiga systems.

10.1.2 Why Not errno?

The traditional C approach of global error codes has its own problems:

```
// C with errno
FILE *f = fopen("data.txt", "r");
if (f == NULL) {
    if (errno == ENOENT) { /* not found */ }
    else if (errno == EACCES) { /* permission denied */ }
```

```
    // Easy to forget, hard to get right
}
```

Problems with errno:

- Global mutable state is inherently error-prone
- Easy to forget to check it
- Can be overwritten by intermediate calls
- Not type-safe — any integer could be assigned
- Thread-unsafe without special handling

AmigaOS uses a similar pattern with `IoErr()`:

```
// AmigaOS with IoErr()
BPTR lock = Lock("myfile", ACCESS_READ);
if (lock == 0) {
    LONG err = IoErr();  // Must call immediately!
    // But what if you call something else first?
    PrintFault(err, "Error");
}
```

The danger: if you call *any* other DOS function before `IoErr()`, the error code may be overwritten.

Coming from C

In C, errors are typically indicated by return values (`-1`, `NULL`) or global `errno`/`IoErr()`. This leads to bugs when callers forget to check.
In Novus, `Result<T, E>` forces you to handle both success and failure cases. You cannot accidentally use an error value as if it were successful.

C (AmigaOS)	Novus
`if (ptr == NULL)`	`match result { ... }`
`if (fd < 0)`	`let file = open(path)?`
`IoErr()`	`Result::Err(DosError::NotFound)`
Manual conversion	`From` trait auto-conversion

10.2 The Result Type

The core error handling type is `Result<T, E>`:

```
enum Result<T, E> {
    Ok(T),    // Success with value
    Err(E),   // Failure with error
}
```

Every function that can fail returns a `Result`:

```
fn parse_number(s: *u8) -> Result<i32, ParseError> {
    // Returns Ok(value) or Err(error)
}
```

```
fn open_file(path: *u8) -> Result<FileHandle, DosError> {
    // Returns Ok(handle) or Err(error)
}
```

10.2.1 Result Methods Reference

The `Result` type provides several methods for working with values:

Method	Description
`is_ok()`	Returns `true` if the result is `Ok`
`is_err()`	Returns `true` if the result is `Err`
`unwrap()`	Extracts the `Ok` value, panics on `Err`
`unwrap_or(default)`	Returns the `Ok` value or the provided default
`expect(msg)`	Like `unwrap()` but with a custom panic message
`ok()`	Converts to `Option<T>`, discarding the error
`err()`	Converts to `Option<E>`, discarding the success value

```
let result: Result<i32, Error> = some_operation()

// Query the state
if result.is_ok() {
    println("Operation succeeded")
}

// Get value with default
let value = result.unwrap_or(0)

// Convert to Option
let maybe_value: Option<i32> = result.ok()
```

10.3 The Option Type

While `Result` represents success or failure, `Option` represents presence or absence:

```
enum Option<T> {
    Some(T),  // Value present
    None,     // No value
}
```

10.3.1 When to Use Option vs Result

- Use **Option** when the only failure mode is "not found" or "not available"
- Use **Result** when there are multiple failure modes to distinguish
- Use **Result** for operations that interact with the OS or hardware

```
// Option: "not found" is the only failure
fn find_user(id: u32) -> Option<User> { ... }
```

```
// Result: multiple failure modes possible
fn load_user(id: u32) -> Result<User, DbError> { ... }
```

10.3.2 Option Methods Reference

Method	Description
`is_some()`	Returns `true` if value is present
`is_none()`	Returns `true` if no value
`unwrap()`	Extracts the value, panics on `None`
`unwrap_or(default)`	Returns value or provided default
`expect(msg)`	Like `unwrap()` with custom panic message

10.3.3 Converting Between Option and Result

Sometimes you need to convert between the two types:

```
// Option -> Result: provide an error for None
fn get_config_value(key: *u8) -> Result<*u8, ConfigError> {
    match config.get(key) {
        Option::Some(val) => Result::Ok(val),
        Option::None => Result::Err(ConfigError::MissingKey)
    }
}
```

Coming from C

In C, null pointers serve double duty: representing "no value" (Option) and "error occurred" (Result). This conflation causes bugs.

```
// C: Is NULL "not found" or "error"?
Node *node = find_node(list, key);
if (node == NULL) {
    // Did find fail, or is the key just missing?
    // Have to check errno to know...
}
```

In Novus, the type system distinguishes these cases. `Option<T>` means "might not exist" while `Result<T, E>` means "might have failed".

10.4 The ? Operator In Depth

The `?` operator is syntactic sugar for early return on error. It transforms verbose error checking into concise, readable code.

10.4.1 What ? Actually Does

When you write:

```
let file = open_file(path)?
```

The compiler generates equivalent code to:

```
let file = match open_file(path) {
    Result::Ok(val) => val,
    Result::Err(e) => return Result::Err(e)
}
```

The `?` operator:

1. Checks if the result is `Err`
2. If `Err`, returns immediately with that error
3. If `Ok`, unwraps the value and continues

10.4.2 Chaining Multiple ? Calls

The real power of `?` shows when chaining fallible operations:

```
fn process_file(path: *u8) -> Result<Data, DosError> {
    let file = open_file(path)?       // Returns Err early if
        failed
    let header = read_header(&file)?  // Returns Err early if
        failed
    let body = read_body(&file)?      // Returns Err early if
        failed
    let data = parse_data(&body)?     // Returns Err early if
        failed
    Result::Ok(data)
}
```

Without `?`, this would require deeply nested match statements:

```
fn process_file(path: *u8) -> Result<Data, DosError> {
    match open_file(path) {
        Result::Ok(file) => {
            match read_header(&file) {
                Result::Ok(header) => {
                    match read_body(&file) {
                        Result::Ok(body) => {
                            match parse_data(&body) {
                                Result::Ok(data) =>
                                    Result::Ok(data),
                                Result::Err(e) => Result::Err(e)
                            }
                        },
                        Result::Err(e) => Result::Err(e)
                    }
                },
                Result::Err(e) => Result::Err(e)
            }
        },
        Result::Err(e) => Result::Err(e)
    }
}
```

10.4.3 Using ? in main()

In Novus, `main()` returns `i32` (the exit code). You can still use `?` by wrapping your logic:

```
fn run() -> Result<(), AppError> {
    let config = load_config()?
    let window = open_window()?
    main_loop(&window)?
    Result::Ok(())
}

pub fn main() -> i32 {
    match run() {
        Result::Ok(_) => return 0,
        Result::Err(err) => {
            println(err.message())
            return 1
        }
    }
}
```

10.4.4 Limitations of ?

The `?` operator can only be used in functions that return `Result`:

```
fn valid() -> Result<(), Error> {
    let x = fallible_op()?  // OK: function returns Result
    Result::Ok(())
}

fn invalid() {
    let x = fallible_op()?  // ERROR: function doesn't return
       Result
}
```

10.5 Standard Error Types

Novus provides error types for each AmigaOS subsystem.

10.5.1 DosError

Errors from the DOS library (file operations, paths):

```
pub enum DosError {
    NotFound,            // File/directory not found (IoErr 205)
    AlreadyExists,       // Object already exists (IoErr 203)
    DirNotFound,         // Directory not found (IoErr 212)
    DirectoryNotEmpty,   // Can't delete non-empty dir (IoErr 216)
    PermissionDenied,    // Access denied (IoErr 214)
    WriteProtected,      // Disk is write-protected (IoErr 223)
    ReadProtected,       // File is read-protected (IoErr 224)
    DiskFull,            // No space on disk (IoErr 221)
    NoDisk,              // No disk in drive (IoErr 218)
    DeviceNotMounted,    // Device not available (IoErr 225)
    InvalidLock,         // Bad file lock (IoErr 211)
    Unknown,             // Unmapped error code
}
```

DosError Variant	IoErr() Code	Description
NotFound	205	Object not found
AlreadyExists	203	Object already exists
DirNotFound	212	Directory not found
DirectoryNotEmpty	216	Directory not empty
PermissionDenied	214	Object is protected
WriteProtected	223	Disk is write-protected
ReadProtected	224	File is read-protected
DiskFull	221	Disk is full
NoDisk	218	No disk in drive
DeviceNotMounted	225	Device not mounted
InvalidLock	211	Invalid lock

10.5.2 ExecError

Errors from the Exec library (memory, tasks, signals):

```
pub enum ExecError {
    NoMem,              // Memory allocation failed
    BadSignal,          // Invalid signal number
    NullReturn,         // Unexpected null return
    BadTask,            // Invalid task pointer
    BadPort,            // Invalid message port
    SignalInUse,        // Signal already allocated
    TaskNotFound,       // FindTask failed
    LibraryNotFound,    // OpenLibrary failed
    IndexOutOfBounds,   // Array bounds violation
    CapacityExceeded,   // Fixed buffer full
}
```

10.5.3 IntuitionError

Errors from the Intuition library (windows, screens, gadgets):

```
pub enum IntuitionError {
    WindowOpenFailed,     // OpenWindow failed
    ScreenOpenFailed,     // OpenScreen failed
    GadgetCreateFailed,   // Gadget creation failed
    MenuCreateFailed,     // Menu creation failed
    InvalidWindow,        // Bad window pointer
    InvalidScreen,        // Bad screen pointer
    NoIDCMP,              // IDCMP setup failed
    BadDisplayMode,       // Invalid display mode
}
```

10.5.4 GraphicsError

Errors from the Graphics library (bitmaps, sprites, fonts):

```
pub enum GraphicsError {
    BitMapAllocFailed,    // BitMap allocation failed
    FontOpenFailed,       // OpenFont failed
    InvalidBitMap,        // Bad BitMap pointer
    InvalidRastPort,      // Bad RastPort pointer
```

```
    InvalidSpriteChannel, // Bad sprite channel (0-7)
    SpriteChannelInUse,   // Channel already allocated
    NoSpritesAvailable,   // All sprites in use
}
```

10.6 The NovusError Wrapper

When a function may fail with errors from multiple subsystems, use `NovusError`:

```
pub enum NovusError {
    Dos(DosError),
    Exec(ExecError),
    Intuition(IntuitionError),
    Graphics(GraphicsError),
}
```

The `From` trait implementations enable automatic conversion with the `?` operator:

```
fn load_and_display(path: *u8) -> Result<(), NovusError> {
    let data = read_file(path)?       // DosError -> NovusError
    let screen = open_screen()?       // IntuitionError ->
        NovusError
    let bitmap = load_bitmap(&data)?  // GraphicsError ->
        NovusError
    display(screen, bitmap)
    Result::Ok(())
}
```

10.7 Error Conversion with From

The `From` trait enables automatic error type conversion when using `?`:

```
trait From<T> {
    fn convert(value: T) -> Self
}
```

When you use `?` in a function returning `Result<T, NovusError>`, and the inner call returns `Result<T, DosError>`, the compiler automatically calls `NovusError::From<DosError>::convert()` to wrap the error.

10.7.1 Manual Conversion

You can also convert errors manually:

```
from std::error::errors import novus_error_from_dos

fn wrapper() -> Result<(), NovusError> {
    let file = match open("file.txt", MODE_OLDFILE) {
        Result::Ok(f) => f,
        Result::Err(e) => return
            Result::Err(novus_error_from_dos(e))
    }
    Result::Ok(())
```

```
}
```

10.7.2 Getting Raw Error Codes

For debugging or interop with C code, get raw AmigaOS error codes:

```
from std::error::errors import dos_error_to_code

let code = dos_error_to_code(DosError::NotFound)  // Returns 205
```

10.8 Error Messages

All standard error types implement a `message()` method:

```
match result {
    Result::Err(err) => {
        print("Error: ")
        println(err.message())
    },
    _ => {}
}
```

Error messages are human-readable strings:

- `DosError::NotFound` → `"Object not found"`
- `ExecError::NoMem` → `"Memory allocation failed"`
- `IntuitionError::WindowOpenFailed` → `"Failed to open window"`

10.9 Error Handling Patterns

This section covers common patterns for handling errors effectively.

10.9.1 Early Return Pattern

Validate inputs at the start of a function and return errors immediately. This keeps the main logic clean:

```
fn process(data: *u8, len: u32) -> Result<(), Error> {
    // Validate all inputs first
    if data == null {
        return Result::Err(Error::InvalidInput)
    }
    if len == 0 {
        return Result::Err(Error::EmptyData)
    }
    if len > MAX_SIZE {
        return Result::Err(Error::TooLarge)
    }

    // All validation passed - main logic here
    do_actual_work(data, len)
}
```

Coming from C

In C, you often see scattered validation throughout functions:

```
int process(char *data, size_t len) {
    int result = init_something();
    if (result < 0) return result;

    if (data == NULL) return -EINVAL;  // Mixed with logic!

    result = do_more_work();
    if (result < 0) goto cleanup;

    if (len == 0) goto cleanup;  // Validation too late!
    // ...
}
```

The early return pattern in Novus collects all validation at the top, making the "happy path" obvious.

10.9.2 Fallback Chains

Try multiple approaches, falling back on failure. This is common for finding configuration files or resources:

```
fn find_config() -> Result<Config, ConfigError> {
    // Try user config first
    match load_from("ENV:app.cfg") {
        Result::Ok(cfg) => return Result::Ok(cfg),
        Result::Err(_) => {}
    }

    // Fall back to archive
    match load_from("ENVARC:app.cfg") {
        Result::Ok(cfg) => return Result::Ok(cfg),
        Result::Err(_) => {}
    }

    // Fall back to system config
    match load_from("S:app.cfg") {
        Result::Ok(cfg) => return Result::Ok(cfg),
        Result::Err(_) => {}
    }

    // Use defaults as last resort
    Result::Ok(default_config())
}
```

10.9.3 Cleanup on Error with defer

Use `defer` to ensure cleanup even when returning early on error:

```
fn process_file(path: *u8) -> Result<Data, DosError> {
    let file = open(path, MODE_OLDFILE)?
    defer { close(file) }  // Always runs, even on error return
```

```
    let header = read_header(&file)?  // May return early
    let body = read_body(&file)?      // May return early

    Result::Ok(combine(header, body))
    // defer runs here on success too
}
```

10.9.4 Error Context

Add context when propagating errors by matching and re-wrapping with domain-specific information:

```
fn load_user_config() -> Result<Config, ConfigError> {
    let path = get_config_path()?

    let file = match open(path, MODE_OLDFILE) {
        Result::Ok(f) => f,
        Result::Err(_) => return
            Result::Err(ConfigError::FileNotFound)
    }

    let data = match read_all(&file) {
        Result::Ok(d) => d,
        Result::Err(_) => return
            Result::Err(ConfigError::ReadFailed)
    }

    parse_config(&data)
}
```

10.9.5 Retry Pattern

For transient failures (disk not ready, network timeout), retry a few times:

```
fn open_with_retry(path: *u8, max_retries: u32) ->
   Result<FileHandle, DosError> {
    var attempts: u32 = 0

    while attempts < max_retries {
        match open(path, MODE_OLDFILE) {
            Result::Ok(file) => return Result::Ok(file),
            Result::Err(DosError::NoDisk) => {
                // Disk not ready - wait and retry
                delay(Duration::from_secs(1))
                attempts = attempts + 1u32
            },
            Result::Err(e) => return Result::Err(e)  //
               Non-retriable
        }
    }

    Result::Err(DosError::NoDisk)
}
```

10.10 Panic: When Errors Are Bugs

A **panic** is for situations where continuing is impossible—invariant violations, impossible states, or programmer errors.

```
// Panic on invariant violation
fn get_first(vec: &Vec<i32>) -> i32 {
    if vec.len() == 0 {
        panic!("get_first called on empty vector")
    }
    return vec[0].unwrap()
}
```

10.10.1 Panic vs Error

Use	When	Example
`Result::Err`	Failure is expected	File not found
`panic!()`	Failure is a bug	Index out of bounds

10.10.2 assert!() for Debug Checks

Use `assert!()` to catch programming errors during development:

```
fn process_buffer(buf: *u8, len: u32) {
    assert!(buf != null, "buffer must not be null")
    assert!(len > 0, "length must be positive")
    // ...
}
```

Assertions are checked in debug builds. They verify invariants that should always hold if the code is correct.

10.11 Creating Custom Errors

Define domain-specific error types for your application:

```
pub enum ConfigError {
    /// Config file not found in any search location
    FileNotFound,

    /// File exists but cannot be read
    ReadFailed(DosError),

    /// File is too large to load
    FileTooLarge(u32),

    /// Syntax error at specific line
    ParseError(u32),

    /// Required key is missing
    MissingKey,

    /// Value is invalid for expected type
    InvalidValue,
```

```
    /// Memory allocation failed
    OutOfMemory,
}
```

10.11.1 Implementing message()

Provide human-readable messages:

```
impl ConfigError {
    pub fn message(&self) -> *u8 {
        match *self {
            ConfigError::FileNotFound =>
                return "Config file not found in any search
                    location",
            ConfigError::ReadFailed(_) =>
                return "Failed to read config file",
            ConfigError::FileTooLarge(_) =>
                return "Config file exceeds maximum size",
            ConfigError::ParseError(_) =>
                return "Syntax error in config file",
            ConfigError::MissingKey =>
                return "Required configuration key is missing",
            ConfigError::InvalidValue =>
                return "Configuration value is invalid",
            ConfigError::OutOfMemory =>
                return "Out of memory loading config",
        }
    }
}
```

10.11.2 Extracting Error Details

For errors with associated data, provide accessor methods:

```
impl ConfigError {
    /// Get the line number for parse errors
    pub fn line_number(&self) -> Option<u32> {
        match *self {
            ConfigError::ParseError(line) => Option::Some(line),
            _ => Option::None
        }
    }

    /// Get the file size for too-large errors
    pub fn file_size(&self) -> Option<u32> {
        match *self {
            ConfigError::FileTooLarge(size) => Option::Some(size),
            _ => Option::None
        }
    }
}
```

10.12 Showcase: Config Loader

This complete program demonstrates comprehensive error handling in Novus. It loads configuration files with fallback locations, custom error types, and proper error reporting.

```
// config_loader.novus - Robust Config Loader

from std::core import Option, Result
from std::error::errors import DosError
from std::ffi::dos import Open, Close, Read, Lock, UnLock, Examine
from std::ffi::amiga_consts import MODE_OLDFILE, ACCESS_READ
from std::io::file import print, println

//
    ------------------------------------------------------------------------
// Custom Error Type with Rich Information
//
    ------------------------------------------------------------------------

pub enum ConfigError {
    FileNotFound,
    ReadFailed(DosError),
    FileTooLarge(u32),
    ParseError(u32),  // Line number
    MissingKey,
    InvalidValue,
    OutOfMemory,
}

impl ConfigError {
    pub fn message(&self) -> *u8 {
        match *self {
            ConfigError::FileNotFound =>
                return "Config file not found in any search
                    location",
            ConfigError::ReadFailed(_) =>
                return "Failed to read config file",
            ConfigError::FileTooLarge(_) =>
                return "Config file exceeds maximum size (64KB)",
            ConfigError::ParseError(_) =>
                return "Syntax error in config file",
            ConfigError::MissingKey =>
                return "Required configuration key is missing",
            ConfigError::InvalidValue =>
                return "Configuration value is invalid",
            ConfigError::OutOfMemory =>
                return "Out of memory loading config",
        }
    }

    pub fn line_number(&self) -> Option<u32> {
        match *self {
            ConfigError::ParseError(line) => Option::Some(line),
            _ => Option::None
        }
    }
}
```

```
//
    ------------------------------------------------------------------------
// Fallback Chain: Try Multiple Locations
//
    ------------------------------------------------------------------------

pub fn load_config(name: *u8) -> Result<Config, ConfigError> {
    var path_buffer = [0u8; 256]

    // Try ENV: first (volatile)
    copy_path(&path_buffer[0], "ENV:", name)
    match try_load_from(&path_buffer[0]) {
        Result::Ok(cfg) => return Result::Ok(cfg),
        Result::Err(_) => {}
    }

    // Try ENVARC: (persistent)
    copy_path(&path_buffer[0], "ENVARC:", name)
    match try_load_from(&path_buffer[0]) {
        Result::Ok(cfg) => return Result::Ok(cfg),
        Result::Err(_) => {}
    }

    // Try S: (startup scripts)
    copy_path(&path_buffer[0], "S:", name)
    match try_load_from(&path_buffer[0]) {
        Result::Ok(cfg) => return Result::Ok(cfg),
        Result::Err(_) => {}
    }

    // Try current directory
    match try_load_from(name) {
        Result::Ok(cfg) => return Result::Ok(cfg),
        Result::Err(e) => return Result::Err(e)
    }
}

//
    ------------------------------------------------------------------------
// Main: Demonstrating Error Handling
//
    ------------------------------------------------------------------------

pub fn main() -> i32 {
    println("Config Loader - Error Handling Showcase")
    println("")

    match load_config("test.config") {
        Result::Ok(config) => {
            println("Config loaded successfully!")
            print("  Source: ")
            println(&config.source_path[0])
            return 0
        },
        Result::Err(err) => {
            print("Error: ")
            println(err.message())
            return 1
```

```
            }
        }
    }
```

This program demonstrates:

- **Custom error types** with variant-specific data (line numbers, file sizes)
- **Fallback chains** trying multiple locations (ENV:, ENVARC:, S:, current dir)
- **Error messages** via the `message()` method
- **Error context** extraction via `line_number()`

The full source code is available in `Novus.Tests/Examples/config_loader_showcase.novus`.

10.13 Summary

- **Use `Result<T, E>`** for all fallible operations
- **Use `Option<T>`** when "not found" is the only failure mode
- **Pattern match** for explicit handling of all cases
- **Use `?`** to propagate errors concisely
- **Use `unwrap_or`** for safe defaults
- **Avoid `unwrap()`** except when failure is impossible
- **Use standard errors** (DosError, ExecError, etc.) for AmigaOS operations
- **Use NovusError** when crossing subsystem boundaries
- **Use `defer`** to ensure cleanup on error paths
- **Use panic** only for invariant violations and bugs
- **Create custom errors** for domain-specific failure modes

Error handling in Novus is explicit but ergonomic. The `?` operator and `Result` combinators make it easy to write code that correctly handles failures without obscuring the happy path.

Chapter 11

Advanced Traits and Generics

> *"Abstraction is not about vagueness, it is about being precise on a new semantic level."* — Edsger W. Dijkstra

Chapter 4 introduced traits as a way to define shared behavior across types. This chapter goes deeper, covering operator overloading, the complete trait ecosystem, advanced generic patterns, and real-world idioms that make Novus code both flexible and efficient.

By the end of this chapter, you'll understand how to:

- Make your types work with Novus operators (`+`, `-`, `*`, `[]`, etc.)
- Implement comparison and ordering for custom types
- Create fallible and infallible type conversions
- Build custom iterators for any data structure
- Write generic numeric algorithms
- Design flexible APIs with reference traits
- Apply common patterns like newtype, extension traits, and blanket implementations

11.1 Operator Overloading

Novus allows you to define how operators work with your custom types by implementing operator traits. When you implement the `Add` trait for a type, the `+` operator works with values of that type.

11.1.1 Arithmetic Operator Traits

The arithmetic operator traits are defined in `std::core`:

```
pub trait Add {
    fn add(&self, other: &Self) -> Self
}

pub trait Sub {
    fn sub(&self, other: &Self) -> Self
}

pub trait Mul {
    fn mul(&self, other: &Self) -> Self
}

pub trait Div {
    fn div(&self, other: &Self) -> Self
```

```
}

pub trait Rem {
    fn rem(&self, other: &Self) -> Self
}

pub trait Neg {
    fn neg(&self) -> Self
}
```

Each trait method takes references and returns an owned value. This design avoids unnecessary copies while keeping the interface simple.

Complete Operator-to-Trait Mapping

Operator	Trait	Method Signature
`a + b`	`Add`	`fn add(&self, other: &Self) -> Self`
`a - b`	`Sub`	`fn sub(&self, other: &Self) -> Self`
`a * b`	`Mul`	`fn mul(&self, other: &Self) -> Self`
`a / b`	`Div`	`fn div(&self, other: &Self) -> Self`
`a % b`	`Rem`	`fn rem(&self, other: &Self) -> Self`
`-a`	`Neg`	`fn neg(&self) -> Self`

Table 11.1: Arithmetic Operator Traits

11.1.2 Implementing Arithmetic Operators

Let's implement arithmetic for a 2D vector type:

```
from std::core import Add, Sub, Neg

struct Vec2 {
    x: i32,
    y: i32,
}

impl Vec2 {
    pub fn new(x: i32, y: i32) -> Vec2 {
        return Vec2 { x: x, y: y }
    }
}

impl Add for Vec2 {
    fn add(&self, other: &Vec2) -> Vec2 {
        return Vec2 {
            x: self.x + other.x,
            y: self.y + other.y
        }
    }
}

impl Sub for Vec2 {
    fn sub(&self, other: &Vec2) -> Vec2 {
        return Vec2 {
            x: self.x - other.x,
```

```
            y: self.y - other.y
        }
    }
}

impl Neg for Vec2 {
    fn neg(&self) -> Vec2 {
        return Vec2 { x: -self.x, y: -self.y }
    }
}
```

Now you can use operators naturally:

```
// ... (with Vec2 and operator impls defined above)
let a = Vec2::new(10, 20)
let b = Vec2::new(5, 15)

let sum = a + b        // Vec2 { x: 15, y: 35 }
let diff = a - b       // Vec2 { x: 5, y: 5 }
let negated = -a       // Vec2 { x: -10, y: -20 }
```

11.1.3 Fixed-Point Arithmetic Example

Here's how `fixed32` (16.16 fixed-point) implements multiplication:

```
// ... (illustrative - actual impl is in compiler)
impl Mul for fixed32 {
    fn mul(&self, other: &fixed32) -> fixed32 {
        // Fixed-point multiplication requires shifting
        // a * b = (a_raw * b_raw) >> 16
        // This is handled by the compiler intrinsic
        return fixed32_mul(*self, *other)
    }
}
```

Performance note: On 68000, 32-bit multiplication is done in software. The 68020+ have hardware `MULU.L` and `MULS.L` instructions. The compiler generates appropriate code based on your `-cpu` target.

11.1.4 Common Mistakes

Mistake 1: Forgetting that methods take references

```
// WRONG - takes self by value
impl Add for Vec2 {
    fn add(self, other: Vec2) -> Vec2 {  // Wrong signature!
        // ...
    }
}

// CORRECT - takes references
impl Add for Vec2 {
    fn add(&self, other: &Vec2) -> Vec2 {
        // ...
    }
}
```

Mistake 2: Division by zero

Operator traits don't have built-in error handling. If your type can encounter division by zero, you have options:

```
// Option 1: Panic (simple but crashes)
impl Div for Fraction {
    fn div(&self, other: &Fraction) -> Fraction {
        if other.numerator == 0 {
            panic!("Division by zero")
        }
        // ... normal division
    }
}

// Option 2: Saturate or return a sentinel value
impl Div for Fixed32 {
    fn div(&self, other: &Fixed32) -> Fixed32 {
        if other.is_zero() {
            return Fixed32::max_value()  // Saturate to max
        }
        // ... normal division
    }
}
```

For fallible operations, consider providing a separate `try_div` method that returns `Result`.

11.1.5 Index Operators

The `Index` and `IndexMut` traits enable the `[]` operator for your types.

```
pub trait Index<I, T> {
    fn index(&self, idx: I) -> T
}

pub trait IndexMut<I, T> {
    fn index_set(&var self, idx: I, value: T)
}
```

Note the asymmetry: `Index` returns a value, while `IndexMut` takes a value to set. This differs from some other languages where both return references.

Implementing Index for a Grid

```
from std::core import Index, IndexMut

struct Grid {
    data: Vec<i32>,
    width: u32,
    height: u32,
}

impl Grid {
    pub fn new(width: u32, height: u32) -> Option<Grid> {
        let size = width * height
        match Vec::with_capacity(size) {
            Option::Some(var v) => {
```

```
                // Initialize with zeros
                for i in 0..size {
                    v.push(0)
                }
                return Option::Some(Grid {
                    data: v,
                    width: width,
                    height: height
                })
            },
            Option::None => return Option::None
        }
    }

    fn calc_index(&self, x: u32, y: u32) -> u32 {
        return y * self.width + x
    }
}

// Read access: let value = grid[(x, y)]
impl Index<(u32, u32), i32> for Grid {
    fn index(&self, pos: (u32, u32)) -> i32 {
        let (x, y) = pos
        let idx = self.calc_index(x, y)
        return self.data.get(idx).unwrap()
    }
}

// Write access: grid[(x, y)] = value
impl IndexMut<(u32, u32), i32> for Grid {
    fn index_set(&var self, pos: (u32, u32), value: i32) {
        let (x, y) = pos
        let idx = self.calc_index(x, y)
        self.data.set(idx, value)
    }
}
```

Usage:

```
// ... (with Grid defined above)
var grid = Grid::new(10, 10).unwrap()

grid[(5, 3)] = 42           // Write using IndexMut
let value = grid[(5, 3)]    // Read using Index
```

Bounds Checking

In debug builds, you should add bounds checking to catch programming errors:

```
// ... (with Grid defined above)
impl Index<(u32, u32), i32> for Grid {
    fn index(&self, pos: (u32, u32)) -> i32 {
        let (x, y) = pos
        if x >= self.width || y >= self.height {
            panic!("Grid index out of bounds")
        }
        let idx = self.calc_index(x, y)
```

```
        return self.data.get(idx).unwrap()
    }
}
```

For production code where bounds are guaranteed, you can use unchecked access for performance.

11.2 Comparison and Equality Traits

Beyond the basic `Eq` trait (covered in Chapter 4), Novus provides `PartialOrd` and `Ord` for ordering comparisons.

11.2.1 The Equality Contract

The `Eq` trait requires these mathematical properties:

- **Reflexive**: `a.eq(&a)` must return `true`
- **Symmetric**: `a.eq(&b)` must equal `b.eq(&a)`
- **Transitive**: if `a.eq(&b)` and `b.eq(&c)`, then `a.eq(&c)`

Violating these properties will cause collections like `HashMap` to behave incorrectly.

11.2.2 PartialOrd — Comparison Operators

`PartialOrd` enables the `<`, `<=`, `>`, and `>=` operators:

```
pub trait PartialOrd {
    fn lt(&self, other: &Self) -> bool   // <
    fn le(&self, other: &Self) -> bool   // <=
    fn gt(&self, other: &Self) -> bool   // >
    fn ge(&self, other: &Self) -> bool   // >=
}
```

Implementing PartialOrd

```
from std::core import Eq, PartialOrd

struct Version {
    major: u32,
    minor: u32,
    patch: u32,
}

impl Eq for Version {
    fn eq(&self, other: &Version) -> bool {
        if self.major != other.major { return false }
        if self.minor != other.minor { return false }
        return self.patch == other.patch
    }
}

impl PartialOrd for Version {
```

```
    fn lt(&self, other: &Version) -> bool {
        if self.major != other.major {
            return self.major < other.major
        }
        if self.minor != other.minor {
            return self.minor < other.minor
        }
        return self.patch < other.patch
    }

    fn le(&self, other: &Version) -> bool {
        return self.lt(other) || self.eq(other)
    }

    fn gt(&self, other: &Version) -> bool {
        return other.lt(self)
    }

    fn ge(&self, other: &Version) -> bool {
        return other.le(self)
    }
}
```

Usage:

```
// ... (with Version defined above)
let v1 = Version { major: 1, minor: 2, patch: 0 }
let v2 = Version { major: 1, minor: 3, patch: 0 }

if v1 < v2 {
    println("v1 is older than v2")
}

if v1 >= v2 {
    println("v1 is at least as new as v2")
}
```

11.2.3 Ord — Total Ordering

`Ord` is for types where every value is comparable. It's required for sorting and binary search.

```
pub trait Ord {
    /// Returns: negative if self < other, zero if equal,
       positive if self > other
    fn cmp(&self, other: &Self) -> i32
}
```

The return value convention:

- Negative (`< 0`): `self` is less than `other`
- Zero (`== 0`): `self` equals `other`
- Positive (`> 0`): `self` is greater than `other`

```
from std::core import Ord
```

```
// ... (with Version and Eq defined above)
impl Ord for Version {
    fn cmp(&self, other: &Version) -> i32 {
        if self.major != other.major {
            return (i32)self.major - (i32)other.major
        }
        if self.minor != other.minor {
            return (i32)self.minor - (i32)other.minor
        }
        return (i32)self.patch - (i32)other.patch
    }
}
```

With `Ord` implemented, your type works with sorting:

```
// ... (with Version and Ord defined above)
var versions: Vec<Version> = load_versions()
versions.sort()  // Sorts using Ord::cmp
```

11.2.4 When to Use PartialOrd vs Ord

- Use `PartialOrd` when comparison might not always be possible (e.g., floating-point with NaN)
- Use `Ord` when all values are comparable (integers, strings, versions)

Floating-point types implement `PartialOrd` but not `Ord` because NaN is not comparable to anything, including itself.

11.2.5 Deriving Comparison Traits

For simple structs, use `@derive`:

```
#[derive(Eq)]
struct Point {
    x: i32,
    y: i32,
}
```

The derived `Eq` compares all fields in declaration order. Note that `@derive` currently supports `Eq` and `Hash`, but not `Ord`.

11.2.6 Common Mistakes

Mistake 1: Inconsistent Eq and Hash

If two values are equal via `Eq`, they *must* produce the same hash:

```
// WRONG - case-insensitive eq but case-sensitive hash
impl Eq for CaseInsensitiveString {
    fn eq(&self, other: &Self) -> bool {
        return self.to_lowercase() == other.to_lowercase()
    }
}

impl Hash for CaseInsensitiveString {
```

```
    fn hash(&self) -> u32 {
        return self.data.hash()  // BUG: "Hello" and "hello" hash
            differently!
    }
}

// CORRECT - both use lowercase
impl Hash for CaseInsensitiveString {
    fn hash(&self) -> u32 {
        return self.to_lowercase().hash()
    }
}
```

Mistake 2: Ord that disagrees with Eq

If `a.cmp(&b) == 0`, then `a.eq(&b)` should return `true`. Violating this causes subtle bugs in sorted collections.

11.3 Conversion Traits

Novus provides a family of traits for type conversion, covered in Chapter 4's `From` section. Here we cover the complete picture.

11.3.1 The Conversion Trait Family

Trait	Method	Use Case
`From<T>`	`convert(T) -> Self`	Infallible conversion from T
`Into<T>`	`into(self) -> T`	Infallible conversion to T
`TryFrom<T, E>`	`try_from(T) -> Result<Self, E>`	Fallible conversion from T
`TryInto<T, E>`	`try_into(self) -> Result<T, E>`	Fallible conversion to T

Table 11.2: Conversion Traits

11.3.2 From and Into

`From<T>` was covered in Chapter 4. The key insight is that `Into` is the reverse direction:

```
pub trait From<T> {
    fn convert(value: T) -> Self
}

pub trait Into<T> {
    fn into(consuming self) -> T
}
```

If you implement `From<A>` for B, Novus can automatically provide `Into<B>` for A. Always prefer implementing `From` over `Into`.

11.3.3 TryFrom and TryInto

When conversion can fail, use the fallible variants:

```
pub trait TryFrom<T, E> {
    fn try_from(value: T) -> Result<Self, E>
}
```

```
pub trait TryInto<T, E> {
    fn try_into(consuming self) -> Result<T, E>
}
```

Example: Parsing with TryFrom

```
from std::core import TryFrom, Result

pub enum ParseError {
    InvalidFormat,
    OutOfRange,
    Empty,
}

impl TryFrom<*u8, ParseError> for u8 {
    fn try_from(s: *u8) -> Result<u8, ParseError> {
        if s == null {
            return Result::Err(ParseError::Empty)
        }

        // Simple ASCII digit parsing
        var value: u32 = 0
        var ptr = s
        while *ptr != 0 {
            let c = *ptr
            if c < '0' || c > '9' {
                return Result::Err(ParseError::InvalidFormat)
            }
            value = value * 10 + (u32)(c - '0')
            if value > 255 {
                return Result::Err(ParseError::OutOfRange)
            }
            ptr = ptr + 1
        }
        return Result::Ok((u8)value)
    }
}
```

Usage:

```
// ... (with u8 TryFrom impl defined above)
match u8::try_from("255") {
    Result::Ok(n) => println(f"Parsed: {n}"),
    Result::Err(ParseError::OutOfRange) => println("Number too
        large"),
    Result::Err(ParseError::InvalidFormat) => println("Not a
        number"),
    Result::Err(ParseError::Empty) => println("Empty string"),
}
```

11.3.4 Designing Error Types for Conversions

Good conversion error types are:

- **Specific**: Different failure modes get different variants
- **Informative**: Include context when useful
- **Lightweight**: Don't allocate in the error path if possible

```
// Good - specific variants, no allocation
pub enum NumericParseError {
    Empty,
    InvalidCharacter,
    Overflow,
    Underflow,
}

// Less good - loses information
pub enum GenericError {
    Failed,
}
```

11.3.5 Choosing the Right Conversion Trait

- Use `From`/`Into` when conversion always succeeds
- Use `TryFrom`/`TryInto` when conversion can fail
- Implement `From`, not `Into` (you get `Into` for free)
- Never panic in `From` — if it can fail, use `TryFrom`

11.4 The Iterator System

Iterators provide a uniform way to traverse sequences. Implementing `Iterator` lets your type work with loops and iterator-consuming functions.

11.4.1 The Iterator Trait

```
pub trait Iterator<T> {
    fn next(&var self) -> Option<T>
}
```

The contract:

- `next()` returns `Some(value)` for each element
- When exhausted, returns `None`
- After returning `None`, should continue returning `None`

11.4.2 Creating a Custom Iterator

Let's implement an iterator for a simple range:

```
from std::core import Iterator, Option

struct RangeIter {
    current: i32,
    end: i32,
}

impl RangeIter {
    pub fn new(start: i32, end: i32) -> RangeIter {
        return RangeIter { current: start, end: end }
    }
}

impl Iterator<i32> for RangeIter {
    fn next(&var self) -> Option<i32> {
        if self.current >= self.end {
            return Option::None
        }
        let value = self.current
        self.current = self.current + 1
        return Option::Some(value)
    }
}
```

Usage:

```
// ... (with RangeIter defined above)
var iter = RangeIter::new(0, 5)
loop {
    match iter.next() {
        Option::Some(n) => println(f"{n}"),
        Option::None => break,
    }
}
// Output: 0, 1, 2, 3, 4
```

11.4.3 Iterating Collections

The `Iterable` trait connects collections to for-loops:

```
pub trait Iterable<T> {
    fn get(&self, index: u32) -> Option<T>
    fn len(&self) -> u32
}
```

When you write `for item in collection`, the compiler uses `len()` to determine bounds and `get()` to access each element by index.

Vec Iterator Example

```
// ... (illustrative - actual impl in stdlib)
struct VecIter<T> {
    vec: *Vec<T>,
    index: u32,
}
```

```
impl<T> Iterator<T> for VecIter<T> {
    fn next(&var self) -> Option<T> {
        if self.index >= (*self.vec).len() {
            return Option::None
        }
        let value = (*self.vec).get(self.index)
        self.index = self.index + 1
        return value
    }
}
```

11.4.4 Iterator for a Linked List

Here's a more complex example — iterating a singly-linked list:

```
from std::core import Iterator, Option

struct Node<T> {
    value: T,
    next: *Node<T>,
}

struct LinkedList<T> {
    head: *Node<T>,
    len: u32,
}

struct LinkedListIter<T> {
    current: *Node<T>,
}

impl<T> LinkedList<T> {
    pub fn iter(&self) -> LinkedListIter<T> {
        return LinkedListIter { current: self.head }
    }
}

impl<T> Iterator<T> for LinkedListIter<T> {
    fn next(&var self) -> Option<T> {
        if self.current == null {
            return Option::None
        }
        let value = (*self.current).value
        self.current = (*self.current).next
        return Option::Some(value)
    }
}
```

11.4.5 Performance Considerations

Iterators in Novus are designed for zero overhead:

- Iterator structs are typically small (a pointer and an index)
- `next()` calls are inlined by the optimizer

- No heap allocation for iteration itself
- Performance matches hand-written loops

When iterators might be slower:

- Very tight loops where function call overhead matters
- When the optimizer can't inline (rare)
- Complex iterator chains (consider breaking into loops)

11.4.6 Common Mistakes

Mistake 1: Modifying the collection while iterating

```
// ... (illustrative - shows incorrect pattern)
// WRONG - undefined behavior
var vec = Vec::new()
vec.push(1)
vec.push(2)
vec.push(3)

var iter = vec.iter()
loop {
    match iter.next() {
        Option::Some(n) => {
            if n == 2 {
                vec.push(4)  // BUG: modifying while iterating!
            }
        },
        Option::None => break,
    }
}
```

Mistake 2: Using an exhausted iterator

```
// ... (illustrative - shows incorrect pattern)
var iter = vec.iter()
// Consume all elements
loop {
    match iter.next() {
        Option::Some(_) => {},
        Option::None => break,
    }
}

// This won't iterate anything - iterator is exhausted
loop {
    match iter.next() {
        Option::Some(n) => println(f"{n}"),  // Never reached
        Option::None => break,
    }
}
```

Create a new iterator if you need to iterate again.

11.5 Numeric Traits

These traits enable generic programming over numeric types — essential for math libraries and algorithms.

11.5.1 Zero and One

```
pub trait Zero {
    fn zero() -> Self
    fn is_zero(&self) -> bool
}

pub trait One {
    fn one() -> Self
}
```

These represent the additive and multiplicative identity elements.

Implementations for Primitive Types

All numeric primitives implement these traits:

```
// ... (implemented in std::core)
impl Zero for i32 {
    fn zero() -> i32 { return 0i32 }
    fn is_zero(&self) -> bool { return *self == 0i32 }
}

impl One for i32 {
    fn one() -> i32 { return 1i32 }
}
```

11.5.2 Writing Generic Numeric Functions

```
from std::core import Zero, One, Add, Mul

fn sum<T>(items: *T, count: u32) -> T where T: Zero + Add {
    var total = T::zero()
    for i in 0..count {
        total = total + *(items + i)
    }
    return total
}

fn product<T>(items: *T, count: u32) -> T where T: One + Mul {
    var result = T::one()
    for i in 0..count {
        result = result * *(items + i)
    }
    return result
}
```

Usage:

```
// ... (with sum and product defined above)
let numbers = [1, 2, 3, 4, 5]
let total = sum(&numbers[0], 5)       // 15
let prod = product(&numbers[0], 5)    // 120
```

11.5.3 Bounded — Min and Max Values

```
pub trait Bounded {
    fn min_value() -> Self
    fn max_value() -> Self
}
```

Bounds for Novus Numeric Types

Type	min_value()	max_value()
i8	−128	127
i16	−32,768	32,767
i32	−2,147,483,648	2,147,483,647
u8	0	255
u16	0	65,535
u32	0	4,294,967,295

Table 11.3: Numeric Type Bounds

11.5.4 Practical Use: Saturating Arithmetic

Saturating arithmetic clamps at bounds instead of overflowing:

```
// ... (illustrative - generic numeric functions)
from std::core import Add, Bounded, PartialOrd

fn saturating_add<T>(a: T, b: T) -> T where T: Add + Bounded +
   PartialOrd {
    let result = a + b
    // Check for overflow: if result wrapped around
    if result.lt(&a) {
        return T::max_value()
    }
    return result
}

fn clamp<T>(value: T, min: T, max: T) -> T where T: PartialOrd {
    if value.lt(&min) { return min }
    if value.gt(&max) { return max }
    return value
}
```

11.5.5 Fixed-Point and Bounded

Fixed-point types also implement `Bounded`:

```
// ... (illustrative - actual values depend on representation)
impl Bounded for fixed32 {
    fn min_value() -> fixed32 {
        // -32768.0 in 16.16 format
        return fixed32_from_raw(-2147483648i32)
    }
    fn max_value() -> fixed32 {
        // 32767.99998... in 16.16 format
        return fixed32_from_raw(2147483647i32)
    }
}
```

11.6 Reference Traits

`AsRef` and `AsMut` enable functions to accept multiple types that can be "viewed as" a common type.

11.6.1 AsRef — Cheap Reference Conversion

```
pub trait AsRef<T> {
    fn as_ref(&self) -> T
}
```

Note: In Novus, `AsRef` returns `T` directly (typically a slice or reference type), not `&T`.

11.6.2 Designing Flexible APIs

Without `AsRef`:

```
// Inflexible - only accepts Str
fn count_lines(s: Str) -> u32 {
    var count: u32 = 0
    for c in s.bytes() {
        if c == '\n' {
            count = count + 1
        }
    }
    return count
}
```

With `AsRef`:

```
// Flexible - accepts String, Str, or anything AsRef<Str>
fn count_lines<S>(s: &S) -> u32 where S: AsRef<Str> {
    let str_view = s.as_ref()
    var count: u32 = 0
    // ... count newlines
    return count
}
```

11.6.3 AsMut — Mutable View Conversion

```
pub trait AsMut<T> {
    fn as_mut(&var self) -> T
}
```

Use when your function needs to modify the underlying data.

11.6.4 Standard Library Usage

Many stdlib functions use `AsRef` for flexibility:

```
// ... (illustrative - file opening API)
pub fn open<P>(path: &P, mode: u32) -> Result<FileHandle,
   DosError>
    where P: AsRef<Str>
{
    let path_str = path.as_ref()
    // ... open the file using path_str
}
```

Callers benefit:

```
// Works with string literal (Str)
let f1 = open(&"config.txt", MODE_OLDFILE)

// Works with owned String
var path = String::new()
path.push_str("data/file.dat")
let f2 = open(&path, MODE_OLDFILE)
```

11.6.5 When Not to Use AsRef

- When you need ownership (use `Into` instead)
- When the conversion is expensive (AsRef should be cheap)
- When it makes the API confusing (simpler is sometimes better)

11.7 Advanced Generics

This section covers more sophisticated generic patterns.

11.7.1 Multiple Trait Bounds

Use `+` to require multiple traits:

```
fn debug_and_hash<T>(value: &T) -> u32 where T: Debug + Hash {
    println(value.debug_string())
    return value.hash()
}
```

11.7.2 Multiple Type Parameters

```
// ... (illustrative - multiple generic parameters)
fn convert_and_store<T, U>(input: T, storage: &var Vec<U>) where
    T: Into<U> {
     let converted: U = input.into()
     storage.push(converted)
}
```

11.7.3 Where Clauses on Structs

Constrain which types can instantiate a generic struct:

```
// ... (illustrative - where clause on struct)
struct SortedVec<T> where T: Ord {
    items: Vec<T>,
}

impl<T> SortedVec<T> where T: Ord {
    pub fn new() -> Option<SortedVec<T>> {
        match Vec::new() {
            Option::Some(v) => {
                return Option::Some(SortedVec { items: v })
            },
            Option::None => return Option::None
        }
    }

    pub fn insert(&var self, value: T) {
        // Find insertion point to maintain sorted order
        var idx: u32 = 0
        for i in 0..self.items.len() {
            match self.items.get(i) {
                Option::Some(item) => {
                    if item.cmp(&value) > 0 {
                        break
                    }
                },
                Option::None => break,
            }
            idx = i + 1
        }
        self.items.insert(idx, value)
    }
}
```

11.7.4 Generic Implementations

Implement traits generically for families of types:

```
// ... (illustrative - Display for all Option<T> where T: Display)
impl<T> Display for Option<T> where T: Display {
    fn fmt(&self, f: &var Formatter) -> bool {
        match *self {
            Option::Some(ref v) => {
                f.write_str("Some(")
                v.fmt(f)
```

```
                f.write_str(")")
            },
            Option::None => {
                f.write_str("None")
            },
        }
        return true
    }
}
```

11.7.5 Bounds on Individual Methods

Add bounds to specific methods rather than the entire impl:

```
// ... (illustrative - method-level where clause)
impl<T> Vec<T> {
    // Available for all Vec<T>
    pub fn len(&self) -> u32 {
        return self.length
    }

    // Only available when T: Eq
    pub fn contains(&self, value: &T) -> bool where T: Eq {
        for i in 0..self.len() {
            match self.get(i) {
                Option::Some(item) => {
                    if item.eq(value) {
                        return true
                    }
                },
                Option::None => {},
            }
        }
        return false
    }
}
```

11.7.6 Const Generic Parameters

For compile-time size parameters:

```
struct FixedBuffer<const N: u32> {
    data: [u8; N],
    len: u32,
}

impl<const N: u32> FixedBuffer<N> {
    pub fn new() -> FixedBuffer<N> {
        return FixedBuffer {
            data: [0u8; N],
            len: 0,
        }
    }

    pub fn capacity(&self) -> u32 {
        return N
```

```
    }
}
```

Usage:

```
// ... (with FixedBuffer defined above)
let small = FixedBuffer::<64>::new()   // 64-byte buffer
let large = FixedBuffer::<1024>::new() // 1KB buffer
```

11.7.7 Monomorphization

Generics in Novus are monomorphized — the compiler generates specialized code for each concrete type used:

```
fn max<T>(a: T, b: T) -> T where T: Ord {
    if a.cmp(&b) > 0 { return a }
    return b
}

// When you call:
let m1 = max(10i32, 20i32)
let m2 = max(5u8, 3u8)

// Compiler generates:
// fn max_i32(a: i32, b: i32) -> i32 { ... }
// fn max_u8(a: u8, b: u8) -> u8 { ... }
```

Benefits: Zero runtime overhead, full optimization.

Cost: Code size increases with each instantiation. On memory-constrained 68k systems, be mindful of instantiating generics with many different types.

11.8 Common Patterns and Idioms

These patterns appear frequently in well-designed Novus code.

11.8.1 The Newtype Pattern

Wrap a type to give it distinct identity:

```
struct Pixels {
    value: i32,
}

struct Bytes {
    value: u32,
}

struct ChipAddr {
    ptr: *u8,
}

impl Pixels {
    pub fn new(value: i32) -> Pixels {
        return Pixels { value: value }
    }
```

```
    pub fn get(&self) -> i32 {
        return self.value
    }
}
```

Now the compiler prevents mixing them up:

```
// ... (with Pixels and Bytes defined above)
fn allocate_pixels(width: Pixels, height: Pixels) -> *u8 {
    // ...
}

fn allocate_bytes(size: Bytes) -> *u8 {
    // ...
}

let w = Pixels::new(320)
let h = Pixels::new(256)
let size = Bytes { value: 1024 }

allocate_pixels(w, h)    // OK
allocate_pixels(w, size) // Compile error! Bytes != Pixels
```

Newtype for Chip RAM Addresses

On the Amiga, chip RAM and fast RAM serve different purposes. Use newtypes to prevent bugs:

```
struct ChipPtr {
    ptr: *u8,  // Pointer to chip RAM
}

struct FastPtr {
    ptr: *u8,  // Pointer to fast RAM
}

fn setup_copper_list(gfx: ChipPtr) {
    // Copper can only access chip RAM
    // Type system ensures we can't accidentally pass fast RAM
}

fn cache_data(buf: FastPtr) {
    // Data caching works best in fast RAM
}
```

11.8.2 Extension Traits

Add methods to types you don't own:

```
trait I32Ext {
    fn is_even(&self) -> bool
    fn is_positive(&self) -> bool
    fn abs(&self) -> i32
}
```

```
impl I32Ext for i32 {
    fn is_even(&self) -> bool {
        return *self % 2 == 0
    }

    fn is_positive(&self) -> bool {
        return *self > 0
    }

    fn abs(&self) -> i32 {
        if *self < 0 { return -*self }
        return *self
    }
}
```

Usage:

```
// ... (with I32Ext defined above)
let x: i32 = -42
if x.is_even() {
    println("Even!")
}
let positive = x.abs()  // 42
```

11.8.3 Blanket Implementations

Implement a trait for all types matching a bound:

```
// ... (illustrative - every Display type gets to_string)
impl<T> ToString for T where T: Display {
    fn to_string(&self) -> Option<String> {
        match String::new() {
            Option::Some(var s) => {
                // Use Display to write to string
                self.fmt(&var s.as_formatter())
                return Option::Some(s)
            },
            Option::None => return Option::None,
        }
    }
}
```

Now any type implementing `Display` automatically gets `to_string()`.

11.8.4 Marker Traits

Traits with no methods, used to mark capabilities:

```
pub trait Copy {}
```

The `Copy` marker tells the compiler a type can be implicitly copied (bitwise) rather than moved. Primitives like `i32`, `u8`, and `bool` are `Copy`.

11.8.5 The Builder Pattern

For complex object construction:

```
struct WindowConfig {
    title: *u8,
    width: u32,
    height: u32,
    borderless: bool,
}

struct WindowBuilder {
    config: WindowConfig,
}

impl WindowBuilder {
    pub fn new() -> WindowBuilder {
        return WindowBuilder {
            config: WindowConfig {
                title: "Untitled",
                width: 320,
                height: 256,
                borderless: false,
            }
        }
    }

    @chain
    pub fn title(&var self, t: *u8) {
        self.config.title = t
    }

    @chain
    pub fn size(&var self, w: u32, h: u32) {
        self.config.width = w
        self.config.height = h
    }

    @chain
    pub fn borderless(&var self) {
        self.config.borderless = true
    }

    pub fn build(&self) -> Result<WindowHandle, IntuitionError> {
        // ... create the window
    }
}
```

Usage:

```
// ... (with WindowBuilder defined above)
var builder = WindowBuilder::new()
let window = builder
    .title("My App")
    .size(640, 480)
    .borderless()
    .build()?
```

11.8.6 The Handle Pattern

Wrap OS resources with RAII cleanup:

```
struct FileHandle {
    lock: *FileLock,
}

impl FileHandle {
    pub fn open(path: *u8, mode: i32) -> Result<FileHandle,
       DosError> {
        let lock = Lock(path, mode)
        if lock == null {
            return Result::Err(dos_last_error())
        }
        return Result::Ok(FileHandle { lock: lock })
    }
}

impl Drop for FileHandle {
    fn drop(&var self) {
        if self.lock != null {
            UnLock(self.lock)
            self.lock = null
        }
    }
}
```

The file is automatically closed when the `FileHandle` goes out of scope.

11.9 Summary

This chapter covered advanced trait and generic features:

- **Operator traits** — `Add`, `Sub`, `Mul`, `Div`, `Rem`, `Neg`, `Index`, `IndexMut`
- **Comparison traits** — `Eq`, `PartialOrd`, `Ord`
- **Conversion traits** — `From`, `Into`, `TryFrom`, `TryInto`
- **Iterator traits** — `Iterator`, `Iterable`
- **Numeric traits** — `Zero`, `One`, `Bounded`
- **Reference traits** — `AsRef`, `AsMut`
- **Advanced generics** — multiple bounds, where clauses, const generics
- **Common patterns** — newtype, extension traits, blanket implementations, builder, handle

Traits are the foundation of generic programming in Novus. Master them, and you'll write code that is both flexible and efficient — code worthy of the Amiga.

Appendix A

Guide for C Programmers

This appendix consolidates the key differences between C and Novus for experienced C programmers transitioning to the language. If you've written AmigaOS code in C, this chapter will help you map familiar concepts to their Novus equivalents.

A.1 Philosophy Differences

Novus and C share a systems programming heritage, but differ in philosophy:

Aspect	C	Novus
Safety	Trust the programmer	Verify at compile time
Defaults	Mutable, public	Immutable, private
Errors	Return codes, `errno`	`Result<T, E>` types
Memory	Manual `malloc`/`free`	RAII with `Drop` trait
Null	Implicit in all pointers	Explicit `*T` vs non-null `&T`

A.2 Variable Declarations

C declares type first, Novus declares intent first:

```
// C
int x = 10;                 // Mutable by default
const int Y = 20;           // Immutable
static int z = 30;          // File-local
```

```
// ... (Novus equivalent - illustrative)
var x = 10                  // Mutable (explicit)
let y = 20                  // Immutable (explicit)
const Z: i32 = 30           // Compile-time constant
```

Key differences:

- Type inference: `var x = 10` infers `i32`
- No semicolons required (optional)
- `let` bindings cannot be reassigned
- `const` must be computable at compile time

A.3 Type System

A.3.1 Primitive Types

C	Novus	C	Novus
`char`	`i8` or `u8`	`unsigned char`	`u8`
`short`	`i16`	`unsigned short`	`u16`
`int` / `long`	`i32`	`unsigned int`	`u32`
`long long`	`i64`	`unsigned long long`	`u64`
`float`	`f32`	`double`	`f64`
`void*`	`*u8`	`NULL`	`null`

Novus also provides fixed-point types not available in standard C:

- `fixed16` — 8.8 fixed-point (useful for Amiga graphics)
- `fixed32` — 16.16 fixed-point

A.3.2 Booleans

```
// C: integers are implicitly boolean
if (ptr) { ... }          // Non-zero = true
if (count) { ... }        // Non-zero = true
int flag = 1;             // "true"
```

```
// Novus: explicit bool type, no implicit conversion
if ptr != null { ... }    // Must compare explicitly
if count > 0 { ... }      // Must compare explicitly
let flag: bool = true     // Dedicated bool type
```

A.3.3 Structs

```
// C: typedef dance required
typedef struct {
    int x;
    int y;
} Point;

Point p = {10, 20};
Point *ptr = &p;
int x = ptr->x;           // Arrow for pointer access
```

```
struct Point {
    x: i32,
    y: i32,
}
```

Usage:

```
// ... (with Point defined above)
let p = Point { x: 10, y: 20 }
let ptr = &p
let x = ptr.x             // Dot works for everything
```

Key differences:

- No `typedef` needed—struct name is the type
- Field syntax is `name: type` (reversed from C)
- No `->` operator—dot (`.`) auto-dereferences
- Struct literals use `Type { field: value }` syntax

A.4 Pointers and References

C has one pointer type; Novus distinguishes three:

Novus	C Equivalent	Properties
`&T`	`const T*`	Non-null, immutable
`&var T`	`T*`	Non-null, mutable
`*T`	`T*`	Nullable, requires `unsafe`

```c
// C: All pointers can be null
void process(Player *p);            // Might modify, might be null
void print_info(const Player *p);   // Won't modify, might be null
```

```
// Novus: Intent is explicit
fn process(p: &var Player)     // Will modify, never null
fn print_info(p: &Player)      // Won't modify, never null
fn maybe_process(p: *Player)   // Might be null (use in unsafe)
```

The call site makes mutability visible:

```
process(&var player)    // Clearly shows modification
print_info(&player)     // Clearly shows read-only
```

A.5 Functions

A.5.1 Declaration Syntax

```c
// C: return type first
int add(int a, int b) {
    return a + b;
}

static int helper(void) { return 42; }  // File-local
```

```
// Novus: fn keyword, return type last
fn add(a: i32, b: i32) -> i32 {
    return a + b
}

fn helper() -> i32 { return 42 }  // Private by default
pub fn api() -> i32 { return 0 }  // Explicitly public
```

A.5.2 Methods

C doesn't have methods; Novus does:

```c
// C: Pass struct explicitly
int rect_area(const Rectangle *r) {
    return r->width * r->height;
}
void rect_scale(Rectangle *r, int factor) {
    r->width *= factor;
    r->height *= factor;
}

// Usage
int a = rect_area(&rect);
rect_scale(&rect, 2);
```

```
// ... (with Rectangle defined elsewhere - illustrative)
impl Rectangle {
    fn area(&self) -> i32 {
        return self.width * self.height
    }
    fn scale(&var self, factor: i32) {
        self.width = self.width * factor
        self.height = self.height * factor
    }
}
```

Usage:

```
// ... (with Rectangle defined above)
let a = rect.area()
rect.scale(2)
```

A.6 Control Flow

A.6.1 If Statements

```c
// C: Parentheses required, braces optional
if (x > 0)
    return 1;
else if (x < 0)
    return -1;
else
    return 0;
```

```
// Novus: No parentheses, braces required
if x > 0 {
    return 1
} else if x < 0 {
    return -1
} else {
    return 0
```

```
}
```

A.6.2 Switch vs Match

```
// C: switch with fall-through, break required
switch (cmd) {
    case CMD_READ:
        handle_read();
        break;            // Easy to forget!
    case CMD_WRITE:
    case CMD_UPDATE:      // Fall-through
        handle_write();
        break;
    default:
        handle_unknown();
}
```

```
// Novus: match with no fall-through
match cmd {
    CMD_READ => handle_read(),
    CMD_WRITE | CMD_UPDATE => handle_write(),  // Explicit OR
    _ => handle_unknown(),
}
```

Key differences:

- No fall-through—each arm is independent
- No `break` needed
- Use `|` for multiple patterns
- `_` is the wildcard (like `default`)
- `match` can be an expression returning a value

A.6.3 Loops

```
// C: Traditional for loop
for (int i = 0; i < 10; i++) {
    process(i);
}
```

```
// Novus: Range-based for (preferred)
for i in 0..10 {
    process(i)
}

// C-style also supported
for var i = 0; i < 10; i++ {
    process(i)
}
```

A.7 Memory Management

A.7.1 The goto cleanup Pattern

```
// C: Manual cleanup with goto
int do_work(void) {
    void *a = NULL, *b = NULL;
    int result = -1;

    a = AllocMem(100, MEMF_ANY);
    if (!a) goto cleanup;

    b = AllocMem(200, MEMF_ANY);
    if (!b) goto cleanup;

    // ... actual work ...
    result = 0;

cleanup:
    if (b) FreeMem(b, 200);  // Must remember size!
    if (a) FreeMem(a, 100);
    return result;
}
```

```
// Novus: RAII handles cleanup automatically
fn do_work() -> Result<(), ExecError> {
    var a = MemoryBlock::try_alloc(100, MEMF_ANY)?
    var b = MemoryBlock::try_alloc(200, MEMF_ANY)?

    // ... actual work ...

    return Result::Ok(())
}  // Both freed automatically, correct order, size tracked
```

A.7.2 RAII vs Manual Management

Aspect	C	Novus
Allocation	`AllocMem(size, flags)`	`MemoryBlock::alloc(size, flags)`
Deallocation	`FreeMem(ptr, size)`	Automatic via `Drop`
Size tracking	Manual (error-prone)	Automatic
Cleanup order	Manual (LIFO required)	Automatic (guaranteed LIFO)
Early return	Requires `goto`	Just `return` or `?`

A.8 Error Handling

```
// C: Return codes and errno
BPTR file = Open("data.txt", MODE_OLDFILE);
if (!file) {
    LONG err = IoErr();
    printf("Error: %ld\n", err);
    return -1;
```

```
}
// ... use file ...
Close(file);
```

```
// Novus: Result type with ? operator
fn read_data() -> Result<String, DosError> {
    let file = open("data.txt", MODE_OLDFILE)?  // Returns on
        error
    let data = read_all(&file)?
    return Result::Ok(data)
}  // file closed automatically
```

The `?` operator:

1. Checks if result is `Err`
2. If error, returns immediately with that error
3. If success, unwraps the value and continues

A.9 Headers and Modules

```
// C: Textual inclusion
#ifndef MYHEADER_H
#define MYHEADER_H

#include <exec/types.h>
#include <proto/exec.h>  // Order can matter!

extern int global_count;
void my_function(void);

#endif
```

```
// Novus: Semantic imports
from std::ffi::exec import AllocMem, FreeMem
from std::ffi::amiga_consts import MEMF_FAST

pub fn my_function() {
    // ...
}
```

Key differences:

- No include guards needed
- Import order doesn't matter
- Only imported names are in scope
- No separate header files
- `pub` explicitly exports (vs C's implicit visibility)

A.10 Visibility

C	Novus
`static` (file-local)	(default, no modifier)
Non-static + header	`pub`
`extern` declaration	`extern` (for FFI)

Note: Novus inverts C's default. In C, symbols are public unless marked `static`. In Novus, symbols are private unless marked `pub`.

A.11 Common Intrinsics

C	Novus
`sizeof(T)`	`@sizeof(T)`
`_Alignof(T)`	`alignof(T)` (in inline asm only)
`memset(&x, 0, sizeof(x))`	`@zeroed(T)`

A.12 AmigaOS NDK Access

The entire AmigaOS NDK is available from Novus. Every function, structure, and constant from the 3.9 NDK has been imported and can be called directly. You don't need wrappers—you can use the raw API just like you would in C.

A.12.1 FFI Modules

The NDK bindings are organized by library in `std::ffi`:

```
// ... (FFI import examples - illustrative)
from std::ffi::exec import AllocMem, FreeMem, FindTask, Wait,
    Signal
from std::ffi::dos import Open, Close, Read, Write, IoErr, Lock,
    UnLock
from std::ffi::graphics import BltBitMap, SetAPen, RectFill, Text
from std::ffi::intuition import OpenWindow, CloseWindow,
    RefreshGList
from std::ffi::gadtools import CreateMenus, FreeMenus, LayoutMenus
from std::ffi::layers import CreateUpfrontLayer, DeleteLayer
from std::ffi::utility import AllocateTagItems, FreeTagItems
```

Structures and constants:

```
// ... (structure and constant imports - illustrative)
from std::ffi::amiga_structs import Window, Screen, BitMap
from std::ffi::amiga_consts import MEMF_CHIP, MEMF_FAST,
    MEMF_CLEAR
```

A.12.2 Calling Raw NDK Functions

Raw AmigaOS calls require `unsafe` blocks because they bypass Novus's safety guarantees:

```
from std::ffi::exec import AllocMem, FreeMem
from std::ffi::amiga_consts import MEMF_CHIP, MEMF_CLEAR
```

```
fn allocate_chip_buffer(size: u32) -> *u8 {
    unsafe {
        return AllocMem(size, MEMF_CHIP | MEMF_CLEAR)
    }
}

fn free_chip_buffer(ptr: *u8, size: u32) {
    if ptr != null {
        unsafe {
            FreeMem(ptr, size)
        }
    }
}
```

This is identical to what you'd write in C, just with explicit `unsafe` markers.

A.12.3 When to Use Raw vs Wrappers

Use	Raw NDK	Stdlib Wrappers
When	Porting C code, one-off calls, no wrapper exists	New code, repeated patterns
Safety	Manual (like C)	Automatic (RAII, Result)
Cleanup	Manual `Free*` calls	Automatic via `Drop`
Errors	Check return values	`Result<T, E>`

Both approaches are valid. The wrappers exist for convenience and safety, but they're built on the same raw FFI you have access to. If a wrapper doesn't exist for what you need, use the raw API directly.

A.12.4 Example: Raw Intuition Window

Here's opening a window the "C way" in Novus:

```
from std::ffi::intuition import OpenWindowTags, CloseWindow
from std::ffi::amiga_consts import WA_Width, WA_Height, WA_Title,
    WA_IDCMP, WA_Flags, WFLG_CLOSEGADGET, WFLG_DRAGBAR,
    IDCMP_CLOSEWINDOW, TAG_DONE

fn main() -> i32 {
    var window: *Window = null

    unsafe {
        window = OpenWindowTags(null,
            WA_Width, 640,
            WA_Height, 480,
            WA_Title, "Raw NDK Window",
            WA_Flags, WFLG_CLOSEGADGET | WFLG_DRAGBAR,
            WA_IDCMP, IDCMP_CLOSEWINDOW,
            TAG_DONE)
    }

    if window == null {
        return 1
    }

    // ... use window ...
```

```
    unsafe {
        CloseWindow(window)
    }

    return 0
}
```

This is nearly identical to C code—the only differences are:

- `unsafe {}` blocks around library calls
- `null` instead of `NULL`
- Novus variable declaration syntax

A.12.5 BCPL Pointers (DOS Library)

DOS library functions use BCPL pointers (`BPTR`/`BSTR`), which are `i32` values, not actual pointers:

```
from std::ffi::dos import Open, Close, Read, Write
from std::ffi::amiga_consts import MODE_OLDFILE

fn read_file_raw(path: *u8) -> i32 {
    // DOS file handle is i32 (BPTR), not *File
    var file: i32 = 0

    unsafe {
        file = Open(path, MODE_OLDFILE)
    }

    if file == 0 {
        return -1
    }

    var buffer = [0u8; 1024]
    var bytes_read: i32 = 0

    unsafe {
        bytes_read = Read(file, &buffer as *u8, 1024)
        Close(file)
    }

    return bytes_read
}
```

A.12.6 Comparison: Raw vs Wrapper

For comparison, here's the same operation using the stdlib wrapper:

```
// C: Direct library calls
struct Window *win = OpenWindowTags(NULL,
    WA_Width, 640,
    WA_Height, 480,
    WA_Title, "My Window",
    TAG_DONE);
if (win) {
```

```
    // ... use window ...
    CloseWindow(win);
}
```

```
// Novus: RAII wrappers preferred
match WindowBuilder::new()
    .size(640, 480)
    .title("My Window")
    .build()
{
    Result::Ok(window) => {
        // ... use window ...
    },  // Closed automatically
    Result::Err(e) => {
        println(e.message())
    }
}
```

A.12.7 Hardware Access

Both languages support direct hardware access, but Novus requires `unsafe`:

```
// C: Direct hardware poke
volatile UWORD *custom = (UWORD *)0xdff000;
custom[0x96/2] = 0x8020;  // DMACON
```

```
// ... (Novus unsafe block - illustrative)
unsafe {
    let custom = 0xdff000 as *u16
    *(custom + 0x96 / 2) = 0x8020
}
```

A.13 Quick Reference Table

C	Novus
`int x = 10;`	`var x = 10`
`const int X = 10;`	`const X: i32 = 10`
`int arr[3] = {1,2,3};`	`let arr = [1, 2, 3]`
`if (ptr)`	`if ptr != null`
`switch/case/break`	`match { pat => }`
`for(i=0;i<n;i++)`	`for i in 0..n`
`ptr->field`	`ptr.field`
`struct S {...};`	`struct S {...}`
`typedef struct`	(not needed)
`malloc/free`	`MemoryBlock`, `Vec`
`sizeof(T)`	`@sizeof(T)`
`NULL`	`null`
`errno`	`Result::Err(...)`
`static`	(default)
`#include`	`from ... import`

A.14 Migration Tips

1. **Start with `var`** — You can always tighten to `let` later
2. **Use stdlib wrappers** — `MemoryBlock`, `Vec`, `WindowHandle` instead of raw FFI
3. **Embrace `Result`** — The `?` operator makes error handling concise
4. **Trust the Drop** — Let RAII handle cleanup instead of manual `goto cleanup`
5. **Check explicit null** — `if ptr != null` instead of `if (ptr)`
6. **Use match** — More powerful and safer than `switch`
7. **Read the sidebars** — "Coming from C" boxes throughout the guide explain specific differences

A.15 What Stays the Same

Not everything is different! These concepts transfer directly:

- Structs are laid out the same way (C ABI compatible)
- Pointer arithmetic works the same
- Bitwise operations are identical
- AmigaOS structures and constants are the same
- The calling convention matches (d0/d1/a0/a1 for args)
- You can still write inline assembly when needed

Novus is designed to feel familiar to C programmers while adding safety. Your AmigaOS knowledge transfers directly—you're just expressing it in a slightly different syntax with more compile-time guarantees.

Appendix B

Quick Reference

This appendix provides a compact reference for Novus syntax and common patterns.

B.1 Variables and Constants

```
// ... (variable examples - illustrative)
var x = 10
x = 20

// Immutable binding
let y = 42

// Compile-time constant
pub const MAX_SIZE: u32 = 1024

// Type annotation (when needed)
let z: i32 = 100
var buffer = [0u8; 256]    // Size inferred
```

B.2 Primitive Types

Type	Size	Range/Description
`i8`	8-bit	−128 to 127
`i16`	16-bit	−32,768 to 32,767
`i32`	32-bit	±2 billion (default)
`i64`	64-bit	Very large signed
`u8`	8-bit	0 to 255
`u16`	16-bit	0 to 65,535
`u32`	32-bit	0 to 4 billion
`u64`	64-bit	Very large unsigned
`f32`	32-bit	Single-precision float
`f64`	64-bit	Double-precision float
`fixed16`	16-bit	8.8 fixed-point
`fixed32`	32-bit	16.16 fixed-point
`bool`	8-bit	`true` or `false`
`*T`	32-bit	Raw pointer (nullable)
`&T`	32-bit	Reference (non-null)
`&var T`	32-bit	Mutable reference

B.3 Literals

```
// ... (literal examples - illustrative)
42          // i32 (default)
42u32       // u32
42i8        // i8
0xFF        // Hexadecimal (C-style)
$FF         // Hexadecimal (Amiga-style)
%1010       // Binary
1_000_000   // Underscores for readability

// Floats
3.14        // f64 (default)
3.14f32     // f32

// Strings
"Hello"              // String literal
"Line1\nLine2"       // With escape
f"Value: {x}"        // F-string (interpolation)

// Characters
'A'         // u8 (65)
'\n'        // Newline
'\x1B'      // Hex byte (ESC)

// Other
true, false // Booleans
null        // Null pointer
```

B.4 Operators

Op	Meaning	Op	Meaning
`+`	Add	`&&`	Logical AND
`-`	Subtract	`||`	Logical OR
`*`	Multiply	`!`	Logical NOT
`/`	Divide	`&`	Bitwise AND
`%`	Modulo	`|`	Bitwise OR
`==`	Equal	`^`	Bitwise XOR
`!=`	Not equal	`~`	Bitwise NOT
`<`	Less than	`«`	Left shift
`>`	Greater than	`»`	Right shift
`<=`	Less or equal	`..`	Range (exclusive)
`>=`	Greater or equal	`..=`	Range (inclusive)

Compound: `+=`, `-=`, `*=`, `/=`, `%=`, `&=`, `|=`, `^=`, `«=`, `»=`
Increment/Decrement: `++x`, `x++`, `--x`, `x--`

B.5 Control Flow

```
// If-else
if condition {
    // ...
} else if other {
    // ...
```

```
} else {
    // ...
}

// If as expression
let max = if a > b { a } else { b }

// Match
match value {
    1 => handle_one(),
    2 | 3 => handle_two_or_three(),
    4..=10 => handle_range(),
    _ => handle_default(),
}

// While loop
while condition {
    // ...
}

// For loop
for i in 0..10 {
    // i goes 0 to 9
}

for item in collection {
    // iterate items
}

// Loop (infinite)
loop {
    if done { break }
}

// Break and continue
break           // Exit loop
continue        // Next iteration
break value     // Return value from loop
```

B.6 Functions

```
// Basic function
fn add(a: i32, b: i32) -> i32 {
    return a + b
}

// No return value
fn print_hello() {
    println("Hello")
}

// Public function
pub fn api_function() -> i32 {
    return 42
}
```

```
// Generic function
fn swap<T>(a: &var T, b: &var T) {
    let tmp = *a
    *a = *b
    *b = tmp
}

// With trait bounds
fn print_all<T>(items: &[T]) where T: Display {
    for item in items {
        println(item.to_string())
    }
}
```

B.7 Structs and Enums

```
// Struct
struct Point {
    x: i32,
    y: i32,
}

// Create instance
let p = Point { x: 10, y: 20 }

// Impl block
impl Point {
    pub fn new(x: i32, y: i32) -> Point {
        return Point { x: x, y: y }
    }

    pub fn distance(&self, other: &Point) -> f32 {
        // ...
    }
}

// Enum
enum Direction {
    Up,
    Down,
    Left,
    Right,
}

// Enum with data
enum Message {
    Quit,
    Move(i32, i32),
    Write(String),
}

// Match enum
match msg {
    Message::Quit => exit(),
```

```
    Message::Move(x, y) => move_to(x, y),
    Message::Write(s) => println(s),
}
```

B.8 Option and Result

```
// ... (Option/Result patterns - illustrative)
let maybe: Option<i32> = Option::Some(42)
let none: Option<i32> = Option::None

match maybe {
    Option::Some(x) => use_value(x),
    Option::None => handle_missing(),
}

let value = maybe.unwrap_or(0)

// Result
let result: Result<i32, Error> = Result::Ok(42)
let error: Result<i32, Error> = Result::Err(Error::NotFound)

match result {
    Result::Ok(x) => use_value(x),
    Result::Err(e) => handle_error(e),
}

// ? operator (propagate error)
fn process() -> Result<i32, Error> {
    let file = open(path)?   // Returns Err early
    let data = read(&file)?
    Result::Ok(parse(data))
}
```

B.9 Memory and References

```
// Immutable reference
let r: &i32 = &x

// Mutable reference
let r: &var i32 = &var x
*r = 10  // Modify through reference

// Raw pointer
let p: *u8 = (*u8)address

// Dereference
let value = *r
let byte = *p

// Null check
if ptr != null {
    // safe to use
```

```
}

// defer (cleanup)
fn process() {
    let resource = acquire()
    defer { release(resource) }
    // ... use resource
}  // release() called here

// Drop trait (RAII)
impl Drop for MyHandle {
    fn drop(&var self) {
        // Automatic cleanup
    }
}
```

B.10 RAII Patterns

Pattern	Use
`Handle`	Wrap OS resources (`WindowHandle`, `FileHandle`)
`Guard`	Temporary locks (`SemaphoreGuard`, `BlitterGuard`)
`Builder`	Accumulate config (`MUIAppBuilder`)

```
// Handle: wrap resource, auto-cleanup
var window = WindowBuilder::new().build()?
// window closed automatically on drop

// Guard: temporary exclusive access
var lock = semaphore.obtain()
// ReleaseSemaphore() called on drop

// Builder: configure then create
var app = MUIAppBuilder::new()
    .title("App")
    .build()?
```

B.11 Arrays and Collections

```
// Fixed array (size inferred)
let arr = [1, 2, 3]
let zeros = [0; 100]  // 100 zeros
let first = arr[0u32]

// Vec (dynamic array)
var v: Vec<i32> = Vec::new()
v.push(1)
v.push(2)
let len = v.len()
let item = v.get(0u32)  // Option<T>
```

```
// Tuple
let point: (i32, i32) = (10, 20)
let (x, y) = point  // Destructure
```

B.12 Modules and Imports

```
// ... (import examples - illustrative)
from std::collections::vec import Vec
from std::io::file import println, read_file

// Multiple imports
from std::core import Option, Result, Drop

// Use in code
let v: Vec<i32> = Vec::new()
```

B.13 Traits

```
trait Printable {
    fn print(&self)
}
```

Implementation:

```
// ... (with Point and Printable defined above)
impl Printable for Point {
    fn print(&self) {
        println(f"({self.x}, {self.y})")
    }
}
```

Common traits: `Drop` (destructor), `Clone`, `Eq`, `Hash`.

B.14 Common Attributes

```
@test                       // Test function
@test(skip = "reason")      // Skipped test
@benchmark                  // Benchmark function
@inline                     // Hint: inline this function
@noinline                   // Prevent inlining
@packed                     // No struct padding
@export                     // Keep function, don't mangle name
@atomic                     // Wrap in Forbid/Permit
#[derive(Eq, Hash)]         // Auto-implement traits
#[stack_size(65536)]        // Set program stack size
```

B.15 Intrinsics

```
@sizeof(T)      // Size of type in bytes
@zeroed(T)      // Zero-initialized value
```

Note: `alignof()` is only available in inline assembly blocks.

B.16 Compiler Commands

```
# Compile single file
novus compile file.novus -o output

# Build project
novus build

# Run tests
novus test file.novus

# Format code
novus fmt

# With options
novus compile file.novus -o out --release --cpu 68040
```

B.17 C to Novus Quick Reference

C	Novus
`int x = 10;`	`var x = 10`
`const int X = 10;`	`const X: i32 = 10`
`if (ptr)`	`if ptr != null`
`switch/case/break`	`match { pat => }`
`for(i=0;i<n;i++)`	`for i in 0..n`
`NULL`	`null`
`struct S {...};`	`struct S {...}`
`typedef struct`	(not needed)
`ptr->field`	`ptr.field` (auto-deref)
`malloc/free`	Use `Vec`, `MemoryBlock`, or allocator
`sizeof(T)`	`@sizeof(T)`
`errno`	`Result::Err(DosError::...)`

B.18 Escape Sequences

Sequence	Meaning
`\n`	Newline
`\r`	Carriage return
`\t`	Tab
`\0`	Null byte
`\\`	Backslash
`\"`	Double quote
`\'`	Single quote
`\xNN`	Hex byte